

Contents

Robot Framework y Automatización de Pruebas	3
Manual Teórico del Curso	3
Información del curso	3
Descripción general	3
Audiencia	3
Objetivos del curso	4
Prerrequisitos	4
Cómo usar este manual	4
Capítulo 1 — Fundamentos de Automatización y Robot Framework	5
Información general	5
1.1 Automatización de pruebas vs. RPA: diferencias, casos de uso y alcance	5
1.2 Ecosistema Robot Framework: arquitectura, librerías nativas y alcance de RFCP	9
1.3 Instalación y configuración del entorno: Python, RF, IDE y plugins	13
1.4 Estructura de un test case: secciones, keywords integradas y primer suite	16
Resumen del capítulo	20
Referencias bibliográficas	21
Capítulo 2 — Sintaxis y Diseño de Suites de Prueba	22
Información general	22
2.1 Test cases, keywords y bibliotecas: BuiltIn, Collections, OperatingSystem	22
2.2 Variables: escalares, listas y diccionarios	25
2.3 Archivos Resource: separación de keywords y parametrización de datos	28
2.4 Tags, Setup/Teardown y organización jerárquica de suites	31
Resumen del capítulo	34
Referencias bibliográficas	34
Capítulo 3 — Control de Flujo y Validaciones	35
Información general	35
3.1 IF / ELSE IF / ELSE: ejecución condicional en keywords y test cases	35
3.2 FOR: iteración sobre listas y rangos	37
3.3 Asserts y fallas controladas: Should Be, Run Keyword And Expect Error	40
3.4 Continue On Failure, Run Keyword And Continue On Failure y recuperación ...	42
Resumen del capítulo	46
Referencias bibliográficas	46
Capítulo 4 — Behavior-Driven Development (BDD) y Pruebas Orientadas a Negocio .	47
Información general	47
4.1 Introducción a BDD: impacto en la alineación TI-Negocio y telecomunicaciones	47
4.2 Sintaxis Gherkin nativa en RF: Given, When, Then, And, But	49

4.3 Abstracción de capas: escenarios de negocio en lenguaje natural → keywords técnicas	52
4.4 Organización de features y escenarios para flujos comerciales agnósticos	55
Resumen del capítulo	58
Referencias bibliográficas	58
Capítulo 5 — RF Avanzado: Buenas Prácticas, Data-Driven y Extensión con Python .	59
Información general	59
5.1 Diseño de keywords reutilizables: separación por capas y convenciones de nombres	59
5.2 Data-driven testing: DataDriver con CSV/JSON y trazabilidad con tags	62
5.3 Librerías personalizadas en Python: keywords, tipos, documentación y decoradores	65
5.4 Empaquetado, versionado y repositorios compartidos de librerías RF	69
Resumen del capítulo	71
Referencias bibliográficas	72
Capítulo 6 — Automatización Web con SeleniumLibrary	73
Información general	73
6.1 Configuración de navegador, Selenium Manager, sesiones y opciones	73
6.2 Localizadores CSS y XPath: selección robusta e interacción con formularios .	75
6.3 Sincronización: Wait Until, timeouts y estabilidad	78
6.4 Evidencias: screenshots, alertas, frames e iframes	82
6.5 Patrón Page Object en RF: capa de keywords por página y mantenibilidad . . .	85
Resumen del capítulo	88
Referencias bibliográficas	88
Capítulo 7 — Automatización de APIs con RequestsLibrary	89
Información general	89
7.1 REST/HTTP: métodos, códigos de estado, headers y payloads	89
7.2 Autenticación: Bearer tokens, sesiones persistentes y datos dinámicos	92
7.3 Validación JSON: sintaxis extendida, aserciones de esquema y contract testing básico	94
7.4 Diseño de suites API: estructura smoke vs. regresión y organización por endpoint	96
Resumen del capítulo	99
Referencias bibliográficas	100
Capítulo 8 — Robot Framework Aplicado a RPA	101
Información general	101
8.1 Automatización de archivos: CSV, Excel, PDF y gestión de carpetas	101
8.2 Orquestación de procesos: entrada/salida, parametrización y variables de proceso	104
8.3 Integración E2E: web + API + archivos en un único flujo RPA	106
8.4 Manejo de errores en RPA: retries, timeouts y control de fallas	108
8.5 Logging, trazabilidad y estandarización en RPA	111

Resumen del capítulo	113
Referencias bibliográficas	113
Capítulo 9 — Ejecución Avanzada, Reporting y Preparación RFCP	115
Información general	115
9.1 CLI avanzado: variables, filtros y paretags	115
9.2 Reporting: report.html, log.html y rebot	117
9.3 Integración CI/CD conceptual: pipelines y quality gates	120
Resumen del capítulo	122
Referencias bibliográficas	123



Robot Framework y Automatización de Pruebas

Manual Teórico del Curso

Preparación para la certificación **RFCP** (Robot Framework Certified Professional)

Información del curso

Campo	Detalle
Versión de la tecnología	Robot Framework 7.x · Python 3.12
Duración total	35 horas (9 sesiones)
Distribución	40% teoría (14 h) · 60% práctica (21 h)

Descripción general

Curso práctico de automatización de pruebas y RPA con Robot Framework. Cubre desde fundamentos hasta técnicas avanzadas: automatización web, API, RPA, BDD, extensión con Python y preparación para la certificación RFCP.

Audiencia

Ingenieros de QA, analistas de pruebas, desarrolladores y profesionales de automatización con conocimientos básicos de programación. Equipos de TI en el sector teleco-

municaciones que buscan adoptar automatización de pruebas y RPA con estándares de calidad.

Objetivos del curso

Al finalizar el curso, el participante será capaz de:

- Diseñar, implementar y mantener suites de prueba automatizadas con Robot Framework.
- Aplicar patrones BDD orientados al negocio.
- Crear librerías personalizadas en Python.
- Automatizar procesos RPA de extremo a extremo.
- Ejecutar pipelines de prueba en entornos CI/CD.

Prerrequisitos

Conocimientos básicos de programación (Python recomendado). Manejo básico de línea de comandos. Familiaridad con conceptos de testing de software. Requerimientos tecnológicos: Python 3.10+, VS Code, acceso a internet.

Cómo usar este manual

Este manual cubre la **teoría** de cada capítulo — los conceptos, la sintaxis y el razonamiento detrás de cada técnica. La **práctica guiada** (paso a paso, con código ejecutable) vive en la Guía del Estudiante (`robfram-guide`), y las diapositivas de cada sesión en `robfram-ppts`. Los tres materiales están alineados capítulo por capítulo.

Capítulo 1 — Fundamentos de Automatización y Robot Framework

Información general

Este capítulo presenta el ecosistema de Robot Framework y resuelve la primera confusión que todo estudiante de automatización enfrenta: ¿qué es exactamente lo que vamos a construir, pruebas o robots de proceso? También dejamos listo el entorno de trabajo y entendemos, en profundidad, cómo está construido un archivo `.robot` por dentro — no solo su sintaxis superficial, sino el modelo de ejecución que hay detrás.

Lecciones de este capítulo:

- 1.1 — Automatización de pruebas vs. RPA: diferencias, casos de uso y alcance
 - 1.2 — Ecosistema Robot Framework: arquitectura, librerías nativas y alcance de RFCP
 - 1.3 — Instalación y configuración del entorno: Python, RF, IDE y plugins
 - 1.4 — Estructura de un test case: secciones, keywords integradas y primer suite
-

1.1 Automatización de pruebas vs. RPA: diferencias, casos de uso y alcance

Objetivos de la lección

Al terminar esta lección serás capaz de:

- Distinguir el propósito de la automatización de pruebas del propósito de la RPA.
- Identificar a qué dominio pertenece un escenario concreto.
- Explicar por qué Robot Framework cubre ambos dominios con una sola sintaxis.

¿Por qué importa?

Antes de escribir una sola línea de código, necesitas saber **qué tipo de problema** estás resolviendo. Un equipo que confunde “verificar calidad” con “ejecutar un proceso” termina eligiendo la arquitectura equivocada: diseña asserts donde necesitaba transacciones, o diseña reintentos infinitos donde necesitaba un veredicto claro de pass/fail. Esta distinción condiciona literalmente cada decisión de diseño del resto del curso.

Conceptos clave

El propósito de la automatización de pruebas

La **automatización de pruebas** existe para **verificar la calidad de un sistema que ya fue construido por alguien más** (un equipo de desarrollo). No crea valor de negocio directamente — su valor es la *confianza*: saber que un cambio de código no rompió algo

que funcionaba. Su producto es un veredicto binario: el sistema se comportó como se esperaba (PASS) o no (FAIL). La audiencia de ese veredicto es el equipo de desarrollo y QA, y normalmente se ejecuta **cada vez que el código cambia**, dentro de un pipeline de integración continua. El ciclo de vida de un test es corto: se escribe, se ejecuta miles de veces, y eventualmente se actualiza o se elimina cuando el comportamiento que valida cambia intencionalmente.

El propósito de la RPA

La **RPA** (*Robotic Process Automation*), en cambio, existe para **ejecutar un proceso de negocio de extremo a extremo**, igual que lo haría una persona siguiendo un procedimiento repetitivo: descargar un archivo, leerlo, transformarlo, subir el resultado a un sistema. Su producto no es un veredicto, es un **resultado operativo** — una factura procesada, un registro creado, un correo enviado. La audiencia es el área de negocio que depende de ese proceso, y se ejecuta según un calendario (todos los días a las 2am) o un evento disparador (llega un correo, se deposita un archivo en una carpeta). El ciclo de vida de un robot RPA suele ser más largo y más crítico operativamente: si falla, alguien en el negocio no recibe su factura procesada hoy.

La analogía que distingue ambos roles

Una analogía útil: la automatización de pruebas es un **inspector de calidad** que revisa cada pieza que sale de la línea de producción, comparándola contra una especificación; la RPA es el **operario** que ensambla esas piezas siguiendo un procedimiento. Ambos trabajan en la misma fábrica, pero confundir sus roles te lleva a elegir la herramienta equivocada para el problema que realmente tienes: pedirle al inspector que ensamble piezas (un test que “arregla” datos en producción) es tan problemático como pedirle al operario que decida si la pieza cumple con la norma (un robot RPA sin ninguna validación de los datos que procesa).

Tabla comparativa completa

Dimensión	Automatización de pruebas	RPA
Propósito	Verificar calidad	Ejecutar un proceso de negocio
Producto	Reporte PASS/FAIL	Resultado operativo (archivo, registro, notificación)
Audiencia	QA / Desarrollo	Áreas de negocio / operaciones
Disparador típico	Cambio de código (CI/CD)	Horario o evento externo
Ciclo de vida	Corto, se actualiza con el código	Largo, crítico operativamente
Falla típica	Detecta un defecto antes de producción	Detiene un proceso de negocio en curso
Ejemplo	"El botón Pagar muestra el mensaje correcto"	"Descargar facturas PDF del correo y cargarlas al ERP"

Cómo elegir el enfoque correcto

La pregunta clave es: “¿necesito **verificar** un sistema, o **ejecutar** un proceso?”. Si el resultado que esperas es un reporte de calidad que alguien de tecnología va a leer, es automatización de pruebas. Si el resultado es un artefacto operativo que el negocio necesita (un archivo, un registro, una notificación), es RPA. Una señal práctica adicional: si tu primer instinto es escribir `Should Be Equal` o `Should Contain`, probablemente estás en el dominio de pruebas; si tu primer instinto es escribir “leer archivo, transformar dato, guardar resultado”, estás en RPA.

Ejemplo comentado

*** Test Cases ***

```
# Automatización de pruebas: el resultado se VERIFICA con un assert.
# Si la verificación falla, el TEST falla – eso es exactamente lo que
# queremos que pase, porque significa que detectamos un defecto.
```

Verificar Que El Botón Pagar Muestra El Mensaje Correcto

```
Click Button    Pagar
Wait Until Element Is Visible    css:#confirmacion
Element Should Contain    css:#confirmacion    Pago procesado correctamente
```

*** Tasks ***

```
# RPA: no hay un "assert" de negocio – el resultado es la ACCIÓN.
# Robot Framework usa la sección *** Tasks *** (en vez de *** Test Cases ***)
# para procesos RPA; el motor de ejecución es el mismo, pero el vocabulario
```

de la sección comunica la intención: aquí no se "prueba" nada, se "hace".

Procesar Facturas Recibidas Por Correo

```
@{adjuntos}= Descargar Adjuntos Del Correo bandeja=facturas@empresa.com
FOR  ${adjunto} IN  @{adjuntos}
    ${datos}= Extraer Datos De Factura  ${adjunto}
    Cargar Factura En ERP  ${datos}
END
```

¿Qué cambia realmente entre ambos bloques? La sección (Test Cases vs. Tasks) y el vocabulario de las keywords. El motor de Robot Framework procesa ambas de forma casi idéntica — la diferencia es semántica, comunica al equipo qué tipo de unidad de trabajo es cada una.

Tabla de referencia rápida

Sección de Robot Framework	Dominio típico	Qué representa cada unidad
*** Test Cases ***	Automatización de pruebas	Una verificación con resultado PASS/FAIL
*** Tasks ***	RPA	Un paso de un proceso de negocio

Errores comunes

- **Escribir un “test” RPA con un assert al final, pero sin ninguna acción real de negocio antes** — confunde verificación con ejecución; si el objetivo es procesar facturas, el “éxito” debe ser que la factura quedó procesada, no solo que una condición lógica se cumplió.
- **Tratar un fallo de RPA como un fallo de test** — en RPA, un fallo a media ejecución puede dejar el proceso de negocio en un estado parcial (por ejemplo, una factura descargada pero no cargada al ERP); la recuperación de ese estado parcial es un problema que no existe en automatización de pruebas, donde cada test es independiente.
- **Usar *** Test Cases *** para todo, incluso procesos RPA** — funciona técnicamente, pero pierde la comunicación semántica que *** Tasks *** le da al equipo.

Puntos clave

- Automatización de pruebas → verificar calidad → reporte PASS/FAIL → ciclo de vida corto, atado al código.
- RPA → ejecutar procesos de negocio → resultado operativo → ciclo de vida largo, atado al calendario/eventos del negocio.
- Robot Framework cubre ambos dominios con una sola sintaxis, distinguiendo *** Test Cases *** de *** Tasks ***.

Autoevaluación

1. Una empresa de telecomunicaciones necesita verificar que el portal de autogestión de clientes calcula correctamente el cargo por exceso de datos. ¿Es esto automatización de pruebas o RPA?
2. La misma empresa necesita, todas las noches a las 11pm, descargar el archivo de consumos del día, calcular los cargos, y subir el resultado al sistema de facturación. ¿Es esto automatización de pruebas o RPA?
3. ¿Qué sección de Robot Framework comunica semánticamente que una unidad de trabajo es un paso de proceso de negocio, no una verificación?
4. Verdadero o falso: en RPA, un fallo a mitad de ejecución no tiene impacto fuera de la prueba misma, igual que en automatización de pruebas.

Respuestas: 1. Automatización de pruebas (se está *verificando* un cálculo). 2. RPA (se está *ejecutando* un proceso de negocio recurrente con resultado operativo). 3. `*** Tasks ***`. 4. Falso — un fallo de RPA a mitad de ejecución puede dejar un proceso de negocio en estado parcial, con impacto operativo real.

1.2 Ecosistema Robot Framework: arquitectura, librerías nativas y alcance de RFCP

Objetivos de la lección

- Describir la arquitectura en capas de Robot Framework.
- Identificar las librerías nativas y su propósito.
- Explicar qué evalúa (y qué no evalúa) la certificación RFCP.

¿Por qué importa?

Reducir Robot Framework a “una herramienta de automatización de pruebas” subestima lo que realmente es, y lleva a decisiones de arquitectura equivocadas: por ejemplo, intentar resolver todo dentro del núcleo cuando la solución correcta es escribir una librería nueva, o no entender por qué cierto comportamiento de una librería de comunidad (como `SeleniumLibrary`) no aplica a otra (como `RequestsLibrary`). Entender la arquitectura en capas te da el modelo mental correcto para diagnosticar problemas y diseñar extensiones.

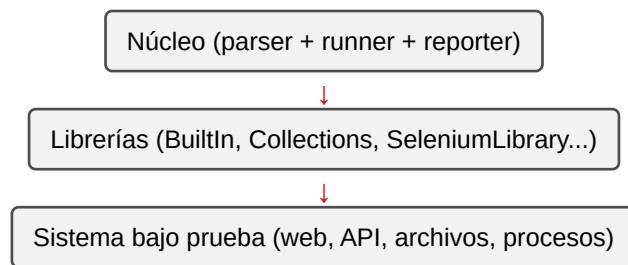
Conceptos clave

La arquitectura en cuatro capas

Robot Framework está diseñado con **separación de responsabilidades**, no es un monolito cerrado:

Datos de prueba (.robot / .resource)





1. **Datos de prueba:** los archivos `.robot` y `.resource` que tú escribes — texto plano, sin lógica de ejecución propia.
2. **Núcleo:** parsea esos archivos, construye un árbol de ejecución, ejecuta cada keyword en orden, y genera los reportes. El núcleo **no sabe nada** de navegadores ni de HTTP — esa lógica vive exclusivamente en las librerías. Esta separación es la razón por la que Robot Framework puede automatizar dominios tan distintos (web, API, archivos, bases de datos) sin cambiar su motor.
3. **Librerías:** traducen keywords en acciones concretas (clic en un botón, petición HTTP, lectura de un Excel). Pueden ser nativas (incluidas con Robot Framework), de comunidad (instaladas con `pip`), o propias (las que tú escribes, como viste — o verás — en la Sesión 5).
4. **Sistema bajo prueba (SUT):** la aplicación, API o proceso que realmente se está automatizando — fuera del control de Robot Framework, solo se interactúa con él a través de las librerías.

Cómo el núcleo resuelve una keyword

Cuando escribes una línea como `Should Be Equal    ${a}    ${b}`, el núcleo busca esa keyword en este orden: primero en el archivo actual (keywords definidas localmente), luego en los `Resource` importados, y finalmente en las `Library` importadas, en el orden en que fueron declaradas en `Settings`. Si dos librerías importadas exponen una keyword con el mismo nombre, Robot Framework lanza un error de ambigüedad **a menos que** se use el prefijo de librería (`BuiltIn.Should Be Equal`) para desambiguar — un detalle que rara vez se nota hasta que un proyecto crece e importa muchas librerías.

Librerías nativas: referencia completa

Las librerías nativas vienen incluidas con la instalación de Robot Framework — no requieren `pip install` adicional:

Librería	Importación	Para qué sirve
BuiltIn	Automática, siempre disponible	Aserciones, logging, control de flujo, conversiones de tipo
Collections	Library Collections	Crear y verificar listas y diccionarios
OperatingSystem	Library OperatingSystem	Archivos, carpetas, variables de entorno, rutas
String	Library String	Manipulación de texto: split, strip, formato
DateTime	Library DateTime	Cálculos y formato de fechas/horas
Process	Library Process	Ejecutar procesos externos del sistema operativo
Telnet	Library Telnet	Conexiones Telnet (uso especializado)
XML	Library XML	Lectura y manipulación de documentos XML

BuiltIn es la única que **no** requiere una línea Library en Settings — todas sus keywords (Should Be Equal, Log, Convert To Integer...) están siempre disponibles en cualquier suite, sin excepción. Las demás bibliotecas, nativas o no, sí deben declararse explícitamente.

El alcance exacto de la certificación RFCP

RFCP (Robot Framework Certified Professional) es la certificación oficial emitida por la **Robot Framework Foundation**. Evalúa el dominio del **lenguaje y la plataforma central** — sintaxis, estructura de archivos, variables, control de flujo, manejo de errores, librerías nativas, y ejecución por CLI. Esto es importante para tu preparación: **NO** incluye conocimiento específico de SeleniumLibrary, RequestsLibrary, DataDriver ni ninguna librería de comunidad — esas son evaluadas, si acaso, en certificaciones o evaluaciones separadas y específicas de cada herramienta. La certificación te valida como alguien que domina Robot Framework **como lenguaje**, independientemente de para qué lo use después: web, API, RPA, bases de datos, o cualquier otro dominio que una librería pueda cubrir en el futuro.

Ejemplo comentado

```
*** Settings ***
# BuiltIn no aparece aquí – está disponible sin declararse.
Library    Collections
Library    OperatingSystem
```

*** Test Cases ***

Demostrar Tres Capas De Librerías En Una Suite

```
# BuiltIn (núcleo del lenguaje): siempre disponible
Log      Iniciando demostración

# Collections (nativa, declarada explícitamente)
${lista}=    Create List    uno    dos    tres
Should Contain    ${lista}    dos

# OperatingSystem (nativa, declarada explícitamente)
${existe}=    Run Keyword And Return Status    File Should Exist    /tmp
Should Be True    ${existe}
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué evalúa RFCP?	El lenguaje y la plataforma central de Robot Framework
¿Incluye SeleniumLibrary?	No
¿Qué librería nunca se declara con Library?	BuiltIn
¿Qué capa traduce keywords en acciones concretas?	Las librerías

Errores comunes

- **Asumir que toda keyword requiere Library en Settings** — BuiltIn es la excepción, y es justo la que más se usa al empezar.
- **Estudiar SeleniumLibrary o RequestsLibrary pensando que entran en el examen RFCP** — son librerías de comunidad, fuera del alcance de esa certificación específica.
- **No declarar una librería nativa pensando que “es nativa, no debería necesitar declaración”** — solo BuiltIn tiene ese privilegio; Collections, OperatingSystem, etc. Sí necesitan Library <nombre> aunque vengan incluidas en la instalación.

Puntos clave

- Cuatro capas: datos de prueba, núcleo, librerías, sistema bajo prueba — el núcleo no contiene lógica de dominio.
- El núcleo resuelve keywords en orden: local → Resource → Library, en el orden de declaración.
- RFCP evalúa el lenguaje central, no librerías de dominio específico como Selenium o Requests.

Autoevaluación

1. ¿Cuál de las siguientes librerías NO necesita una línea `Library` en `Settings` para usarse?
a) `Collections` b) `OperatingSystem` c) `BuiltIn` d) `String`
2. ¿La certificación RFCEP evalúa el conocimiento de `SeleniumLibrary`?
3. Si dos librerías importadas exponen una keyword con el mismo nombre, ¿qué mecanismo permite desambiguar cuál usar?
4. ¿Qué capa de la arquitectura de Robot Framework “no sabe nada” de navegadores ni HTTP?

Respuestas: 1. c) `BuiltIn`. 2. No — RFCEP evalúa el lenguaje central, no librerías de dominio específico. 3. El prefijo de librería, por ejemplo `BuiltIn.Should Be Equal`. 4. El núcleo (parser + runner + reporter).

1.3 Instalación y configuración del entorno: Python, RF, IDE y plugins

Objetivos de la lección

- Explicar por qué el orden de instalación importa.
- Describir el rol de cada componente del entorno (Python, pip, Robot Framework, VS Code).
- Aplicar correctamente el uso de entornos virtuales (`venv`).

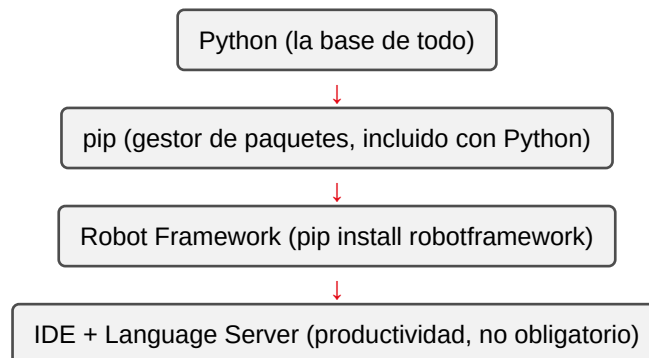
¿Por qué importa?

Un entorno mal instalado es la causa más frecuente de errores confusos al empezar un curso de automatización — y consume horas de diagnóstico que se evitan siguiendo un orden estricto y entendiendo **por qué** ese orden importa, no solo memorizándolo.

Conceptos clave

Las cuatro capas del entorno, en orden estricto

El entorno se instala en cuatro capas, cada una dependiente de la anterior:



Python es la base: Robot Framework está escrito en Python y se ejecuta sobre su intérprete. **pip** es el gestor de paquetes que viene incluido con cualquier instalación moderna de Python; es lo que usarás para instalar Robot Framework y todas las librerías de comunidad del curso. **Robot Framework** se instala con `pip install robotframework`, igual que cualquier paquete de Python. El **IDE** (VS Code + extensión Language Server) potencia la productividad — autocompletado, resaltado de sintaxis, navegación a definiciones — pero **no es estrictamente necesario** para ejecutar pruebas; Robot Framework funciona igual desde cualquier editor de texto plano o incluso sin editor, escribiendo el archivo `.robot` por otro medio.

Por qué el entorno virtual no es opcional

Un punto que se repite durante todo el curso: **Robot Framework nunca se instala de forma global**. Siempre dentro de un entorno virtual (`venv`) propio del proyecto. Esto evita que dos proyectos con versiones distintas de Robot Framework (o de sus librerías) entren en conflicto en la misma máquina — un problema real cuando, por ejemplo, el Proyecto A necesita `robotframework-seleniumlibrary==6.x` (compatible con Selenium 4) y el Proyecto B todavía depende de una versión anterior. Sin entornos virtuales, instalar la versión que necesita el Proyecto B rompería al Proyecto A en la misma máquina.

Un entorno virtual es, en esencia, una copia aislada del intérprete de Python con su propio directorio de paquetes instalados — `pip install` dentro de un `venv` activo nunca afecta paquetes instalados fuera de él.

El Language Server y su rol real

La extensión recomendada para VS Code es **Robot Framework Language Server** (`robocorp.robotframework-lsp`), que aporta resaltado de sintaxis, autocompletado de keywords (incluyendo las de librerías que importaste) y navegación a la definición de una keyword con `Ctrl+click`. Internamente, el Language Server analiza tu entorno virtual activo para saber qué librerías están instaladas y ofrecerte autocompletado preciso — por eso es importante que VS Code tenga seleccionado el intérprete de Python correcto (el del `venv` del proyecto), no el del sistema.

Ejemplo comentado

```
# 1. Verificar Python (la base)
python3 --version
# Salida esperada: Python 3.10.x o superior

# 2. Crear el entorno virtual del proyecto (aislado, nunca global)
python3 -m venv venv

# 3. Activarlo — el prompt debe mostrar (venv)
source venv/bin/activate      # macOS / Linux
# venv\Scripts\activate.bat   # Windows (cmd)

# 4. Con el venv activo, pip instala SOL0 dentro de ese entorno
```

```
pip install --upgrade pip
pip install robotframework
```

```
# 5. Verificar que la instalación quedó dentro del venv, no global
pip show robotframework
# El campo "Location" debe apuntar dentro de la carpeta venv/
```

Tabla de referencia rápida

Comando	Qué hace
<code>python3 -m venv venv</code>	Crea un entorno virtual aislado en la carpeta <code>venv/</code>
<code>source venv/bin/activate</code>	Activa el entorno virtual (Linux/macOS)
<code>pip install robotframework</code>	Instala Robot Framework dentro del venv activo
<code>pip show robotframework</code>	Confirma dónde quedó instalado un paquete
<code>deactivate</code>	Sale del entorno virtual activo

Errores comunes

- **Instalar Robot Framework sin activar el venv primero** — queda instalado globalmente, y los problemas de conflicto de versiones entre proyectos aparecen meses después, cuando ya es difícil rastrear la causa.
- **Olvidar verificar qué intérprete tiene seleccionado VS Code** — si el Language Server usa el Python global en vez del venv del proyecto, el autocompletado no reconocerá las librerías instaladas en el venv.
- **Confundir “crear” el venv con “activar” el venv** — `python3 -m venv venv` solo crea la carpeta; sin `source venv/bin/activate` (o su equivalente en Windows), `pip install` seguirá afectando el entorno global.

Puntos clave

- Orden estricto: Python → pip → Robot Framework → IDE — cada capa depende de la anterior.
- Robot Framework se instala siempre dentro de un venv, nunca de forma global, para evitar conflictos de versiones entre proyectos.
- El IDE mejora la productividad, pero no es requisito técnico para ejecutar pruebas.

Autoevaluación

1. ¿Qué comando crea (sin activar) un entorno virtual llamado `venv`?
2. Si instalas `robotframework` sin haber activado el venv, ¿dónde queda instalado?
3. ¿Es obligatorio usar VS Code para ejecutar Robot Framework?
4. ¿Qué problema concreto evita usar un entorno virtual por proyecto?

Respuestas: 1. `python3 -m venv venv`. 2. Globalmente, en el Python del sistema (no en el venv). 3. No, es opcional — mejora productividad pero no es un requisito técnico. 4. Conflictos de versión entre proyectos que necesitan versiones distintas de Robot Framework o de sus librerías en la misma máquina.

1.4 Estructura de un test case: secciones, keywords integradas y primer suite

Objetivos de la lección

- Identificar las cuatro secciones de un archivo `.robot`.
- Explicar la responsabilidad de cada sección.
- Reconocer keywords esenciales de la librería `BuiltIn`.
- Interpretar los tres artefactos de reporte que genera toda ejecución.

¿Por qué importa?

Al escribir tu primera prueba automatizada, el mayor obstáculo no suele ser la lógica de negocio, sino entender **dónde va cada cosa**. Sin un modelo mental claro de las cuatro secciones, es fácil mezclar responsabilidades (poner lógica de configuración dentro de un test case, o variables específicas de un solo test en la sección global) — algo que se vuelve costoso de corregir cuando la suite ya creció.

Conceptos clave

Las cuatro secciones y su responsabilidad

Robot Framework organiza un archivo `.robot` en cuatro secciones, cada una marcada con tres asteriscos:

Sección	Responsabilidad	¿Obligatoria?
*** Settings ***	Metadatos de la suite: qué librerías importa, qué Resource usa, documentación, Setup/Tear-down a nivel de suite	No
*** Variables ***	Valores reutilizables: escalares (<code>\${var}</code>), listas (<code>@{lista}</code>), diccionarios (<code>&{dict}</code>)	No
*** Test Cases ***	Los casos de prueba — cada uno con un nombre y una secuencia de keywords	Sí (o *** Tasks *** en RPA)
*** Keywords ***	Keywords propias del proyecto, reutilizables entre test cases de la misma suite	No

Una suite mínima solo necesita `*** Test Cases ***` (o `*** Tasks ***`) — las otras tres secciones son opcionales y se agregan según la necesidad real del proyecto, no por costumbre.

Anatomía interna de un test case

Cada test case dentro de `*** Test Cases ***` es, técnicamente, una secuencia ordenada de llamadas a keywords. El nombre del test case es **texto libre** — no tiene restricciones de sintaxis especiales más allá de la indentación (las keywords que lo componen van indentadas debajo del nombre). Internamente, cuando ejecutas `robot archivo.robot`, el runner crea un objeto de ejecución por cada test case, ejecuta sus keywords en orden, y captura el resultado (PASS si ninguna keyword falló, FAIL si alguna lanzó una excepción no controlada).

Las keywords integradas más usadas de BuiltIn

Como viste en la lección 1.2, `BuiltIn` se importa automáticamente. Sus keywords más usadas al empezar:

Keyword	Para qué sirve
Log	Escribe un mensaje en el reporte (nivel INFO por defecto)
Should Be Equal	Aserción de igualdad exacta entre dos valores
Should Be True	Aserción de que una expresión es verdadera
Should Contain	Verifica que un valor existe dentro de una colección o substring
Set Variable	Crea una variable local en tiempo de ejecución, dentro de un test o keyword
Convert To Integer	Convierte un valor (típicamente texto) a entero

Los tres artefactos de toda ejecución

Cuando ejecutas una suite con el comando `robot`, se generan **siempre** tres artefactos, sin excepción:

output.xml • Datos en bruto de la ejecución

- La fuente de verdad
- Lo que consumen herramientas externas (CI/CD)

log.html / report.html • `log.html`: detalle de cada keyword

- `report.html`: resumen ejecutivo
- Ambos se derivan de `output.xml`

`output.xml` contiene el detalle completo de la ejecución — cada keyword, sus argumentos, el resultado, los mensajes de Log — y es la **fuentes de verdad**: si lo conservas, puedes regenerar `log.html` y `report.html` en cualquier momento con la herramienta `rebot` (que verás formalmente en la Sesión 9), incluso combinando varias ejecuciones en un solo reporte. `log.html` muestra el detalle de cada keyword ejecutada, útil para diagnosticar un fallo paso a paso. `report.html` es el resumen ejecutivo, con estadísticas globales y por tag — el que normalmente revisa un líder de equipo o un stakeholder no técnico.

Ejemplo comentado

```
*** Settings ***
# La sección Settings define metadatos de la suite y las librerías que
# importa. BuiltIn es nativa y NO requiere importación explícita.
Documentation    Suite de demostración – cuatro secciones principales.

*** Variables ***
# Variables escalares (${...}): un solo valor cada una.
```

```

${NOMBRE_CURSO}      Robot Framework 7
${VERSION_RF}        7

# Variable de lista (@{...}): varios valores en orden.
@{DOMINIOS_RF}       Automatización de Pruebas    RPA    Automatización Web

```

*** Test Cases ***

```

# Cada bloque con nombre en esta sección es un caso de prueba
# independiente. La indentación usa 4 espacios o un tab.
TC-01 Verificar que las variables de suite están definidas correctamente
    [Documentation]    Valida que las variables declaradas en Variables
    ...               tienen los valores esperados, usando Should Be Equal.
    Log      Iniciando verificación de variables de suite
    Should Be Equal As Integers    ${VERSION_RF}    7
    Log      Verificación completada exitosamente

TC-02 Verificar que Robot Framework abarca múltiples dominios
    [Documentation]    Recorre @{DOMINIOS_RF} y confirma su contenido.
    Should Contain    ${DOMINIOS_RF}    RPA

```

*** Keywords ***

```

# La sección Keywords define keywords reutilizables, propias de esta suite.
Verificar Dominio En Lista
    [Documentation]    Verifica que un dominio dado existe en @{DOMINIOS_RF}.
    [Arguments]    ${dominio}
    Should Contain    ${DOMINIOS_RF}    ${dominio}

```

Al ejecutar `robot --outputdir reports tests/primer_suite.robot`, la consola muestra 2 tests, 2 passed, 0 failed, y la carpeta `reports/` queda con `output.xml`, `log.html` y `report.html` — los tres artefactos descritos arriba.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué sección es obligatoria en una suite de pruebas?	Test Cases (o Tasks en RPA)
¿Qué archivo es la “fuente de verdad” de una ejecución?	<code>output.xml</code>
¿Qué keyword convierte texto a entero?	<code>Convert To Integer</code>
¿Con cuántos espacios mínimos se indenta el cuerpo de un test case?	4 espacios o un tab

Errores comunes

- **Mezclar tabs y espacios en la indentación de un mismo archivo** — Robot Framework acepta ambos, pero mezclarlos dentro del mismo bloque produce errores de parseo confusos.
- **Definir una variable que solo necesita un test dentro de `*** Variables ***`** (alcance de toda la suite) en vez de usar `Set Variable` dentro del test case que la necesita — ensucia el alcance global innecesariamente.
- **Borrar reports/ pensando que se perderá información importante** — es completamente seguro, esos tres archivos se regeneran en cada ejecución; lo único irremplazable es el código fuente `.robot`.

Puntos clave

- Cuatro secciones: `Settings`, `Variables`, `Test Cases`, `Keywords` — solo `Test Cases (O Tasks)` es obligatoria.
- `BuiltIn` se importa automáticamente y aporta las keywords esenciales (`Log`, `Should Be Equal`, `Set Variable`, `Convert To Integer`).
- Toda ejecución genera `output.xml` (fuente de verdad), `log.html` (detalle) y `report.html` (resumen ejecutivo).

Autoevaluación

1. ¿Qué sección de un archivo `.robot` es la única obligatoria?
2. Si pierdes `log.html` y `report.html` pero conservas `output.xml`, ¿puedes recuperarlos? ¿Con qué herramienta?
3. ¿Qué keyword de `BuiltIn` usarías para verificar que un valor exacto coincide con otro?
4. Verdadero o falso: una suite `.robot` siempre debe tener las cuatro secciones.

Respuestas: 1. `Test Cases (O Tasks)` en contexto RPA). 2. Sí, con `rebot` sobre el `output.xml` conservado. 3. `Should Be Equal`. 4. Falso — solo `Test Cases/Tasks` es obligatoria; las otras tres son opcionales según la necesidad del proyecto.

Resumen del capítulo

La automatización de pruebas verifica calidad (producto: reporte PASS/FAIL, ciclo corto); la RPA ejecuta procesos de negocio (producto: resultado operativo, ciclo largo) — Robot Framework cubre ambos con la misma sintaxis, distinguiendo `Test Cases` de `Tasks`. Su arquitectura se organiza en cuatro capas (datos de prueba, núcleo, librerías, sistema bajo prueba), y la certificación RFCEP evalúa el lenguaje central, no librerías de dominio. El entorno se instala en orden estricto (Python → pip → Robot Framework → IDE) siempre dentro de un `venv`, para evitar conflictos de versiones entre proyectos. Un archivo `.robot` tiene cuatro secciones, de las cuales solo `Test Cases/Tasks` es obligatoria, y toda ejecución genera tres artefactos: `output.xml` (fuente de verdad), `log.html` (detalle) y `report.html` (resumen).

Referencias bibliográficas

- Documentación oficial de Robot Framework: <https://robotframework.org/>
- Robot Framework User Guide: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>
- Referencia de keywords BuiltIn: <https://robotframework.org/robotframework/latest/libraries/BuiltIn.html>
- Robot Framework Foundation (certificación RFCP): <https://robotframework.org/foundation/>
- Diferencias entre automatización de pruebas y RPA: <https://www.guru99.com/rpa-vs-test-automation.html>

Capítulo 2 — Sintaxis y Diseño de Suites de Prueba

Información general

Este capítulo profundiza en la sintaxis que sostiene cualquier suite de prueba seria: cómo se relacionan test cases, keywords y librerías; los tres tipos de variables y su precedencia; cómo separar keywords en archivos reutilizables; y cómo organizar una suite grande con tags y Setup/Teardown. Cada lección incluye ejemplos de código completos — no solo reglas de sintaxis aisladas.

Lecciones de este capítulo:

- 2.1 — Test cases, keywords y bibliotecas: BuiltIn, Collections, OperatingSystem
 - 2.2 — Variables: escalares, listas y diccionarios
 - 2.3 — Archivos Resource: separación de keywords y parametrización de datos
 - 2.4 — Tags, Setup/Teardown y organización jerárquica de suites
-

2.1 Test cases, keywords y bibliotecas: BuiltIn, Collections, OperatingSystem

Objetivos de la lección

- Describir la relación entre test case, keyword y biblioteca.
- Diferenciar las responsabilidades de BuiltIn, Collections y OperatingSystem.
- Aplicar keywords de las tres bibliotecas en una sola suite.

¿Por qué importa?

Cuando una suite crece más allá de unos pocos test cases, la diferencia entre “qué se prueba” y “cómo se hace” deja de ser una sutileza estilística y se convierte en la diferencia entre una suite mantenible y una que nadie quiere tocar. Entender esta jerarquía desde el principio evita que termines con test cases de 40 líneas que mezclan negocio y detalles técnicos.

Conceptos clave

La jerarquía de tres niveles

En Robot Framework, **test case**, **keyword** y **biblioteca** forman una jerarquía de tres niveles, cada uno con un nivel de abstracción distinto:

- Un **test case** es una unidad de prueba con nombre, legible por cualquier persona del equipo (incluso sin conocimientos técnicos) — describe **el qué**.

- Una **keyword** es una acción reutilizable invocada desde uno o más test cases — el “verbo” de la prueba, describe **el cómo**, a un nivel todavía legible.
- Una **biblioteca** es el contenedor que agrupa keywords relacionadas y las expone al motor de ejecución — es el “proveedor” de las keywords.

Esta jerarquía existe para que un test case **describa el qué**, no **el cómo**: leer `Iniciar Sesión Como Administrador` te dice la intención sin obligarte a leer la implementación (que puede vivir en una keyword propia, posiblemente en varias líneas de Selenium o de llamadas API).

BuiltIn: la biblioteca que siempre está ahí

BuiltIn es la única biblioteca que no requiere una línea `Library` — sus keywords están disponibles sin que el autor de la suite tenga que pensar en importarlas. Categorías principales: aserciones (`Should Be *`), conversión de tipos (`Convert To Integer`, `Convert To String`), control de flujo (`Run Keyword If`, ya vista parcialmente), y logging (`Log`, `Log To Console`).

Collections: operar sobre listas y diccionarios

Collections agrega keywords para trabajar con las dos estructuras de datos compuestas de Robot Framework: listas y diccionarios. Las más usadas: `Create List`, `Create Dictionary`, `Append To List`, `Get From Dictionary`, `Should Contain` (también funciona sobre listas), `Dictionary Should Contain Key`. Sin esta biblioteca, modelar datos compuestos en una suite sería mucho más limitado.

OperatingSystem: el puente hacia el sistema de archivos

OperatingSystem expone keywords para interactuar con archivos, carpetas y variables de entorno del sistema operativo donde corre la suite: `File Should Exist`, `Create Directory`, `Get File`, `Remove File`, `Get Environment Variable`. Es la biblioteca que usarás constantemente en procesos RPA (Capítulo 8), donde leer/escribir archivos es una operación central.

Tabla comparativa de las tres bibliotecas

Biblioteca	Importación	Propósito	Keywords representativas
BuiltIn	Automática, siempre disponible	Aserciones, logging, conversiones, control de flujo	<code>Should Be Equal</code> , <code>Log</code> , <code>Convert To Integer</code>
Collections	<code>Library Collections</code>	Listas y diccionarios	<code>Create Dictionary</code> , <code>Append To List</code>
OperatingSystem	<code>Library OperatingSystem</code>	Archivos, carpetas, entorno	<code>File Should Exist</code> , <code>Get Environment Variable</code>

Ejemplo comentado

```
*** Settings ***
Library      Collections
Library      OperatingSystem

*** Test Cases ***
Registrar Un Nuevo Cliente Y Verificar Su Archivo De Respaldo
    [Documentation]    Combina las tres bibliotecas: BuiltIn (implícita)
    ...                para aserciones, Collections para modelar el
    ...                cliente, OperatingSystem para el archivo.

    # Collections: modela el cliente como diccionario
    ${cliente}=        Create Dictionary    nombre=Ana Pérez    plan=Premium

    # BuiltIn: aserción directa sobre el diccionario
    Should Be Equal    ${cliente}[plan]    Premium

    # OperatingSystem: verifica que existe un archivo de respaldo
    File Should Exist    ${CURDIR}/respaldo_clientes.csv

    # BuiltIn: logging del resultado
    Log    Cliente ${cliente}[nombre] registrado con plan ${cliente}[plan]
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué representa “el qué” en la jerarquía de RF?	El test case
¿Qué biblioteca expone Create Dictionary?	Collections
¿Qué biblioteca expone File Should Exist?	OperatingSystem
¿Necesita Library Collections en Settings?	Sí, no es automática como BuiltIn

Errores comunes

- **Poner lógica de negocio compleja directamente en el test case** en vez de extraerla a una keyword propia — el test case deja de ser legible para alguien no técnico.
- **Olvidar Library Collections** al usar Create Dictionary por primera vez, asumiendo que es tan automática como BuiltIn.
- **Usar OperatingSystem.Get File para archivos binarios** (como imágenes o PDFs) — esa keyword está pensada para texto; archivos binarios requieren un manejo distinto (por ejemplo, desde una librería Python propia, como verás en el Capítulo 5).

Puntos clave

- Test case = qué verificar; keyword = cómo hacerlo; biblioteca = quién provee la keyword.
- BuiltIn se carga automáticamente; el resto de bibliotecas requiere Library <nombre>.
- Collections trabaja con listas/diccionarios; OperatingSystem, con el sistema de archivos.

Autoevaluación

1. ¿Qué biblioteca necesitas declarar para usar Append To List?
2. ¿Por qué un test case no debería contener lógica de negocio extensa directamente?
3. Verdadero o falso: OperatingSystem.Get File es apropiada para leer archivos PDF.

Respuestas: 1. Collections. 2. Porque deja de ser legible para alguien no técnico — esa lógica debería vivir en una keyword propia. 3. Falso — esa keyword está pensada para texto, no para binarios.

2.2 Variables: escalares, listas y diccionarios

Objetivos de la lección

- Diferenciar los tres tipos de variables de Robot Framework.
- Elegir el tipo correcto según el dato que se necesita modelar.
- Explicar la precedencia entre variables definidas en distintos lugares.

¿Por qué importa?

Las variables son el mecanismo central para parametrizar una suite — evitan repetir el mismo valor en distintos lugares y permiten cambiar configuración (URLs, credenciales, datos de prueba) sin tocar la lógica. Elegir el tipo equivocado (por ejemplo, una lista cuando necesitabas un diccionario) genera código artificialmente complicado más adelante.

Conceptos clave

Variables escalares: el tipo más común

Una variable escalar (`${var}`) almacena un único valor — texto, número, booleano, o incluso un objeto Python completo (como una lista o un diccionario, si así se asignó). Es el tipo más común porque, incluso cuando el valor es una lista, puedes referenciarlo como escalar si no necesitas iterarlo elemento por elemento en ese punto del código.

Variables de lista: expansión posicional

Una variable de lista (`@{lista}`) almacena varios valores en orden. La diferencia clave está en el **prefijo usado al referenciarla**: con `@{lista}`, Robot Framework expande sus elementos como argumentos posicionales separados (útil al pasarla a una keyword que espera varios argumentos); con `${lista}`, se trata como un solo objeto de tipo lista de Python (útil para pasarla completa a una keyword que espera un solo argumento de tipo lista, como muchas de Collections).

Variables de diccionario: pares clave-valor

Una variable de diccionario (&{dict}) almacena pares clave-valor — el equivalente a `**kwargs` en Python. Se accede a un valor específico con `${dict}[clave]` (nota el prefijo `${...}`, no `&{...}`, al acceder a un valor individual — un detalle de sintaxis que confunde a muchos al empezar).

Tabla comparativa de los tres tipos

`${escalar}` y `@{lista}` • `${escalar}`: un único valor

- `@{lista}`: varios valores en orden
- `@{...}` expande como argumentos posicionales

`&{dict}` • Pares clave-valor

- Se accede con `${dict}[clave]`
- Equivalente a `**kwargs` de Python

Tipo	Sintaxis de declaración	Sintaxis de acceso a un valor	Caso de uso típico
Escalar	<code>\${URL}</code> <code>https://api.com</code>	<code>\${URL}</code>	Un valor de configuración
Lista	<code>@{NUMEROS}</code> 1 2 3	<code>\${NUMEROS}</code> [0] (índice) o <code>@{NUMEROS}</code> (expandida)	Varios valores relacionados
Diccionario	<code>&{CLIENTE}</code> nombre=Ana plan=Premium	<code>\${CLIENTE}[nombre]</code>	Datos con nombre de campo

Precedencia: qué variable “gana” cuando hay varias con el mismo nombre

Cuando una variable tiene el mismo nombre en distintos lugares, la precedencia (de más específico a más general) es: **argumento de línea de comandos** (`--variable NOMBRE:valor`) > **variable definida dentro del test case o keyword en ejecución** > **variable de la sección `*** Variables ***`** de la suite que se está ejecutando > **variable importada desde un Resource**. Este orden tiene una lógica: lo más cercano al punto de ejecución (CLI, luego el propio test) siempre puede sobrescribir lo más general (configuración compartida en un Resource).

Ejemplo comentado

```
*** Settings ***
Resource    ../resources/configuracion.resource
```

```
*** Variables ***
# Esta variable de suite tiene precedencia sobre la del Resource importado,
```

```
# pero será sobrescrita si se pasa --variable BASE_URL:... por CLI.
${BASE_URL}          https://staging.miempresa.com

@{ENDPOINTS}         /clientes    /planes    /facturas

&{CREDENCIALES}      usuario=admin    clave=Admin123!

*** Test Cases ***
Demostrar Los Tres Tipos De Variables En Un Solo Test
    Log    URL base: ${BASE_URL}

    # @{...} expande como argumentos posicionales
    FOR    ${endpoint}    IN    @{ENDPOINTS}
        Log    Endpoint disponible: ${endpoint}
    END

    # Acceso a un valor del diccionario con ${...}, no &{...}
    Should Be Equal    &{CREDENCIALES}[usuario]    admin

# El valor de CLI tiene la máxima precedencia: sobrescribe el de Variables
robot --variable BASE_URL:https://produccion.miempresa.com tests/suite.robot
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué prefijo expande una lista como argumentos posicionales?	@{...}
¿Cómo se accede a un valor de un diccionario?	\${dict}[clave] (con \${...}, no &{...})
¿Qué tiene mayor precedencia, --variable de CLI o *** Variables ***?	--variable de CLI

Errores comunes

- **Usar &{dict} para acceder a un valor individual** en vez de \${dict}[clave] — &{...} es solo para declarar o expandir el diccionario completo como kwargs, no para leer un campo.
- **Asumir que una variable de Resource siempre “gana”** sobre la de la suite — es exactamente al revés: la variable de la suite que importa el Resource tiene precedencia.
- **Mezclar tipos sin necesidad** — modelar un cliente con varias variables escalares sueltas (\${CLIENTE_NOMBRE}, \${CLIENTE_PLAN}) cuando un solo diccionario (&{CLIENTE}) sería más claro y escalable.

Puntos clave

- \${var} para un solo valor; @{lista} para varios; &{dict} para pares clave-valor.

- El prefijo usado al referenciar una variable determina cómo se expande (escalar vs. lista/dict completo vs. acceso a un elemento).
- La precedencia de variables va de más específico (CLI) a más general (Resource importado).

Autoevaluación

1. ¿Qué tipo de variable usarías para modelar un cliente con nombre, plan y fecha de alta?
2. ¿Cómo se accede al primer elemento de una variable de lista `@{NUMEROS}`?
3. Si defines `${BASE_URL}` en la suite y también en un Resource importado, ¿cuál gana?
4. ¿Qué tiene mayor precedencia que ambas?

Respuestas: 1. Un diccionario (`&{CLIENTE}`). 2. `${NUMEROS}[0]`. 3. La definida en la suite (más específica que el Resource). 4. Un valor pasado por `--variable` en la línea de comandos.

2.3 Archivos Resource: separación de keywords y parametrización de datos

Objetivos de la lección

- Explicar el propósito de un archivo Resource.
- Diferenciar qué secciones puede contener un Resource frente a una suite.
- Aplicar la importación de un Resource desde una suite.

¿Por qué importa?

A medida que una suite de pruebas crece, mantener todas las keywords, variables y configuraciones en un solo archivo `.robot` se vuelve difícil de sostener: corregir un bug en una keyword usada por 10 test cases significa encontrarla entre cientos de líneas. Un archivo Resource resuelve esto formalmente, no como una convención informal.

Conceptos clave

Qué es y qué no es un archivo Resource

Un **archivo Resource** (extensión `.resource`, o también `.robot` aunque no es la convención recomendada) centraliza keywords y variables que **cualquier suite puede importar**. La diferencia formal con una suite: un Resource **no puede contener** la sección `*** Test Cases ***` (ni `*** Tasks ***`) — solo Settings, Variables y Keywords. Esta restricción no es arbitraria: comunica una intención clara — “este archivo no ejecuta nada por sí mismo, solo provee”.

Tipo de archivo	Test Cases/Tasks	Keywords/Variables	Se ejecuta directamente con robot
Suite (.robot con tests)	Sí	Sí	Sí
Resource (.resource)	No	Sí	No (se importa)

Cómo se importa y qué queda disponible

Un Resource se importa en una suite con la línea `Resource <ruta>` en `*** Settings ***`. A partir de ahí, **todas** las keywords y variables definidas en el Resource quedan disponibles en la suite, como si estuvieran escritas ahí mismo — no hay un mecanismo de “importación selectiva” (como el `from x import y` de Python); se importa el archivo completo.

Los Resources también pueden importar Resources

Un archivo Resource puede, a su vez, importar otros Resources en su propia sección `*** Settings ***` — permitiendo construir una jerarquía de capas (por ejemplo, un Resource de “keywords técnicas de API” importado por un Resource de “keywords de dominio de facturación”, que a su vez es importado por la suite de pruebas). Esta composición es la base del patrón de capas que formalizarás en el Capítulo 4 (BDD) y el Capítulo 6 (Page Object).

Los tres beneficios concretos

1. **Reutilización:** la misma keyword sirve para varias suites, sin copiar y pegar código.
2. **Mantenibilidad:** corregir un bug en una keyword lo corrige automáticamente en todos los lugares que la usan — un solo punto de cambio.
3. **Legibilidad:** el archivo .robot de la suite queda enfocado en *qué* se prueba, sin mezclar *cómo* se hace internamente.

Ejemplo comentado

```
# archivo: resources/clientes_keywords.resource
*** Settings ***
Documentation      Keywords reutilizables para gestión de clientes de telecom.
Library            Collections

*** Variables ***
${PLAN_BASICO}     Básico
${PLAN_PREMIUM}    Premium

*** Keywords ***
Crear Cliente
    [Documentation]  Crea un diccionario que representa un cliente.
```

```

[Arguments]    ${nombre}    ${plan}
${cliente}=    Create Dictionary    nombre=${nombre}    plan=${plan}
RETURN        ${cliente}

Validar Plan Asignado
[Arguments]    ${cliente}    ${plan_esperado}
Should Be Equal    ${cliente}[plan]    ${plan_esperado}

# archivo: tests/gestion_clientes.robot
*** Settings ***
Documentation     Suite que importa keywords desde un archivo Resource.
Resource          ../resources/clientes_keywords.resource

*** Test Cases ***
Crear Cliente Con Plan Básico
    # ${PLAN_BASICO} viene del Resource — esta suite NUNCA lo declaró.
    ${cliente}=    Crear Cliente    Ana Pérez    ${PLAN_BASICO}
    Validar Plan Asignado    ${cliente}    ${PLAN_BASICO}

```

Observa que **ni Crear Cliente ni \${PLAN_BASICO} están definidas en la suite** — vienen del Resource importado. Eso es justamente lo que se gana al separar capas: la suite es más corta y describe solo el escenario de prueba.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué sección NO puede tener un archivo Resource?	Test Cases (ni Tasks)
¿Cómo se importa un Resource en una suite?	Resource <ruta> EN Settings
¿Puede un Resource importar otro Resource?	Sí
¿Se puede importar solo una keyword específica de un Resource?	No, se importa el archivo completo

Errores comunes

- **Intentar agregar `*** Test Cases ***` a un archivo `.resource`** — Robot Framework lo rechaza; si necesitas tests, ese archivo debe ser una suite, no un Resource.
- **Duplicar una keyword en varios Resources** en vez de centralizarla en uno solo e importarlo donde se necesite — recrea el problema que el patrón Resource busca resolver.
- **Rutas relativas incorrectas al importar** — la ruta en Resource <ruta> es relativa a la ubicación del archivo que la declara, no al directorio desde donde ejecutas robot.

Puntos clave

- Un Resource centraliza keywords/variables reutilizables; no contiene Test Cases/Tasks.
- Se importa con Resource <ruta> en Settings, completo (no selectivo).
- Los Resources pueden importar otros Resources, habilitando arquitecturas en capas.

Autoevaluación

1. ¿Qué pasa si intentas agregar `*** Test Cases ***` a un archivo `.resource`?
2. ¿Puedes importar solo la keyword `Crear Cliente` de un Resource que tiene 10 keywords, sin las otras 9?
3. ¿Puede un archivo Resource importar a otro Resource?

Respuestas: 1. Robot Framework lo rechaza — un Resource no puede tener esa sección. 2. No, la importación de un Resource siempre es completa. 3. Sí, habilitando jerarquías de capas.

2.4 Tags, Setup/Teardown y organización jerárquica de suites

Objetivos de la lección

- Explicar el propósito de los tags y cómo filtran la ejecución.
- Describir los cuatro niveles de Setup/Teardown.
- Aplicar organización jerárquica con `__init__.robot`.

¿Por qué importa?

Cuando un proyecto de automatización crece a cientos de test cases, ejecutar siempre la suite completa deja de ser práctico — necesitas poder ejecutar “solo lo crítico” o “solo lo de facturación”. Y cuando varias suites comparten el mismo estado inicial (por ejemplo, todas necesitan una sesión de base de datos abierta), repetir esa configuración en cada test case es frágil y costoso de mantener.

Conceptos clave

Tags: clasificación y filtrado

Los **tags** son etiquetas de texto libre asignadas a un test case con `[Tags]`, o a toda la suite con `Test Tags` en Settings (heredado por todos los test cases de esa suite). Desde la línea de comandos, `--include TAG` ejecuta solo los tests con ese tag, y `--exclude TAG` los omite. Un esquema típico combina varias dimensiones en los mismos test cases: prioridad (`smoke`, `regresion`), dominio (`facturacion`, `clientes`) y ambiente (`staging`, `produccion`) — un mismo test puede llevar varios tags simultáneamente.

Los cuatro niveles de Setup/Teardown

Nivel	¿Cuándo se ejecuta?	¿Se ejecuta si el paso anterior falla?
Suite Setup	Una vez, antes del primer test de la suite	N/A (es el primero)
Test Setup	Antes de cada test case	Sí, salvo que Suite Setup haya fallado
Test Teardown	Después de cada test case	Sí, incluso si el test falló
Suite Teardown	Una vez, después del último test	Sí, incluso si algún test falló

El comportamiento más importante de recordar: **Teardown se ejecuta siempre**, sin importar el resultado del paso que protege — es el lugar correcto para liberar recursos (cerrar un navegador, una conexión de base de datos, borrar archivos temporales). Depender de que el test pase para hacer la limpieza es un error de diseño que deja recursos abiertos cuando algo falla — justo cuando más necesitas que el entorno quede limpio para el siguiente intento.

Organización jerárquica con directorios

Para proyectos grandes, Robot Framework permite que una suite sea, en vez de un único archivo, **un directorio completo**, donde cada subcarpeta es una “sub-suite” anidada. Un archivo especial `__init__.robot` dentro de un directorio define el Setup/Teardown que aplica a **todas** las suites de esa carpeta — útil para configuración compartida (por ejemplo, abrir una conexión de base de datos una sola vez para todos los tests de un módulo, en vez de repetirlo en cada archivo).

```
tests/
├── __init__.robot          # Setup/Teardown para TODO el proyecto
├── clientes/
│   ├── __init__.robot      # Setup/Teardown solo para este módulo
│   ├── alta_cliente.robot
│   └── baja_cliente.robot
└── facturacion/
    ├── __init__.robot
    └── generar_factura.robot
```

Ejemplo comentado

```
*** Settings ***
Documentation    Suite con tags y los cuatro niveles de Setup/Teardown.
Suite Setup      Log      Iniciando suite de regresión de clientes
Suite Teardown   Log      Finalizando suite de regresión de clientes
Test Setup       Log      Preparando datos para el siguiente test
Test Teardown    Log      Limpieza posterior al test
```



```

*** Test Cases ***
Cliente Premium Tiene Soporte Prioritario
    [Tags]    regresion    premium
    Should Be True    ${True}

Sistema De Gestión De Clientes Responde
    [Tags]    smoke
    Should Be True    ${True}

# Solo los tests marcados como smoke (rápido, para cada despliegue)
robot --include smoke tests/suite.robot

# Todo lo de regresión EXCEPTO lo marcado como básico
robot --include regresion --exclude basico tests/suite.robot

```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué nivel de Setup/Teardown se ejecuta una sola vez por suite?	Suite Setup/Suite Teardown
¿Se ejecuta Test Teardown si el test falló?	Sí, siempre
¿Qué archivo define Setup/Teardown para todas las suites de un directorio?	__init__.robot
¿Qué hace --exclude TAG?	Omite los tests que tienen ese tag

Errores comunes

- **Poner lógica de limpieza crítica en el cuerpo del test en vez de en Teardown** — si el test falla antes de llegar a esa línea, la limpieza nunca ocurre.
- **Asumir que Suite Teardown no se ejecuta si un test falló** — sí se ejecuta, siempre, igual que Test Teardown.
- **Usar tags inconsistentes** (Smoke en un test, smoke en otro) — aunque Robot Framework normaliza algunos aspectos, mantener una convención de escritura consistente evita confusión en el esquema de tags del equipo.

Puntos clave

- [Tags] clasifica un test case; Test Tags en Settings aplica a toda la suite; --include/--exclude filtran la ejecución.
- Suite Setup/Teardown: una vez por suite. Test Setup/Teardown: una vez por test, siempre — incluso ante fallo.
- Un directorio con __init__.robot actúa como una suite jerárquica con Setup/Teardown compartido.

Autoevaluación

1. Si un test falla a mitad de ejecución, ¿se ejecuta su Test Teardown?

2. ¿Qué comando ejecuta solo los tests con el tag `smoke`, excluyendo los que también tengan `wip`?
3. ¿Qué archivo especial define configuración compartida para todas las suites de un directorio?

Respuestas: 1. Sí, siempre se ejecuta, sin importar el resultado del test. 2. `robot --include smoke --exclude wip tests/`. 3. `__init__.robot`.

Resumen del capítulo

Los test cases describen el qué; las keywords describen el cómo; las bibliotecas (`BuiltIn`, `Collections`, `OperatingSystem`) proveen el vocabulario técnico. Las variables tienen tres formas — escalar, lista, diccionario — según cuántos valores necesitan modelar, con una precedencia clara entre CLI, test/keyword, suite y Resource. Los archivos Resource separan keywords reutilizables del archivo de la suite, sin poder contener Test Cases/Tasks, y pueden componerse en jerarquías. Los tags filtran qué se ejecuta, y Setup/Teardown prepara y limpia el entorno en cuatro niveles posibles, con Teardown garantizado incluso ante fallo.

Referencias bibliográficas

- Resource files: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#resource-files>
- Librería Collections: <https://robotframework.org/robotframework/latest/libraries/Collections.html>
- Setups y Teardowns: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#setups-and-teardowns>

Capítulo 3 — Control de Flujo y Validaciones

Información general

Este capítulo agrega lógica condicional e iterativa a las suites, y profundiza en cómo Robot Framework maneja las fallas — tanto las que detienen la ejecución por defecto como las que se pueden controlar deliberadamente. Es uno de los capítulos con más peso en el examen RFCP, porque la discriminación entre keywords de manejo de fallas similares es una fuente común de error.

Lecciones de este capítulo:

- 3.1 — IF / ELSE IF / ELSE: ejecución condicional en keywords y test cases
 - 3.2 — FOR: iteración sobre listas y rangos
 - 3.3 — Asserts y fallas controladas: Should Be, Run Keyword And Expect Error
 - 3.4 — Continue On Failure, Run Keyword And Continue On Failure y recuperación
-

3.1 IF / ELSE IF / ELSE: ejecución condicional en keywords y test cases

Objetivos de la lección

- Explicar la sintaxis del bloque IF nativo de Robot Framework.
- Aplicar ELSE IF y ELSE para cubrir múltiples condiciones.
- Diferenciar el bloque IF nativo del enfoque heredado Run Keyword If.

¿Por qué importa?

Rara vez todos los escenarios de prueba son idénticos. Sin lógica condicional, te ves obligado a duplicar test cases casi idénticos solo para cubrir una rama de decisión distinta — lo que multiplica el costo de mantenimiento sin agregar cobertura real.

Conceptos clave

Sintaxis nativa del bloque IF (RF 4.0+)

El bloque IF (sintaxis nativa desde Robot Framework 4.0) permite ejecutar keywords distintas según el resultado de una condición:

```
IF      ${condicion}
    Log    Se cumple la condición
ELSE IF  ${otra_condicion}
    Log    Se cumple la otra condición
ELSE
    Log    Ninguna condición se cumplió
END
```

Cada bloque IF debe cerrarse con END, en el mismo nivel de indentación que la palabra IF que lo abrió. La condición se evalúa como una expresión Python, lo que permite usar operadores relacionales (<, >, <=, >=, ==, !=) directamente sobre variables, así como operadores lógicos (and, or, not) para condiciones compuestas.

Condiciones compuestas y anidamiento

```
IF    ${edad} >= 18 and ${tiene_documento}
    Log    Cliente puede contratar el servicio
END
```

Anidar bloques IF dentro de otro IF es válido, pero cada nivel adicional de anidamiento reduce la legibilidad — una buena práctica es extraer la lógica anidada a una keyword propia cuando supera dos niveles.

El enfoque heredado: Run Keyword If

Antes de Robot Framework 4.0, el único mecanismo disponible era la keyword Run Keyword If, de la biblioteca BuiltIn:

```
Run Keyword If    ${condicion}    Log    Se cumple
...    ELSE    Log    No se cumple
```

Sigue siendo válida y compatible (la encontrarás en proyectos legados), pero el bloque IF nativo es más legible — múltiples ramas con ELSE IF son mucho más claras que encadenar Run Keyword If/ELSE IF/ELSE en una sola línea larga — y es el que se enseña y se usa en este curso.

Ejemplo comentado

```
*** Keywords ***
```

Clasificar Consumo

```
[Documentation]    Clasifica un consumo en GB como bajo, medio o alto,
...                usando IF / ELSE IF / ELSE con operadores relacionales.
[Arguments]    ${consumo}
IF    ${consumo} < 50
    Log    ${consumo}GB -> Consumo bajo
ELSE IF    ${consumo} < 100
    Log    ${consumo}GB -> Consumo medio
ELSE
    Log    ${consumo}GB -> Consumo alto
END
```

Con `${consumo} = 85`: la primera condición (`85 < 50`) es falsa, la segunda (`85 < 100`) es verdadera, así que se ejecuta esa rama y el bloque termina ahí — ELSE nunca se evalúa cuando una rama anterior ya se cumplió.

Tabla de referencia rápida

Construcción	Disponible desde	¿Permite múltiples ramas?
IF/ELSE IF/ELSE nativo	Robot Framework 4.0	Sí, de forma clara
Run Keyword If (heredado)	Cualquier versión	Sí, pero menos legible con muchas ramas

Operador relacional	Significado
==, !=	Igual, distinto
<, >, <=, >=	Comparación numérica
and, or, not	Combinación lógica de condiciones

Errores comunes

- **Olvidar el END de cierre** — el error de sintaxis más frecuente en este tema; Robot Framework reporta un error de parseo que puede no señalar exactamente la línea faltante.
- **Comparar strings sin comillas en la condición**, por ejemplo `IF ${estado} == FAIL` en vez de `IF '${estado}' == 'FAIL'` — sin comillas, Python (que evalúa la expresión) puede interpretar FAIL como un nombre de variable inexistente, no como el texto literal.
- **Anidar más de 2-3 niveles de IF** — señal de que la lógica debería extraerse a una o más keywords propias.

Puntos clave

- El bloque IF/ELSE IF/ELSE nativo está disponible desde Robot Framework 4.0 y siempre se cierra con END.
- Las condiciones se evalúan como expresiones Python: operadores relacionales y lógicos funcionan igual que en Python.
- Run Keyword If es el equivalente heredado, anterior a la sintaxis nativa, todavía válido pero menos legible con varias ramas.

Autoevaluación

1. ¿Qué palabra cierra obligatoriamente un bloque IF?
2. ¿Por qué `'${estado}' == 'FAIL'` necesita las comillas, a diferencia de `${numero} > 10`?
3. ¿Desde qué versión de Robot Framework está disponible el bloque IF nativo?

Respuestas: 1. END. 2. Porque se está comparando texto (string); sin comillas, la expresión Python podría interpretar FAIL como un nombre de variable, no como texto literal. 3. Desde la versión 4.0.

3.2 FOR: iteración sobre listas y rangos

Objetivos de la lección

- Aplicar FOR ... IN ... END para iterar sobre una lista.
- Usar IN RANGE para iterar sobre una secuencia numérica.
- Combinar FOR con IF para validar colecciones de datos.

¿Por qué importa?

Cuando una verificación debe repetirse para varios valores — una lista de clientes, de consumos, de endpoints —, repetir el código manualmente para cada uno no escala: si la lógica de validación cambia, hay que actualizarla en cada copia.

Conceptos clave

Sintaxis básica: iterar sobre una lista

```
FOR    ${elemento}    IN    @{lista}
    Log    Elemento actual: ${elemento}
END
```

El bloque FOR itera sobre una lista, asignando cada elemento a la variable declarada (\${elemento} en el ejemplo) en cada vuelta. Igual que IF, todo bloque FOR debe cerrarse con END.

Iterar sobre un rango numérico

```
FOR    ${indice}    IN RANGE    5
    Log    Iteración número ${indice}
END
```

IN RANGE genera una secuencia numérica (0, 1, 2, 3, 4 en el ejemplo) sin necesitar una lista previa — equivalente al range() de Python. También admite IN RANGE inicio fin y IN RANGE inicio fin paso.

El patrón más usado: FOR + IF anidado

Combinar FOR con IF es uno de los patrones más usados en suites reales: recorrer una colección de datos y, para cada elemento, decidir una acción según una condición.

```
FOR    ${cliente}    IN    @{clientes}
    IF    ${cliente}[consumo] > ${cliente}[limite]
        Log    ${cliente}[nombre] excede su límite    WARN
    ELSE
        Log    ${cliente}[nombre] está dentro de su límite
    END
END
```

Este patrón es exactamente el que practicarás recorriendo una lista de diccionarios — cada diccionario representando un cliente con su consumo y límite contratado.

Ejemplo comentado

```
*** Test Cases ***
Detectar Clientes Que Exceden Su Límite De Plan
```

```

[Documentation]    FOR + IF anidado: el patrón estándar para validar
...               colecciones completas de datos.
${cliente_1}=     Create Dictionary    nombre=Ana    limite=${50}
consumo=${45}
${cliente_2}=     Create Dictionary    nombre=Luis    limite=${50}
consumo=${72}
@{clientes}=      Create List    ${cliente_1}    ${cliente_2}

FOR    ${cliente}    IN    @{clientes}
    IF    ${cliente}[consumo] > ${cliente}[limite]
        Log    ${cliente}[nombre] excede su límite de ${cliente}[limite]GB
WARN
    ELSE
        Log    ${cliente}[nombre] está dentro de su límite
    END
END
END

```

¿Por qué `limite=${50}` y no `limite=50`? El prefijo `${50}` convierte el texto en un número entero de Python al construir el diccionario. Sin eso, `limite` sería el string "50", y comparar `${cliente}[consumo] > ${cliente}[limite]` con strings no compara números — compara texto, lo cual produce resultados incorrectos e inconsistentes para números de más de un dígito.

Tabla de referencia rápida

Sintaxis	Qué genera
FOR \${x} IN @{lista}	Itera sobre los elementos de la lista
FOR \${x} IN RANGE 5	Itera 0, 1, 2, 3, 4
FOR \${x} IN RANGE 2 5	Itera 2, 3, 4
FOR \${x} IN RANGE 0 10 2	Itera 0, 2, 4, 6, 8 (con paso 2)

Errores comunes

- **Olvidar el END de cierre** — igual que con IF, es el error de sintaxis más frecuente.
- **Construir un diccionario con números sin el prefijo \${...}**, dejándolos como texto, y luego comparar esos valores numéricamente — produce resultados incorrectos silenciosos (no un error de sintaxis, sino un resultado lógico equivocado).
- **Anidar FOR dentro de FOR sin necesidad real** — válido, pero revisa primero si la lógica puede simplificarse o extraerse a una keyword.

Puntos clave

- FOR ... IN ... END itera sobre listas; FOR ... IN RANGE ... END itera sobre una secuencia numérica.
- FOR + IF anidado es el patrón estándar para validar colecciones de datos.

- Construir diccionarios con valores numéricos requiere el prefijo `${...}` para que la comparación posterior sea numérica, no textual.

Autoevaluación

1. ¿Qué genera `FOR ${x} IN RANGE 3`?
2. Si construyes `Create Dictionary consumo=50` (sin `${...}`), ¿qué tipo de dato queda almacenado en `consumo`?
3. ¿Qué patrón combinado se usa para validar una colección completa de datos, aplicando una condición a cada elemento?

Respuestas: 1. La secuencia 0, 1, 2. 2. Un string (texto), no un número entero. 3. `FOR + IF` anidado.

3.3 Asserts y fallas controladas: Should Be, Run Keyword And Expect Error

Objetivos de la lección

- Aplicar la familia de keywords `Should Be` para aserciones.
- Usar `Run Keyword And Expect Error` para validar errores esperados.
- Diferenciar una falla controlada de una falla no controlada.

¿Por qué importa?

Verificar que el sistema bajo prueba se comporta exactamente como se espera — ni más, ni menos — es el corazón de cualquier test. Pero hay un caso que se pasa por alto fácilmente: validar que el sistema **falla correctamente** ante una entrada inválida es tan importante como validar que funciona ante una entrada válida.

Conceptos clave

La familia `Should Be`: aserciones básicas

La familia de keywords `Should Be` (de `BuiltIn`) cubre las aserciones más comunes: `Should Be Equal` (igualdad exacta), `Should Be True` (la expresión es verdadera), `Should Be Empty` (una colección o string está vacío), `Should Be Equal As Integers/As Numbers` (compara convirtiendo a tipo numérico primero, evitando el problema de comparar "7" contra 7 directamente). Cada una falla con un mensaje claro si la condición no se cumple, deteniendo el test en ese punto — el comportamiento por defecto, que verás formalizado en la lección 3.4 como *fail fast*.

Cuando esperas que algo falle: `Run Keyword And Expect Error`

Hay un caso distinto al anterior: cuando **esperas que algo falle**, y ese fallo es justamente lo que quieres validar. Por ejemplo, confirmar que el sistema rechaza correctamente una entrada inválida. Para esto existe `Run Keyword And Expect Error`:

Run Keyword And Expect Error ValueError: * Convertir A Entero no-es-numero

Esta keyword ejecuta Convertir A Entero esperando que falle con un error que coincida con el patrón ValueError: * (el * es un wildcard que acepta cualquier texto después de los dos puntos). Si el error ocurre **exactamente como se esperaba**, el test pasa — lo contrario de lo que pasaría si el error fuera inesperado (en cuyo caso Run Keyword And Expect Error también falla, porque el comportamiento real no coincidió con el esperado).

Patrones de coincidencia de error

El patrón de error admite varios formatos: texto exacto (ValueError: el valor no es numérico), wildcard parcial (ValueError: *), o incluso expresiones regulares (regexp: ValueError:.*, prefijo especial). Usar wildcards es la práctica más común, porque no depende del texto exacto del mensaje de error, que puede variar entre versiones de una librería.

Falla controlada vs. no controlada

	Falla no controlada	Falla controlada
¿Qué es?	Un error inesperado que detiene el test	Un error esperado, validado deliberadamente
Resultado del test	FAIL	PASS (si el error coincide con lo esperado)
Ejemplo	Selector CSS no encontrado por accidente	Verificar que un login con clave incorrecta es rechazado

Ejemplo comentado

*** Keywords ***

Convertir A Entero

```
[Documentation]    Convierte un valor a entero; lanza ValueError si
...                el valor no es numérico (vía Convert To Integer).
[Arguments]       ${valor}
${resultado}=     Convert To Integer    ${valor}
RETURN            ${resultado}
```

*** Test Cases ***

Capturar Un Error Esperado Sin Marcar El Test Como Fallido

```
[Documentation]    Run Keyword And Expect Error confirma que una
...                operación inválida lanza exactamente el error
...                esperado — y el test queda en PASS si así ocurre.
Run Keyword And Expect Error    ValueError: *    Convertir A Entero    no-es-
numero
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué keyword valida que un error esperado ocurrió, sin fallar el test?	Run Keyword And Expect Error
¿Qué hace el * en un patrón de error?	Wildcard, acepta cualquier texto en esa posición
¿Qué keyword compara dos valores convirtiéndolos a número primero?	Should Be Equal As Integers / As Numbers

Errores comunes

- **Usar Should Be Equal para comparar un string "7" contra el entero 7** sin conversión — puede comportarse de forma inesperada según el contexto; usa Should Be Equal As Integers cuando la comparación es numérica.
- **Escribir el patrón de error sin wildcard**, dependiendo del mensaje exacto — frágil ante actualizaciones de una librería que cambien ligeramente el texto del error.
- **Confundir “el test pasa” con “no hubo ningún error”** — en Run Keyword And Expect Error, el test pasa **porque** hubo el error esperado, no porque no hubo ninguno.

Puntos clave

- Should Be Equal, Should Be True, etc. son las aserciones básicas de BuiltIn.
- Run Keyword And Expect Error invierte la lógica: el test pasa si el error esperado ocurre exactamente como se especificó.
- Los patrones de error admiten wildcards (*) para no depender del mensaje exacto.

Autoevaluación

1. ¿Qué keyword usarías para validar que el sistema rechaza correctamente una contraseña inválida con un mensaje de error específico?
2. ¿Qué pasa si Run Keyword And Expect Error espera ValueError: * pero la keyword no lanza ningún error?
3. ¿Por qué es preferible usar un wildcard en el patrón de error en vez del mensaje exacto?

Respuestas: 1. Run Keyword And Expect Error. 2. El test falla — se esperaba un error y no ocurrió ninguno. 3. Porque el mensaje exacto puede cambiar entre versiones de una librería; el wildcard hace la validación más resiliente.

3.4 Continue On Failure, Run Keyword And Continue On Failure y recuperación

Objetivos de la lección

- Explicar el comportamiento *fail fast* por defecto de Robot Framework.
- Aplicar Run Keyword And Continue On Failure y Run Keyword And Ignore Error.
- Diferenciar los tres patrones de manejo de fallas del capítulo.

¿Por qué importa?

Por defecto, Robot Framework sigue un modelo **fail fast**: en cuanto una keyword falla, el test se detiene ahí mismo — las líneas siguientes no se ejecutan. Esto es correcto la mayoría de las veces (no tiene sentido seguir probando un sistema que ya falló), pero hay situaciones donde conviene ver **todos** los errores de un test, no solo el primero — por ejemplo, validando varios campos de un formulario a la vez, para reportar de una sola vez cuáles fallaron.

Conceptos clave

Run Keyword And Continue On Failure: ver todos los errores

Run Keyword And Continue On Failure envuelve una keyword para que, si falla, **el fallo se registre pero la ejecución continúe** con las siguientes líneas del test:

```
Run Keyword And Continue On Failure    Should Be Equal    ${1}    ${1}
msg=Campo A
Run Keyword And Continue On Failure    Should Be Equal    ${1}    ${2}
msg=Campo B
Run Keyword And Continue On Failure    Should Be Equal    ${3}    ${3}
msg=Campo C
```

Importante: el test sigue marcado como FAIL al final si alguna verificación falló de verdad — esta keyword no oculta errores, solo evita que detengan la ejecución antes de tiempo. En el ejemplo, los tres Should Be Equal se ejecutan (Campo A y Campo C pasan, Campo B falla), pero el test completo queda en FAIL porque al menos una verificación falló.

Run Keyword And Ignore Error: decidir tú el flujo

Cuando necesitas **decidir tú mismo** qué hacer según el resultado de una keyword, sin que el fallo se propague como una excepción, usas Run Keyword And Ignore Error. Esta keyword devuelve dos valores: el estado (el string 'PASS' o 'FAIL') y el mensaje. Con eso puedes usar un bloque IF para decidir el flujo:

```
${estado}    ${mensaje}=    Run Keyword And Ignore Error    Convertir A Entero
abc
IF    '${estado}' == 'FAIL'
    Log    Falló como se esperaba: ${mensaje}
ELSE
    Fail    Se esperaba que la conversión fallara
END
```

Las tres patrones del capítulo, comparados explícitamente

Continue On Failure
(ver todos los errores)

Expect Error
(fallo esperado = PASS)

Ignore Error
(decides tú el flujo)

Patrón	¿Qué hace?	¿Cuándo usarlo?
Run Keyword And Continue On Failure	Ejecuta la keyword; si falla, registra el fallo pero sigue	Validar varios campos y ver todos los errores juntos
Run Keyword And Expect Error	Ejecuta esperando que falle con un mensaje específico; si falla así, el test queda en PASS	Probar que el sistema rec-haza correctamente una entrada inválida
Run Keyword And Ignore Error	Devuelve el estado y el mensaje, sin detener nada	Decidir tú mismo, con IF, qué hacer según el resultado

Estos tres patrones no son intercambiables: cada uno resuelve un caso de uso distinto, y elegir el correcto depende de qué necesitas hacer con el resultado del fallo (verlo todo, validarlo como esperado, o decidir tú el flujo). Confundirlos es uno de los errores más comunes al preparar el examen RFCP, precisamente porque sus nombres son similares.

Ejemplo comentado

*** Test Cases ***

Validar Varios Campos Sin Detenerse En El Primer Fallo

```
[Documentation]    El test SÍ queda en FAIL al final – es intencional:
...               el objetivo es ver los 3 mensajes, no evitar el FAIL.
Run Keyword And Continue On Failure    Should Be Equal    ${1}    ${1}
msg=Campo A
Run Keyword And Continue On Failure    Should Be Equal    ${1}    ${2}
msg=Campo B (fallo intencional)
Run Keyword And Continue On Failure    Should Be Equal    ${3}    ${3}
msg=Campo C
```

Ejecutando solo este test, log.html muestra los tres mensajes — Campo A y Campo C sí se evaluaron, no se detuvo la ejecución en el Campo B — pero el resultado final del test es FAIL, porque el Campo B realmente falló.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Cuál es el comportamiento por defecto de Robot Framework ante un fallo?	Fail fast: detiene el test inmediatamente
¿Qué keyword permite ver todos los errores de un test, no solo el primero?	Run Keyword And Continue On Failure
¿Qué devuelve Run Keyword And Ignore Error?	El estado ('PASS'/'FAIL') y el mensaje Error?
¿Run Keyword And Continue On Failure puede dejar el test en PASS si algo realmente falló?	No — el test queda en FAIL si alguna verificación falló de verdad

Errores comunes

- **Pensar que Run Keyword And Continue On Failure “perdona” el fallo** — no lo hace; el test sigue marcado como FAIL, solo permite ver más información antes de terminar.
- **Usar Run Keyword And Ignore Error cuando en realidad quieres Run Keyword And Expect Error** — si ya sabes exactamente qué error esperas y quieres que el test pase si ocurre, Expect Error es más directo y explícito.
- **Comparar el estado devuelto sin comillas**: IF \${estado} == FAIL en vez de IF '\${estado}' == 'FAIL' — el mismo error de comparación de strings visto en la lección 3.1.

Puntos clave

- Comportamiento por defecto: *fail fast* — una keyword fallida detiene el test inmediatamente.
- Run Keyword And Continue On Failure: registra el fallo, pero sigue ejecutando; el test queda en FAIL si algo falló de verdad.
- Run Keyword And Ignore Error: devuelve (estado, mensaje) para que decidas tú el flujo con IF.
- Run Keyword And Expect Error (lección 3.3) es el tercer patrón: valida un error esperado como condición de éxito.

Autoevaluación

1. ¿Cuál de los tres patrones de este capítulo usarías para validar 5 campos de un formulario y ver todos los errores en un solo reporte?
2. ¿Cuál usarías para que el test pase específicamente porque ocurrió un ValueError esperado?
3. ¿Cuál usarías si necesitas decidir manualmente, con un IF, qué hacer según si una keyword pasó o falló?

4. Verdadero o falso: Run Keyword And Continue On Failure puede hacer que un test con un fallo real termine en PASS.

Respuestas: 1. Run Keyword And Continue On Failure. 2. Run Keyword And Expect Error. 3. Run Keyword And Ignore Error. 4. Falso — el test queda en FAIL si alguna verificación falló de verdad; esta keyword solo evita que se detenga antes de tiempo.

Resumen del capítulo

El bloque IF/ELSE IF/ELSE nativo (disponible desde RF 4.0) y el bloque FOR (incluyendo IN RANGE) cubren la lógica condicional e iterativa de una suite — ambos se cierran siempre con END, y su combinación (FOR + IF anidado) es el patrón estándar para validar colecciones de datos. Las aserciones Should Be * validan el comportamiento esperado, mientras que Run Keyword And Expect Error valida errores esperados sin fallar el test. Robot Framework usa *fail fast* por defecto; Run Keyword And Continue On Failure y Run Keyword And Ignore Error son los dos mecanismos restantes para tomar más control sobre cómo se maneja un fallo — los tres patrones de manejo de fallas (junto con Expect Error) no son intercambiables, cada uno responde a una pregunta distinta sobre qué hacer con un error.

Referencias bibliográficas

- IF / ELSE: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#if-else-syntax>
- FOR loops: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#for-loops>
- Run Keyword And Continue On Failure: <https://robotframework.org/robotframework/latest/libraries/BuiltIn.html#Run%20Keyword%20And%20Continue%20On%20Failure>
- Run Keyword And Expect Error: <https://robotframework.org/robotframework/latest/libraries/BuiltIn.html#Run%20Keyword%20And%20Expect%20Error>

Capítulo 4 — Behavior-Driven Development (BDD) y Pruebas Orientadas a Negocio

Información general

Este capítulo introduce BDD: un enfoque que acerca el lenguaje de las pruebas al lenguaje del negocio, particularmente valioso en telecomunicaciones, donde los flujos (activación de servicios, portabilidad, facturación) involucran reglas regulatorias y comerciales que un analista de negocio necesita poder leer y validar sin depender de un ingeniero.

Lecciones de este capítulo:

- 4.1 — Introducción a BDD: impacto en la alineación TI-Negocio y telecomunicaciones
 - 4.2 — Sintaxis Gherkin nativa en RF: Given, When, Then, And, But
 - 4.3 — Abstracción de capas: escenarios de negocio en lenguaje natural → keywords técnicas
 - 4.4 — Organización de features y escenarios para flujos comerciales agnósticos
-

4.1 Introducción a BDD: impacto en la alineación TI-Negocio y telecomunicaciones

Objetivos de la lección

- Explicar qué es BDD y de dónde surge.
- Describir el valor de BDD para reducir la brecha entre TI y negocio.
- Identificar casos de uso de BDD específicos de telecomunicaciones.

¿Por qué importa?

La mayoría de los defectos de software no nacen de un error de programación, sino de un malentendido sobre qué debía construirse. BDD ataca directamente esa causa raíz, no el síntoma.

Conceptos clave

Qué es BDD y de dónde viene

BDD (*Behavior-Driven Development*) es una evolución de TDD (*Test-Driven Development*) enfocada en describir el **comportamiento** del sistema desde la perspectiva del negocio, no de la implementación técnica. Mientras TDD pregunta “¿este código hace lo que el código debería hacer?”, BDD pregunta “¿el sistema se comporta como el negocio espera que se comporte?” — un cambio de perspectiva que mueve la conversación de la implementación al resultado observable.

Las Tres Amigos: el origen del proceso

El origen práctico de BDD está en la dinámica de las **Tres Amigos**: una reunión entre alguien que representa al negocio, alguien que desarrolla y alguien que prueba, para acordar — **antes** de escribir una sola línea de código — qué comportamiento exacto se espera, usando ejemplos concretos en lugar de descripciones abstractas. Esta conversación temprana es, en la práctica, más valiosa que la sintaxis Gherkin en sí misma: el formato Given/When/Then es solo el vehículo para capturar lo que ya se acordó.

Por qué reduce la brecha TI-Negocio

La brecha conceptual entre TI y negocio es una causa frecuente de defectos: un requisito ambiguo (“el sistema debe validar el crédito del cliente”) se interpreta de una forma por el desarrollador (verificar que el campo no esté vacío) y de otra por quien lo solicitó (verificar que el cliente tenga saldo disponible para el monto exacto del plan). BDD reduce esa brecha escribiendo los escenarios en un lenguaje estructurado pero natural — Gherkin —, que cualquiera de las Tres Amigos puede leer y validar sin ambigüedad, porque el escenario usa datos concretos, no descripciones generales.

Casos de uso concretos en telecomunicaciones

En telecomunicaciones, este valor es especialmente concreto porque varios procesos tienen reglas regulatorias estrictas:

- **Portabilidad numérica**: tiene tiempos máximos de respuesta definidos por regulación; un escenario BDD (“Dado un cliente solicita portabilidad, cuando se procesa dentro del tiempo regulatorio, entonces se confirma el cambio de operador”) sirve simultáneamente como especificación, documentación viva, y prueba automatizada — sin que el área de cumplimiento regulatorio necesite leer código.
- **Activación de planes**: las reglas de crédito, promociones y restricciones por tipo de cliente cambian con frecuencia comercial; BDD permite que el equipo comercial valide directamente los escenarios sin depender de un intermediario técnico.
- **Facturación**: los cálculos de prorrateo, impuestos y descuentos suelen tener casos límite (cambios de plan a mitad de ciclo, por ejemplo) que se comunican mejor como ejemplos concretos que como reglas abstractas.

Ejemplo comentado

```
# Esto NO es código ejecutable – es el "boceto" del escenario que el  
# negocio y TI acuerdan ANTES de escribir nada en Robot Framework.
```

Escenario: Activación de plan de datos para cliente con crédito suficiente

```
Dado un cliente con crédito suficiente existe en el sistema  
Cuando el cliente solicita la activación de un plan de datos adicional  
Entonces el plan queda activo y el cliente recibe una confirmación
```

Este boceto, una vez acordado entre negocio y TI, se traduce casi literalmente a Robot Framework en la lección 4.2 — esa traducción casi directa es justamente el punto: el escenario de negocio y el código de prueba quedan alineados palabra por palabra.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿De qué enfoque evoluciona BDD?	TDD
¿Qué pregunta central hace BDD, a diferencia de TDD?	¿El sistema se comporta como el negocio espera?
¿Qué reunión origina la dinámica de BDD?	Las Tres Amigos (negocio, desarrollo, pruebas)

Errores comunes

- **Saltarse la conversación de las Tres Amigos y escribir Gherkin directamente desde TI** — se pierde el propósito central de BDD; el formato sin el acuerdo previo es solo sintaxis, no alineación real.
- **Escribir escenarios con descripciones abstractas en vez de ejemplos concretos** (“el sistema valida correctamente” en vez de “un cliente con crédito de Q100 y un plan de Q50 puede activarlo”) — pierde la claridad que hace que BDD funcione.
- **Asumir que BDD es solo “una forma bonita de escribir tests”** — su valor real está en la conversación previa y en la documentación viva compartida, no en la sintaxis por sí sola.

Puntos clave

- BDD describe comportamiento en lenguaje de negocio, usando ejemplos concretos, no implementación abstracta.
- Las “Tres Amigos” acuerdan el comportamiento esperado antes de codificar — la conversación es más valiosa que la sintaxis.
- En telecom, BDD sirve como especificación + documentación + prueba en un solo artefacto, especialmente valioso en procesos regulados (portabilidad, facturación).

Autoevaluación

1. ¿Qué pregunta hace BDD que TDD no enfatiza de la misma forma?
2. ¿Por qué un escenario BDD con datos concretos (Q100 de crédito, plan de Q50) es mejor que uno abstracto (“el sistema valida correctamente”)?
3. Menciona un proceso de telecomunicaciones donde BDD aporta valor regulatorio específico.

Respuestas: 1. ¿El sistema se comporta como el negocio espera? (en vez de ¿el código hace lo que el código debe hacer?). 2. Porque elimina la ambigüedad — cualquiera de las Tres Amigos puede verificar el ejemplo concreto sin interpretación. 3. Portabilidad numérica (tiempos de respuesta regulados).

4.2 Sintaxis Gherkin nativa en RF: Given, When, Then, And, But

Objetivos de la lección

- Explicar cómo Robot Framework procesa los prefijos Gherkin.
- Aplicar Given/When/Then/And/But en un test case.
- Compartir estado entre pasos Given/When/Then con Set Test Variable.

¿Por qué importa?

Entender que Gherkin en Robot Framework no es “magia” especial, sino un mecanismo de texto simple, te permite usarlo con confianza y depurarlo cuando algo no funciona — sin tener que aprender una sintaxis paralela.

Conceptos clave

El mecanismo exacto: eliminación de prefijo

Robot Framework **no tiene una sintaxis BDD separada** — los prefijos Given, When, Then, And y But son simplemente texto que Robot Framework **elimina** antes de resolver el nombre de la keyword. Esto significa que:

Given un cliente con crédito suficiente existe en el sistema

busca **exactamente la misma keyword** que:

un cliente con crédito suficiente existe en el sistema

El prefijo existe únicamente para que un humano entienda el rol de cada paso — no para que el motor de ejecución tome una decisión distinta.

El rol semántico de cada prefijo

- Given — establece el **contexto** inicial (qué condiciones existen antes de la acción).
- When — ejecuta la **acción** que se quiere probar (el evento central del escenario).
- Then — verifica el **resultado** esperado (la consecuencia observable de la acción).
- And / But — continúan el paso anterior (otro Given, otro When, u otro Then), usados cuando un mismo rol necesita varios pasos consecutivos.

Para Robot Framework, los cinco prefijos son **mecánicamente equivalentes**: el motor no distingue significados entre ellos, simplemente los descarta antes de buscar la keyword. La diferencia entre ellos es una convención de Gherkin para el lector humano, no una regla que el motor interprete o valide.

Compartir estado entre pasos: Set Test Variable

Un desafío práctico al escribir escenarios Given/When/Then en varias keywords distintas: el contexto que Given establece necesita estar disponible cuando When y Then se ejecutan. Set Test Variable resuelve esto creando una variable visible para **todo el test case actual**, sin importar en qué keyword se definió:

*** Keywords ***

un cliente con crédito suficiente existe en el sistema

Set Test Variable \${CREDITO_DISPONIBLE} \${100}

```

el cliente solicita la activación de un plan de datos adicional
  ${costo_plan}= Set Variable    ${50}
  IF    ${CREDITO_DISPONIBLE} >= ${costo_plan}
    Set Test Variable    ${RESULTADO_ACTIVACION}    ACTIVO
  ELSE
    Set Test Variable    ${RESULTADO_ACTIVACION}    RECHAZADO
  END

```

```

el plan queda activo y el cliente recibe una confirmación
  Should Be Equal    ${RESULTADO_ACTIVACION}    ACTIVO

```

Sin `Set Test Variable` (usando, por ejemplo, `Set Variable` normal), la variable `${CREDITO_DISPONIBLE}` solo existiría dentro de la keyword donde se creó — `When` no podría leerla, porque cada keyword tiene su propio alcance local por defecto.

Ejemplo comentado

```

*** Settings ***

```

```

Resource    ../resources/activacion_keywords.resource

```

```

*** Test Cases ***

```

```

Activación De Plan De Datos Para Cliente Con Crédito Suficiente

```

```

  Given un cliente con crédito suficiente existe en el sistema
  When el cliente solicita la activación de un plan de datos adicional
  Then el plan queda activo y el cliente recibe una confirmación

```

```

Activación Rechazada Por Crédito Insuficiente

```

```

  Given un cliente con crédito insuficiente existe en el sistema
  When el cliente solicita la activación de un plan de datos adicional
  Then la activación es rechazada y se informa el motivo

```

Cualquier persona del negocio, sin conocer Robot Framework, puede leer estos dos test cases y entender exactamente qué se está probando — ese es el resultado tangible de aplicar BDD correctamente.

Tabla de referencia rápida

Prefijo	Rol semántico	¿Lo interpreta el motor de RF?
Given	Contexto	No — se descarta antes de buscar la keyword
When	Acción	No
Then	Resultado esperado	No
And/But	Continuación del paso anterior	No

Errores comunes

- **Esperar que Robot Framework valide el orden Given → When → Then** — no lo hace; podrías escribir Then antes de Given y el motor lo ejecutaría igual (aunque sería confuso de leer y rompería la convención).
- **Usar Set Variable en vez de Set Test Variable** cuando el valor necesita estar disponible en otro paso del mismo test — la variable quedaría fuera de alcance.
- **Escribir el nombre de la keyword distinto entre el Resource y el test case** (incluso un espacio extra) — Robot Framework no encuentra la keyword porque el texto, sin el prefijo, debe coincidir exactamente.

Puntos clave

- Los prefijos Gherkin son solo legibilidad — Robot Framework los descarta mecánicamente al resolver la keyword, sin validar su orden ni su semántica.
- Given/When/Then corresponden a contexto/acción/resultado; And/But continúan el paso anterior.
- Set Test Variable comparte una variable entre todas las keywords de un mismo test case — necesario para mantener estado entre Given/When/Then.

Autoevaluación

1. ¿Qué hace Robot Framework con el texto Given antes de buscar la keyword?
2. Si escribes los pasos en el orden Then/Given/When, ¿el motor lo rechaza?
3. ¿Qué keyword usarías para que una variable definida en el paso Given esté disponible en el paso Then?

Respuestas: 1. Lo elimina/descarta — no afecta la búsqueda de la keyword. 2. No, el motor no valida el orden; solo sería una mala práctica de legibilidad. 3. Set Test Variable.

4.3 Abstracción de capas: escenarios de negocio en lenguaje natural → keywords técnicas

Objetivos de la lección

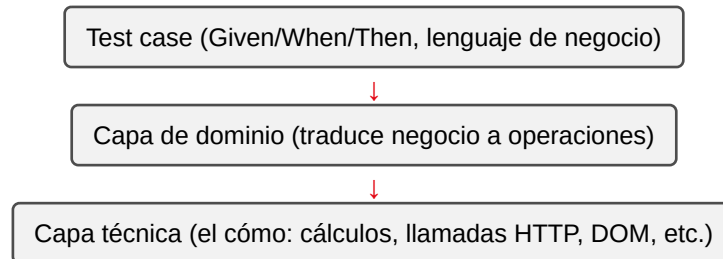
- Explicar por qué Given/When/Then no es suficiente sin separación de capas.
- Describir el patrón de tres capas: test case, dominio, técnica.
- Aplicar el criterio práctico para asignar una keyword a su capa correcta.

¿Por qué importa?

Escribir Given/When/Then no garantiza, por sí solo, que el escenario sea legible para el negocio. Si la keyword detrás de when contiene cálculos numéricos crudos o detalles de implementación, el escenario sigue siendo técnico — solo que con un prefijo Gherkin encima, lo cual puede ser peor que no usar BDD: da una falsa sensación de legibilidad de negocio.

Conceptos clave

Las tres capas



1. **Test case:** solo lenguaje de negocio, en Given/When/Then. No debería contener ningún número, ningún selector, ninguna URL.
2. **Capa de dominio:** keywords con nombres de negocio (Activar Plan Para Cliente, Un Cliente Con Crédito Suficiente Solicita Activar Un Plan) que traducen ese lenguaje a operaciones concretas, llamando a la capa técnica. Es la única capa que el test case conoce directamente.
3. **Capa técnica:** el detalle de implementación (cálculos, llamadas a una API, interacción con el DOM). El test case **nunca** debería llamar directamente a esta capa, saltándose la capa de dominio.

El criterio práctico de asignación

El criterio para decidir a qué capa pertenece una keyword es preguntarse: **¿un analista de negocio entendería esta keyword sin conocer la tecnología subyacente?** Si la respuesta es sí, es capa de dominio. Si necesita conocer detalles técnicos (un selector CSS, el nombre exacto de un endpoint, una fórmula de cálculo), es capa técnica.

Keyword	¿Capa?	Por qué
El plan queda activo	Dominio	Lenguaje de negocio puro
Calcular Resultado De Activacion	Técnica	Contiene la lógica de comparación numérica
Click Element css:#boton-activar	Técnica	Selector CSS, detalle de implementación
Un cliente con crédito suficiente solicita activar un plan	Dominio	Describe la intención de negocio, delega el cálculo a la capa técnica

El beneficio de aislar el impacto del cambio tecnológico

Esta separación también aísla el impacto de los cambios tecnológicos: migrar de REST a GraphQL para consumir el sistema de activaciones, por ejemplo, **solo modifica la capa técnica** — los escenarios Gherkin y la capa de dominio permanecen completamente intactos, porque nunca mencionaron la tecnología en primer lugar.

Ejemplo comentado

```
# Capa TÉCNICA: el detalle de cómo se calcula el resultado.
*** Keywords ***
Calcular Resultado De Activacion
    [Arguments]    ${credito}    ${costo}
    IF    ${credito} >= ${costo}
        RETURN    ACTIVO
    ELSE
        RETURN    RECHAZADO
    END

# Capa de DOMINIO: traduce el lenguaje de negocio a la capa técnica.
# Es la ÚNICA capa que el test case conoce.
*** Keywords ***
un cliente con crédito suficiente solicita activar un plan
    ${resultado}=    Calcular Resultado De Activacion    ${100}    ${50}
    Set Test Variable    ${RESULTADO}    ${resultado}

el plan queda activo
    Should Be Equal    ${RESULTADO}    ACTIVO

# Test case: SOLO lenguaje de negocio.
*** Test Cases ***
Activación De Plan Con Crédito Suficiente
    Given un cliente con crédito suficiente solicita activar un plan
    Then el plan queda activo
```

Observa que el test case no contiene ningún número, ningún detalle de cómo se calcula el resultado — todo eso vive en las capas que el test case no conoce directamente.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Cuántas capas tiene el patrón de abstracción de BDD?	Tres: test case, dominio, técnica
¿Qué capa conoce directamente el test case?	Solo la capa de dominio
¿Qué capa cambia si migras de REST a GraphQL?	Solo la capa técnica
Criterio para asignar una keyword a la capa de dominio	¿Un analista de negocio la entendería sin contexto técnico?

Errores comunes

- **Llamar directamente a una keyword técnica desde el test case**, saltándose la capa de dominio — rompe el aislamiento que el patrón busca dar.

- **Poner un cálculo o un selector dentro de una keyword de dominio** — mezcla responsabilidades; ese detalle debería vivir en la capa técnica.
- **Crear una capa de dominio “de paso” que solo reenvía la llamada sin traducir nada** — si la keyword de dominio tiene exactamente los mismos parámetros y nombre técnico que la de la capa técnica, no está cumpliendo su propósito de traducción.

Puntos clave

- Tres capas: test case (negocio) → dominio (traducción) → técnica (implementación) — el test case solo conoce la capa de dominio.
- Criterio de separación: ¿un analista de negocio entendería esta keyword sin contexto técnico?
- Un cambio tecnológico (REST → GraphQL, por ejemplo) solo debería afectar la capa técnica.

Autoevaluación

1. ¿Qué capa del patrón de abstracción debería conocer directamente un test case?
2. Si migras la capa técnica de Selenium a Playwright, ¿qué otras capas deberían cambiar?
3. Aplica el criterio práctico: ¿la keyword Verificar Que El Código De Respuesta HTTP Es 200 es de dominio o técnica?

Respuestas: 1. Solo la capa de dominio. 2. Ninguna otra — el cambio debería quedar aislado en la capa técnica. 3. Técnica — menciona un detalle de implementación (código HTTP) que un analista de negocio no necesitaría conocer.

4.4 Organización de features y escenarios para flujos comerciales agnósticos

Objetivos de la lección

- Explicar qué significa que un escenario sea “agnóstico” a la tecnología.
- Describir una estructura de carpetas para proyectos BDD.
- Aplicar tags multidimensionales en un esquema de clasificación.

¿Por qué importa?

Sin una convención de organización, un proyecto BDD que crece termina con escenarios de negocio mezclados físicamente con detalles técnicos en el mismo árbol de carpetas — perdiendo, en la práctica, el beneficio de la separación de capas que se diseñó en la lección anterior.

Conceptos clave

Qué significa “agnóstico”

Un escenario **agnóstico** describe una regla de negocio sin mencionar tecnología: “un cliente con crédito insuficiente no puede activar un plan” es agnóstico; “el endpoint POST / activaciones devuelve 402” **no lo es** — esa segunda frase pertenece a la capa técnica, no al escenario de negocio. Un escenario agnóstico sigue siendo válido aunque la tecnología subyacente cambie por completo.

Estructura de carpetas recomendada

```
tests/
├── features/
│   ├── contratos/
│   │   └── activacion_servicio.robot
│   └── facturacion/
│       └── calculo_prorrateo.robot
└── technical/
    ├── api_keywords.resource
    └── web_keywords.resource
```

Los escenarios de negocio (features/) se organizan **por dominio** (contratos, facturación, clientes) — la misma forma en que el negocio piensa su propio trabajo. Las keywords técnicas (technical/) se organizan **por tecnología** (API, web) — la forma en que un ingeniero piensa la implementación. Esta diferencia de criterio de organización no es casualidad: cada carpeta está pensada para la audiencia que más la usa.

Tags multidimensionales

Las etiquetas se vuelven más valiosas en proyectos BDD organizados, porque permiten clasificar escenarios en **varias dimensiones simultáneamente**: dominio de negocio, prioridad y canal/integración. Por ejemplo, un escenario puede llevar los tags contratos, regresion y api a la vez, y filtrarse por cualquiera de esos criterios desde la línea de comandos — útil para ejecutar, por ejemplo, “todo lo de contratos que sea regresión, sin importar el canal” o “todo lo que toque la API, sin importar el dominio de negocio”.

Dimensión	Ejemplos de tags
Dominio de negocio	contratos, facturacion, clientes
Prioridad	smoke, regresion
Canal/integración	web, api, movil

Ejemplo comentado

```
# archivo: tests/features/contratos/activacion_servicio.robot
*** Settings ***
Documentation    Feature: Activación de Servicio (agnóstico a tecnología)
Resource        ../../technical/api_keywords.resource
```


*** Test Cases ***

La Activación Es Rechazada Por Crédito Insuficiente

[Tags] contratos regresion api

Given un cliente con crédito insuficiente existe en el sistema

When el cliente solicita la activación de un plan de datos adicional

Then la activación es rechazada y se informa el motivo

Filtrar por una sola dimensión

robot --include contratos tests/

Combinar dos dimensiones (paretags, formalizado en la Sesión 9)

robot --include contratosANDregresion tests/

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Cómo se organiza features/?	Por dominio de negocio
¿Cómo se organiza technical/?	Por tecnología
¿Es agnóstico un escenario que menciona un código HTTP?	No
¿Cuántas dimensiones de tags se mencionan como ejemplo?	Tres: dominio, prioridad, canal

Errores comunes

- **Organizar features/ por tecnología en vez de por dominio** — pierde el sentido de la separación; el negocio no piensa en “API” o “web”, piensa en “contratos” o “facturación”.
- **Mencionar un detalle técnico dentro de un escenario Gherkin** (“el campo status_code debe ser 200”) — rompe el carácter agnóstico del escenario.
- **No usar tags multidimensionales**, dejando solo un tag por test case — limita drásticamente la flexibilidad de filtrado en proyectos grandes.

Puntos clave

- Un escenario agnóstico describe una regla de negocio, no un detalle técnico — sigue siendo válido aunque cambie la tecnología subyacente.
- Las carpetas separan features/ (negocio, por dominio) de technical/ (tecnología, por tipo de implementación).
- Los tags multidimensionales (dominio + prioridad + canal) facilitan el filtrado preciso de suites grandes.

Autoevaluación

1. ¿Por qué features/ se organiza por dominio de negocio y no por tecnología?

2. ¿Es agnóstico el escenario “el cliente recibe un SMS de confirmación”? ¿Y “la API responde con el header X-Confirmacion”?
3. Menciona las tres dimensiones de tags descritas en esta lección.

Respuestas: 1. Porque esa es la forma en que el negocio piensa su propio trabajo — facilita que negocio navegue y entienda la organización. 2. El primero es agnóstico (describe un resultado de negocio); el segundo no lo es (menciona un detalle técnico de implementación). 3. Dominio de negocio, prioridad, canal/integración.

Resumen del capítulo

BDD reduce la brecha TI-Negocio describiendo comportamiento en lenguaje natural estructurado (Gherkin), acordado primero en la dinámica de las Tres Amigos — particularmente valioso en telecomunicaciones por sus reglas regulatorias. Robot Framework no requiere sintaxis adicional: los prefijos Given/When/Then/And/But son simple texto descartado mecánicamente al resolver la keyword; `Set Test Variable` comparte estado entre los pasos de un mismo test. El valor real de BDD aparece al separar en tres capas — test case, dominio, técnica —, con el criterio de “¿lo entendería un analista de negocio?” para asignar cada keyword a su capa, lo que aísla el impacto de cambios tecnológicos. Los escenarios agnósticos y una organización de carpetas por dominio de negocio (no por tecnología), reforzada con tags multidimensionales, sostienen esta separación a medida que el proyecto crece.

Referencias bibliográficas

- BDD con Gherkin en RF: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#behavior-driven-style>
- Introducción a BDD (Cucumber/Gherkin, referencia conceptual): <https://cucumber.io/docs/bdd/>

Capítulo 5 — RF Avanzado: Buenas Prácticas, Data-Driven y Extensión con Python

Información general

Este capítulo cierra los fundamentos del lenguaje antes de pasar a los dominios de aplicación (web, API, RPA). Cubre cómo diseñar keywords reutilizables a escala, cómo evitar escribir un test case por cada combinación de datos, y cómo extender Robot Framework cuando las librerías nativas no alcanzan — con ejemplos de código completos en cada lección, no solo reglas abstractas.

Lecciones de este capítulo:

- 5.1 — Diseño de keywords reutilizables: separación por capas y convenciones de nombres
 - 5.2 — Data-driven testing: DataDriver con CSV/JSON y trazabilidad con tags
 - 5.3 — Librerías personalizadas en Python: keywords, tipos, documentación y decoradores
 - 5.4 — Empaquetado, versionado y repositorios compartidos de librerías RF
-

5.1 Diseño de keywords reutilizables: separación por capas y convenciones de nombres

Objetivos de la lección

- Aplicar el patrón de tres capas (visto en BDD) a keywords reutilizables en general.
- Adoptar convenciones de nombres consistentes.
- Reconocer cuándo una keyword debería dividirse en otras más pequeñas.

¿Por qué importa?

Un proyecto de automatización que crece sin convenciones claras de diseño de keywords se vuelve, en la práctica, ilegible para cualquiera que no sea su autor original — incluso para el propio autor, meses después.

Conceptos clave

El mismo patrón de capas, generalizado

El patrón de separación por capas que conociste en el Capítulo 4 (test case → dominio → técnica) no es exclusivo de BDD: aplica a **cualquier** proyecto de automatización que quiera escalar, con o sin Gherkin. Una keyword de **capa técnica** (Hacer Clic En Boton Login) interactúa directamente con la tecnología; una keyword de **capa de acción**

combina varias técnicas en un paso coherente (Iniciar Sesión, que internamente llama a “escribir usuario”, “escribir contraseña” y “hacer clic en login”); una keyword de **capa de negocio** describe el resultado deseado en lenguaje de dominio (El Usuario Está Autenticado).

Convenciones de nombres que reducen fricción

Las convenciones de nombres reducen la fricción de un equipo que crece:

- **Title Case** para keywords: Verificar Que El Usuario Está Autenticado, no verificar_usuario ni VERIFICAR_USUARIO.
- **Empezar con un verbo de acción**: Crear, Verificar, Calcular, Obtener — comunica inmediatamente qué hace la keyword.
- **Evitar abreviaturas ambiguas**: Calcular Costo Total Del Plan es preferible a Calc Costo Tot Plan — el ahorro de caracteres no compensa la pérdida de claridad.
- **Nombrar por el resultado, no por el mecanismo**, en la capa de negocio: El Plan Queda Activo en vez de Verificar Status Code 200.

Estas convenciones no son estéticas — afectan directamente qué tan rápido alguien nuevo en el equipo entiende una suite ajena, y qué tan fácil es para el Language Server (Capítulo 1) ofrecer autocompletado útil.

Cuándo dividir una keyword

Una señal práctica de que una keyword debería dividirse: si su [Documentation] necesita la palabra “y” más de una vez para describir lo que hace (“valida el formato del email y registra el cliente y envía la confirmación”), probablemente está haciendo demasiado y debería separarse en keywords más pequeñas, cada una con una sola responsabilidad, orquestadas desde una keyword de nivel superior.

Ejemplo comentado

```
*** Keywords ***
# Capa técnica – interacción directa con SeleniumLibrary
Escribir En Campo Usuario
    [Arguments]    ${usuario}
    Input Text      id:username    ${usuario}

Escribir En Campo Contraseña
    [Arguments]    ${password}
    Input Password  id:password    ${password}

Hacer Clic En Botón Login
    Click Button    css:button[type='submit']

# Capa de acción – combina varias técnicas en un paso coherente
Iniciar Sesión
    [Arguments]    ${usuario}    ${password}
    Escribir En Campo Usuario    ${usuario}
```

```
Escribir En Campo Contraseña    ${password}
Hacer Clic En Botón Login
```

Capa de negocio – lenguaje de dominio, sin ningún detalle técnico

El Usuario Está Autenticado Correctamente

```
[Arguments]    ${usuario}    ${password}
Iniciar Sesión    ${usuario}    ${password}
Wait Until Element Is Visible    css:#bienvenida
```

Cada capa tiene exactamente una responsabilidad, y un test case solo debería llamar a la capa de negocio (El Usuario Está Autenticado Correctamente), nunca directamente a Escribir En Campo Usuario.

Tabla de referencia rápida

Capa	Ejemplo	Conoce detalles técnicos
Técnica	Escribir En Campo Usuario	Sí (selector CSS/id)
Acción	Iniciar Sesión	Indirectamente (llama a la técnica)
Negocio	El Usuario Está Autenticado Correctamente	No

Errores comunes

- **Nombrar keywords por el mecanismo en la capa de negocio** (Verificar Response 200 en vez de El Plan Queda Activo) — mezcla niveles de abstracción.
- **Crear una keyword que hace 5 cosas distintas sin dividirla** — dificulta la reutilización parcial (¿qué pasa si solo necesitas la mitad de esa lógica en otro escenario?).
- **Usar snake_case o mayúsculas completas para nombres de keywords**, rompiendo la convención Title Case del resto del proyecto.

Puntos clave

- El patrón de tres capas (técnica → acción → negocio) aplica a cualquier suite, no solo a BDD.
- Las convenciones de nombres (Title Case, verbo inicial, sin abreviaturas ambiguas) reducen la curva de entendimiento de un proyecto.
- Si la documentación de una keyword necesita varios “y” para describirla, probablemente debería dividirse.

Autoevaluación

1. ¿En qué capa debería vivir el detalle de un selector CSS?
2. ¿Por qué Calcular Costo Tot Plan es una convención de nombre desaconsejada?
3. ¿Qué señal indica que una keyword probablemente hace demasiado y debería dividirse?

Respuestas: 1. La capa técnica. 2. Usa abreviaturas ambiguas que reducen la claridad sin un ahorro real significativo. 3. Que su documentación necesite la palabra “y” varias veces para describir lo que hace.

5.2 Data-driven testing: DataDriver con CSV/JSON y trazabilidad con tags

Objetivos de la lección

- Explicar el problema que resuelve el testing data-driven.
- Describir cómo DataDriver genera tests desde un archivo externo.
- Configurar correctamente `dialect` y `encoding` según el origen de los datos.

¿Por qué importa?

Cuando la misma lógica de prueba debe validarse contra docenas de combinaciones de datos, escribir un test case por combinación duplica esfuerzo y dificulta el mantenimiento: un cambio en la lógica obliga a tocar N test cases en vez de uno.

Conceptos clave

El problema que resuelve

El **testing data-driven** separa la lógica (una sola keyword “plantilla”) de los datos (un archivo externo, CSV o JSON). En vez de escribir:

```
*** Test Cases ***
Activar Plan Cliente Con Credito 100 Y Costo 50
...
Activar Plan Cliente Con Credito 0 Y Costo 50
...
Activar Plan Cliente Con Credito 200 Y Costo 150
...
```

con la lógica repetida (o casi repetida) en cada uno, defines la lógica **una sola vez** y dejas que los datos vengan de afuera.

Cómo DataDriver genera los tests

DataDriver (librería de comunidad, se instala con `pip install robotframework-datadrivers`) implementa este patrón con Test Template: declaras una keyword plantilla en Settings, y DataDriver genera **un test case por cada fila** del archivo de datos, usando los nombres de columna (con el formato `${nombre_argumento}`) para mapear a los argumentos de la keyword.

```
*** Settings ***
Library           DataDriver      ${CURDIR}/casos.csv      dialect=excel
encoding=utf_8
Test Template     Verificar Resultado De Activacion
```

```

*** Test Cases ***
Caso De Ejemplo      # Este test "plantilla" es reemplazado por DataDriver
*** Test Cases ***,${{credito}},${costo},${resultado_esperado},[Tags]
Cliente con crédito suficiente,100,50,ACTIVO,"smoke,positivo"
Cliente sin crédito,0,50,RECHAZADO,"regresion,negativo"

```

Un detalle técnico que casi siempre necesita ajustarse

DataDriver tiene valores por defecto pensados para configuraciones europeas: `dialect='Excel-EU'` (separador `;`, no `,`) y `encoding='cp1252'` (no UTF-8). En la práctica, casi siempre necesitas sobrecribir ambos parámetros explícitamente si tu archivo usa comas como separador y contiene caracteres con tilde o ñ — de lo contrario obtienes errores de “argumento no asignado” (porque el separador no coincide y las columnas no se detectan correctamente) o texto corrupto (mojibake, por ejemplo `crÃ©dito` en vez de crédito, por una codificación incorrecta).

Trazabilidad con la columna [Tags]

La columna especial [Tags] en el archivo de datos permite asignar tags **por fila** (con varios tags separados por coma dentro de la misma celda, entre comillas si hay coma en el contenido). Esto entrega **trazabilidad**: cada test generado lleva sus propios tags, visibles en las estadísticas “by tag” de `report.html`, sin escribir código adicional para segmentarlos.

Una limitación documentada del filtrado por CLI

El filtrado por `--include/--exclude` de la línea de comandos (que aprendiste en el Capítulo 2) ocurre **antes** de que DataDriver genere los tests reales — actúa sobre el test case “plantilla” original, que no tiene tags propios. Por eso, `--include` de CLI normalmente **no filtra correctamente** los tests generados por DataDriver. La forma correcta de filtrar es usando los parámetros propios de la librería (`Library DataDriver ... include=...`) en vez del flag de CLI, o revisar la segmentación directamente en `report.html` sin intentar filtrar la ejecución.

Ejemplo comentado

```

*** Settings ***
Documentation      Suite data-driven con DataDriver, segmentada por tags.
Library            DataDriver      ${CURDIR}/../data/casos_activacion.csv
dialect=excel      encoding=utf_8
Test Template      Verificar Resultado De Activacion

*** Test Cases ***
Caso De Ejemplo De Activación

*** Keywords ***

```

Verificar Resultado De Activacion

```
[Arguments]    ${credito}    ${costo}    ${resultado_esperado}
${resultado}=  Calcular Resultado De Activacion    ${credito}    ${costo}
Should Be Equal    ${resultado}    ${resultado_esperado}
```

Calcular Resultado De Activacion

```
[Arguments]    ${credito}    ${costo}
${credito_num}=    Convert To Integer    ${credito}
${costo_num}=      Convert To Integer    ${costo}
IF    ${credito_num} >= ${costo_num}
    RETURN    ACTIVO
ELSE
    RETURN    RECHAZADO
END
```

Con un CSV de 4 filas, esta suite genera 4 test cases reales — visibles individualmente en `report.html` con sus propios nombres (tomados de la primera columna del CSV) y tags.

Tabla de referencia rápida

Parámetro de Library DataDriver	Valor por defecto	Valor típico necesario
dialect	Excel-EU (separador ;)	excel (separador ,)
encoding	cp1252	utf_8
Pregunta del examen RFCP (estilo)	Respuesta	
¿Qué genera DataDriver por cada fila del archivo de datos?	Un test case independiente	
¿Qué columna especial define tags por fila?	[Tags]	
¿ <code>--include</code> de CLI filtra correctamente tests generados por DataDriver?	No, hay una limitación documentada	

Errores comunes

- **No especificar `dialect=excel` con un CSV separado por comas** — produce errores de argumento no asignado, porque DataDriver intenta separar por ; y no encuentra las columnas esperadas.
- **No especificar `encoding=utf_8` con texto en español** — produce caracteres corruptos en los nombres de los tests generados.
- **Intentar filtrar tests de DataDriver con `--include` de CLI** y sorprenderse de que no funciona como con una suite normal — es una limitación documentada, no un bug del proyecto.

Puntos clave

- DataDriver separa lógica (keyword plantilla) de datos (archivo CSV/JSON externo), generando un test por fila.
- Los valores por defecto de `dialect` y `encoding` son europeos; casi siempre se sobreescriben explícitamente.
- `--include/--exclude` de CLI tiene una limitación documentada con tests generados dinámicamente; usa la segmentación de `report.html` o los parámetros propios de la librería.

Autoevaluación

1. ¿Qué dos parámetros de Library DataDriver casi siempre necesitas sobreescribir con un CSV típico en español?
2. ¿Qué error produce omitir `dialect=excel` con un CSV separado por comas?
3. ¿Por qué `--include smoke` puede no funcionar como esperas con una suite que usa DataDriver?

Respuestas: 1. `dialect` (a `excel`) y `encoding` (a `utf_8`). 2. Error de “argumento no asignado”, porque las columnas no se detectan correctamente con el separador equivocado. 3. Porque el filtrado de CLI ocurre antes de que DataDriver genere los tests reales — actúa sobre el test plantilla, que no tiene tags propios.

5.3 Librerías personalizadas en Python: keywords, tipos, documentación y decoradores

Objetivos de la lección

- Explicar cómo una función Python se convierte en keyword.
- Aplicar el decorador `@keyword` para personalizar nombre y tags.
- Aplicar la práctica de probar con `pytest` antes de integrar a Robot Framework.

¿Por qué importa?

Cuando las librerías nativas y de comunidad no cubren una necesidad específica del dominio, Robot Framework permite escribir keywords directamente en Python — sin esto, ciertos problemas (validaciones de dominio muy específicas, cálculos complejos) serían torpes de expresar solo con keywords `.robot`.

Conceptos clave

La conversión automática `snake_case` → `Title Case`

Cualquier función pública (sin guion bajo al inicio) de un módulo `.py` importado con `Library` se convierte automáticamente en keyword. El nombre se traduce de `snake_case` a `Title Case` Con Espacios: una función `validar_numero_telefono` se convierte en la keyword `Validar Numero Telefono`.

El decorador @keyword: control explícito

El decorador @keyword, de robot.api.deco, permite personalizar ese comportamiento por defecto:

```
from robot.api.deco import keyword

@keyword("Validar Numero De Telefono Guatemalteco")
def validar_numero_telefono(numero: str) -> bool:
    ...
```

- **Cambiar el nombre visible:** sin el decorador, esta función se vería como Validar Numero Telefono (sin “De” ni “Guatemalteco”) — el decorador da control total sobre el nombre que verá quien escriba .robot.
- **Asignar tags automáticos:** @keyword("...", tags=["facturacion"]) agrega ese tag a cualquier test que use la keyword, sin que el autor del test tenga que declararlo manualmente.
- **Declarar tipos de argumento:** combinado con *type hints* de Python (numero: str), Robot Framework puede validar o convertir automáticamente el tipo del argumento recibido.

El docstring como documentación dual

El **docstring** de la función cumple un rol doble: es la documentación del código Python para cualquier desarrollador que lo lea, y es también la **documentación oficial de la keyword** dentro de Robot Framework, visible en log.html y en el autocompletado de los IDEs con Language Server (Capítulo 1). Esto significa que escribir un buen docstring no es solo buena práctica de Python — tiene un beneficio directo y visible para quien use la keyword desde .robot.

```
def calcular_costo_total(costo_base: float, impuesto_porcentaje: float = 12.0) -> float:
    """Calcula el costo total de un plan agregando el impuesto.

    Example:
    | ${total}=    Calcular Costo Total Del Plan    100
    | Should Be Equal As Numbers    ${total}    112.0
    """
    ...
```

El bloque Example: dentro del docstring, con la sintaxis de tabla de Robot Framework, aparece formateado correctamente en la documentación generada — una convención que vale la pena adoptar.

Por qué probar con pytest antes de integrar

Una práctica recomendada, no solo para librerías Python sino para cualquier lógica no trivial: **probar la función con pytest antes de integrarla a Robot Framework**. Esto separa dos preguntas distintas — ¿la lógica Python es correcta? y ¿la integración con Robot Framework funciona? — facilitando enormemente el diagnóstico cuando algo falla:

si el test de pytest pasa pero el test de Robot Framework falla, el problema está en la integración (nombres de keyword, tipos de argumento), no en la lógica.

Ejemplo comentado

```
"""Librería Python personalizada para Robot Framework."""
from __future__ import annotations
import re
from robot.api.deco import keyword

NUMERO_TELEFONO_GT_PATTERN = re.compile(r"^[2-7]\d{7}$")

@keyword("Validar Numero De Telefono Guatemalteco")
def validar_numero_telefono(numero: str) -> bool:
    """Valida que un número guatemalteco tenga 8 dígitos y empiece
    entre 2 y 7 (rangos asignados por la Superintendencia de
    Telecomunicaciones de Guatemala).
    """
    return bool(NUMERO_TELEFONO_GT_PATTERN.fullmatch(numero))

@keyword("Calcular Costo Total Del Plan", tags=["facturacion"])
def calcular_costo_total(costo_base: float, impuesto_porcentaje: float = 12.0) -> float:
    """Calcula el costo total de un plan agregando el impuesto (IVA de
    Guatemala = 12% por defecto)."""
    if costo_base < 0:
        raise ValueError(f"El costo base no puede ser negativo: {costo_base}")
    total = costo_base * (1 + impuesto_porcentaje / 100)
    return round(total, 2)

# Probado primero con pytest, ANTES de usarlo desde Robot Framework
def test_calcular_costo_total_con_impuesto_por_defecto():
    assert calcular_costo_total(100) == 112.0

def test_calcular_costo_total_con_costo_negativo_lanza_value_error():
    with pytest.raises(ValueError):
        calcular_costo_total(-50)
```

Tabla de referencia rápida

Elemento Python	Efecto en Robot Framework
Función pública (sin <code>_</code> inicial) en un <code>.py</code> importado con <code>Library</code>	Se convierte automáticamente en keyword (<code>snake_case</code> → <code>Title Case</code>)
<code>@keyword("Nombre Personalizado")</code>	Cambia el nombre visible de la keyword
<code>@keyword(tags=[...])</code>	Asigna tags automáticos a tests que usan esa keyword
Docstring de la función	Documentación oficial de la keyword en <code>log.html</code>

Errores comunes

- **Nombrar la función con un guion bajo inicial** (`_validar_numero`) pensando que se convertirá en keyword igual — las funciones “privadas” (con `_`) **no** se exponen como keywords.
- **Omitir el docstring** pensando que es solo para Python — se pierde la documentación visible desde Robot Framework, afectando a cualquiera que use la keyword sin leer el código fuente.
- **Saltar la prueba con pytest** e ir directo a probar desde Robot Framework — dificulta saber si un fallo es de lógica o de integración.

Puntos clave

- Funciones públicas en un `.py` se convierten automáticamente en keywords (`snake_case` → `Title Case`); las funciones con `_` inicial no se exponen.
- `@keyword` personaliza nombre, tags y tipos de argumento, dando control explícito sobre el comportamiento por defecto.
- El docstring es documentación de Python **y** de la keyword en Robot Framework simultáneamente.
- Probar con `pytest` antes de integrar separa el error de lógica del error de integración.

Autoevaluación

1. ¿Qué pasa si una función Python se llama `_calcular_total` (con guion bajo inicial)?
2. ¿Para qué sirve el parámetro `tags=` del decorador `@keyword`?
3. ¿Por qué es recomendable probar una función Python con `pytest` antes de integrarla a Robot Framework?

Respuestas: 1. No se expone como keyword — las funciones con guion bajo inicial se consideran privadas. 2. Asigna automáticamente esos tags a cualquier test que use esa keyword, sin que el autor del test los declare manualmente. 3. Para separar si un fallo es de lógica (`pytest`) o de integración (Robot Framework), facilitando el diagnóstico.

5.4 Empaquetado, versionado y repositorios compartidos de librerías RF

Objetivos de la lección

- Describir cómo empaquetar una librería Python como artefacto distribuible.
- Aplicar versionado semántico (SemVer) a una librería propia.
- Diferenciar índices públicos y privados de paquetes Python.

¿Por qué importa?

Sin empaquetado ni versionado, compartir una librería Python entre varios proyectos de un equipo significa copiar y pegar archivos manualmente — perdiendo trazabilidad de qué versión usa cada proyecto y arriesgando que una corrección en un proyecto no llegue nunca a los demás.

Conceptos clave

Por qué empaquetar

Cuando un equipo de automatización crece más allá de un solo proyecto, las librerías Python personalizadas se benefician de convertirse en **paquetes distribuibles** (.whl), instalables con `pip install` igual que cualquier librería de PyPI, en lugar de copiarse manualmente entre proyectos. Un paquete mínimo necesita una estructura estándar de Python:

```
mi_libreria_rf/  
├── src/  
│   └── MiLibreriaRF/  
│       ├── __init__.py  
│       └── keywords.py  
└── pyproject.toml
```

El archivo `pyproject.toml` declara metadatos del paquete (nombre, versión, dependencias) y se construye con herramientas como `build` y `twine` para publicarlo en un índice de paquetes.

Versionado semántico (SemVer)

El **versionado semántico** (MAJOR.MINOR.PATCH) comunica el impacto de un cambio sin necesidad de leer el changelog completo:

Componente	Cuándo incrementarlo	Ejemplo
PATCH	Correcciones que no rompen nada existente	1.2.0 → 1.2.1
MINOR	Keywords nuevas que tampoco rompen lo existente	1.2.1 → 1.3.0
MAJOR	Cambios incompatibles (renombrar/eliminar una keyword usada por otros)	1.3.0 → 2.0.0

Este mismo curso usa SemVer para versionar cada sesión cerrada en sus repositorios — la convención no es exclusiva de librerías Python, es un lenguaje compartido para comunicar el alcance de un cambio en cualquier artefacto de software.

Índices públicos vs. privados

PyPI (el índice público estándar de Python) es apropiado para librerías open source que cualquiera puede usar. Para librerías internas de una empresa — que contienen lógica de negocio o credenciales de configuración que no deberían ser públicas — existen **índices privados** (GitLab Package Registry, Azure Artifacts, JFrog Artifactory) que funcionan **exactamente igual** desde la perspectiva de `pip install`, solo apuntando a una URL distinta mediante el parámetro `--index-url`.

Ejemplo comentado

```
# pyproject.toml – configuración mínima para empaquetar una librería RF
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "mi-libreria-rf"
version = "1.2.0"
dependencies = ["robotframework>=7.0"]

# Construir el paquete distribuible (.whl)
python -m build

# Publicar en un índice privado de la empresa
twine upload --repository-url https://gitlab.empresa.com/api/v4/projects/123/packages/pypi dist/*

# Instalar desde ese índice privado en otro proyecto
pip install mi-libreria-rf --index-url https://gitlab.empresa.com/api/v4/projects/123/packages/pypi/simple
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué incrementa SemVer ante un cambio incompatible?	MAJOR
¿Qué incrementa SemVer ante una corrección que no rompe nada?	PATCH
¿Qué archivo declara metadatos de un paquete Python moderno?	pyproject.toml
¿Un índice privado usa una sintaxis distinta de <code>pip install</code> ?	No, solo cambia la URL del índice

Errores comunes

- **Incrementar solo PATCH al renombrar una keyword existente** — eso es un cambio incompatible (MAJOR), porque rompe a cualquier proyecto que ya use el nombre anterior.
- **Publicar una librería con lógica de negocio interna en el índice público de PyPI** por desconocer la opción de índices privados.
- **Copiar y pegar el código de una librería entre proyectos** en vez de empaquetarla — pierde trazabilidad de versión y dificulta propagar correcciones.

Puntos clave

- Empaquetar una librería propia la convierte en instalable con `pip`, evitando copiar archivos entre proyectos.
- SemVer (MAJOR.MINOR.PATCH) comunica el impacto de un cambio: parche, funcionalidad nueva, o cambio incompatible.
- Los índices privados permiten compartir librerías internas sin publicarlas en PyPI público, usando la misma sintaxis de `pip install` con una URL distinta.

Autoevaluación

1. Si renombas una keyword existente que otros proyectos ya usan, ¿qué componente de SemVer deberías incrementar?
2. ¿Qué herramienta(s) se mencionan para construir y publicar un paquete Python?
3. ¿Cambia la sintaxis de `pip install` al usar un índice privado en vez de PyPI público?

Respuestas: 1. MAJOR — es un cambio incompatible. 2. `build` (para construir) y `twine` (para publicar). 3. No, solo cambia la URL del índice (`--index-url`), la sintaxis del comando es la misma.

Resumen del capítulo

El diseño de keywords reutilizables se beneficia del mismo patrón de capas que BDD (técnica → acción → negocio), más convenciones de nombres consistentes y la disciplina de dividir keywords que hacen demasiado. El testing data-driven (DataDriver) separa lógica de datos, generando un test por fila de un CSV/JSON externo — con cuidado explícito en los valores por defecto de `dialect/encoding` (europeos) y en la limitación de filtrado por CLI con tests generados dinámicamente. Las librerías Python extienden Robot Framework con funciones convertidas automáticamente en keywords (`snake_case` → Title Case), personalizables con `@keyword`, documentadas con docstrings que sirven doble propósito, e idealmente probadas con `pytest` antes de integrarse. Para escalar a varios proyectos, esas librerías se empaquetan con `pyproject.toml`, se versionan con SemVer, y se distribuyen vía índices públicos o privados sin cambiar la sintaxis de instalación.

Referencias bibliográficas

- Creating test libraries: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#creating-test-libraries>
- robotframework-datadriver: <https://github.com/Snooz82/robotframework-datadriver>
- Decorador `@keyword`: <https://robot-framework.readthedocs.io/en/stable/autodoc/robot.api.html#robot.api.deco.keyword>
- Versionado semántico: <https://semver.org/lang/es/>

Capítulo 6 — Automatización Web con SeleniumLibrary

Información general

Este capítulo aplica todo lo aprendido sobre sintaxis, variables y manejo de fallas a un dominio concreto: la automatización de interfaces web, usando SeleniumLibrary — la librería de comunidad más usada para este propósito en Robot Framework. Cada lección incluye ejemplos de código reales, incluyendo una lección aprendida del desarrollo de este mismo curso sobre un error de sincronización entre tests.

Lecciones de este capítulo:

- 6.1 — Configuración de navegador, Selenium Manager, sesiones y opciones
 - 6.2 — Localizadores CSS y XPath: selección robusta, acciones e interacción con formularios
 - 6.3 — Sincronización: Wait Until, timeouts, elementos dinámicos y estabilidad
 - 6.4 — Evidencias: screenshots automáticos, manejo de alertas, frames e iframes
 - 6.5 — Patrón Page Object en RF: capa de keywords por página y mantenibilidad
-

6.1 Configuración de navegador, Selenium Manager, sesiones y opciones

Objetivos de la lección

- Explicar el rol de SeleniumLibrary como puente hacia el navegador.
- Describir cómo Selenium Manager resuelve la gestión de drivers.
- Configurar sesiones de navegador, incluyendo el modo headless.

¿Por qué importa?

Antes de Selenium Manager, sincronizar la versión del navegador con la versión de su driver era una fuente constante de fricción y de errores difíciles de diagnosticar (“el navegador no abre” sin mensaje claro). Entender qué resuelve automáticamente la herramienta moderna evita perder tiempo en un problema que ya no debería ocurrir.

Conceptos clave

SeleniumLibrary como puente

SeleniumLibrary conecta keywords .robot con la API de **Selenium WebDriver**, que a su vez controla un navegador real (o headless, sin interfaz visual) a través de un *driver* específico — ChromeDriver para Chrome, GeckoDriver para Firefox. La librería traduce cada keyword (Click Element, Input Text) en llamadas equivalentes de la API de Selenium.

Selenium Manager: fin de la gestión manual de drivers

Históricamente, sincronizar la versión del navegador instalado con la versión exacta del driver correspondiente era responsabilidad manual del equipo — y un navegador que se autoactualiza (como Chrome) podía romper esa sincronización sin aviso. Desde **Selenium 4.6, Selenium Manager** resuelve esto automáticamente: detecta la versión del navegador instalado en la máquina y descarga el driver compatible en segundo plano, sin configuración manual ni necesidad de instalar chromedriver por separado.

Abrir una sesión: Open Browser y sus opciones

```
Open Browser    https://miempresa.com/login    headlesschrome
```

Open Browser abre una sesión — identificada internamente por un alias si necesitas manejar varias sesiones simultáneas (por ejemplo, para probar un flujo entre dos usuarios distintos al mismo tiempo). El segundo argumento indica el navegador y modo: chrome, firefox, headlesschrome (sin interfaz visual). El **modo headless** es esencial para ejecutar en pipelines de CI/CD, donde no hay pantalla disponible — y suele ser más rápido que el modo con interfaz, porque no renderiza gráficamente nada.

Opciones adicionales del navegador

Para necesidades más específicas (tamaño de ventana, perfil de usuario, deshabilitar notificaciones), Open Browser acepta un parámetro options con la sintaxis de Selenium directamente:

```
Open Browser    ${URL}    chrome    options=add_argument("--window-size=1920,1080")
```

Ejemplo comentado

```
*** Settings ***
```

```
Library         SeleniumLibrary
```

```
*** Test Cases ***
```

```
Abrir El Portal En Modo Headless Para CI/CD
```

```
    [Documentation]    headlesschrome es el modo recomendado para  
    ...                ejecución automática sin interfaz gráfica.
```

```
    Open Browser    https://the-internet.herokuapp.com/login    headlesschrome  
    Title Should Be    The Internet  
    Close Browser
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué API usa SeleniumLibrary internamente?	Selenium WebDriver
¿Desde qué versión de Selenium existe Selenium Manager?	4.6
¿Qué modo de navegador es esencial para CI/CD?	Headless (headlesschrome)
¿Hay que instalar chromedriver manualmente con Selenium 4.6+?	No, Selenium Manager lo gestiona automáticamente

Errores comunes

- **Instalar manualmente chromedriver “por si acaso”** con Selenium 4.6+ — innecesario, y puede generar conflicto de versiones si la instalación manual queda desactualizada respecto al navegador.
- **Olvidar el modo headless en un pipeline de CI/CD** sin pantalla disponible — el navegador falla al abrir, con un error que no siempre comunica claramente la causa real.
- **Abrir múltiples sesiones sin gestionar el alias correctamente** — sin un alias claro, es fácil terminar interactuando con la sesión equivocada cuando hay varias abiertas.

Puntos clave

- SeleniumLibrary traduce keywords `.robot` a llamadas de Selenium WebDriver, que a su vez controla un navegador real a través de un driver.
- Selenium Manager (desde 4.6) descarga el driver correcto automáticamente, eliminando la sincronización manual de versiones.
- El modo headless (headlesschrome) es esencial para ejecutar en CI/CD sin interfaz gráfica, y suele ser más rápido.

Autoevaluación

1. ¿Qué problema resolvía manualmente un equipo antes de Selenium Manager?
2. ¿Qué modo de Open Browser usarías en un pipeline de CI/CD sin pantalla?
3. Verdadero o falso: con Selenium 4.6+, sigue siendo necesario instalar chromedriver aparte.

Respuestas: 1. Sincronizar la versión del navegador con la versión exacta de su driver. 2. headlesschrome (o el equivalente headless del navegador elegido). 3. Falso — Selenium Manager lo gestiona automáticamente.

6.2 Localizadores CSS y XPath: selección robusta e interacción con formularios

Objetivos de la lección

- Diferenciar localizadores CSS y XPath.
- Aplicar las keywords de interacción según el tipo de control HTML.
- Diseñar localizadores robustos ante cambios menores de la interfaz.

¿Por qué importa?

Un localizador frágil (por ejemplo, basado en la posición exacta de un elemento en el DOM) se rompe con cualquier cambio menor de diseño, generando falsos fallos que erosionan la confianza del equipo en la suite de automatización.

Conceptos clave

CSS vs. XPath: cuándo usar cada uno

Un **localizador** es la dirección que usa Selenium para encontrar un elemento en el DOM. Los dos lenguajes más usados:

- **CSS**: más legible, generalmente más rápido de evaluar, cubre la mayoría de los casos prácticos (selección por id, clase, atributo, jerarquía).
- **XPath**: más expresivo — puede navegar **hacia arriba** en el árbol del DOM (un padre desde un hijo) y seleccionar elementos **por su texto visible** (`//button[text()='Enviar']`), algo que CSS no puede hacer de forma nativa.

SeleniumLibrary distingue el lenguaje con un prefijo explícito: `css:` o `xpath:`. Si omites el prefijo, Robot Framework intenta inferir el formato (generalmente asumiendo XPath si empieza con `//` o `()`), pero declarar el prefijo explícitamente es la práctica recomendada — elimina la ambigüedad.

Necesidad	Lenguaje recomendado
Seleccionar por id, clase o atributo simple	CSS (<code>css:#id</code> , <code>css:.clase</code>)
Seleccionar por texto visible del elemento	XPath (<code>xpath://button[text()='Enviar']</code>)
Navegar hacia un elemento padre	XPath (<code>xpath://input[@id='x']/..</code>)
Selección jerárquica simple (hijo directo)	Ambos funcionan; CSS suele ser más legible

Keywords de interacción por tipo de control

Cada tipo de control HTML tiene su keyword específica para interactuar con él:

Control HTML	Keyword
Campo de texto	Input Text
Campo de contraseña	Input Password (oculta el valor en los logs)
Botón	Click Button
Cualquier elemento clicable genérico	Click Element
Lista desplegable (<select>)	Select From List By Label / By Value / By Index
Checkbox	Select Checkbox / Unselect Checkbox

Input Password existe como keyword separada de Input Text por una razón concreta: oculta el valor real en log.html, evitando que una contraseña de prueba quede expuesta en texto plano en un artefacto que puede compartirse o almacenarse.

Diseñar localizadores robustos

Un localizador robusto prefiere atributos estables (id, data-testid si la aplicación lo expone para pruebas) sobre estructura posicional (div > div > span:nth-child(3)), que se rompe con cualquier cambio de maquetación que no afecte la funcionalidad real de la página.

Ejemplo comentado

*** Test Cases ***

Inicio De Sesión Con CSS Y XPath Combinados

```
[Documentation]    Demuestra ambos lenguajes de localización en un
...                mismo flujo, eligiendo el más apropiado para cada caso.
Open Browser      https://the-internet.herokuapp.com/login    headlesschrome

# CSS: selección simple por id
Input Text        id:username    tomsmith
Input Password    id:password    SuperSecretPassword!

# CSS: selección por atributo
Click Button      css:button[type='submit']

# XPath: selección por texto visible (CSS no puede hacer esto de forma
nativa)
Page Should Contain Element    xpath://h2[contains(text(), 'Secure Area')]

Close Browser
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué keyword oculta el valor en los logs?	Input Password
¿Qué lenguaje permite seleccionar por texto visible?	XPath
¿Qué prefijo indica a SeleniumLibrary usar CSS?	css:
¿Qué keyword interactúa con un <select> por su texto visible?	Select From List By Label

Errores comunes

- **Usar Input Text para un campo de contraseña** — funciona técnicamente, pero el valor queda expuesto en texto plano en `log.html`; usa `Input Password`.
- **Construir localizadores basados en posición exacta** (`div:nth-child(4) > span`) en vez de atributos estables — se rompe con cambios de maquetación irrelevantes para la funcionalidad.
- **Omitir el prefijo `css:/xpath`**: confiando en la inferencia automática — funciona la mayoría de las veces, pero declarar el prefijo explícitamente elimina cualquier ambigüedad y hace el código más legible.

Puntos clave

- CSS es más legible y rápido para la mayoría de los casos; XPath es más expresivo (navega hacia arriba, busca por texto visible).
- El prefijo (`css:`, `xpath:`) le indica a SeleniumLibrary qué motor de localización usar; declararlo explícitamente es la práctica recomendada.
- Cada tipo de control HTML tiene su propia keyword de interacción — `Input Password` oculta el valor en los logs por una razón de seguridad concreta.

Autoevaluación

1. ¿Qué lenguaje de localización usarías para seleccionar un botón por su texto visible “Enviar”?
2. ¿Por qué `Input Password` es preferible a `Input Text` para un campo de contraseña?
3. ¿Qué tipo de localizador (posicional vs. por atributo estable) es más resistente a cambios de diseño?

Respuestas: 1. XPath (`xpath://button[text()='Enviar']`) — CSS no soporta selección por texto de forma nativa. 2. Porque oculta el valor real en `log.html`, evitando exponer la contraseña en un artefacto compartible. 3. Por atributo estable (`id`, `data-testid`).

6.3 Sincronización: Wait Until, timeouts y estabilidad

Objetivos de la lección

- Explicar por qué `Sleep` es una mala práctica en automatización web.
- Aplicar la familia de keywords `Wait Until` para sincronización robusta.
- Identificar la contaminación de sesión entre tests como causa de inestabilidad.

¿Por qué importa?

La inestabilidad por sincronización (*flaky tests*) es, probablemente, la causa más frecuente de desconfianza hacia una suite de automatización web: si una suite falla intermitentemente sin que el sistema realmente tenga un defecto, el equipo deja de confiar en sus resultados y empieza a ignorarlos — el peor desenlace posible para una suite de pruebas.

Conceptos clave

El problema: contenido cargado de forma asíncrona

El problema más frecuente en automatización web es la **inestabilidad por sincronización**: el script intenta interactuar con un elemento antes de que la página termine de cargarlo, generalmente porque hay contenido cargado de forma asíncrona (AJAX, JavaScript) después de la carga inicial del HTML.

Por qué `Sleep` es la solución equivocada

La solución correcta **nunca** es `Sleep` con un tiempo fijo:

```
# Antipatrón – evitar
Click Button    Guardar
Sleep          3s
Element Should Be Visible    css:#confirmacion
```

Este enfoque tiene dos problemas simultáneos: **ralentiza la suite** cuando el elemento aparece en 200ms (esperaste 3 segundos innecesarios), y **sigue fallando** cuando el sistema tarda más de 3 segundos en un momento de carga alta — el peor de ambos mundos.

La familia `Wait Until`: espera activa con timeout

```
Click Button    Guardar
Wait Until Element Is Visible    css:#confirmacion    timeout=10s
```

La familia `Wait Until *` (`Wait Until Element Is Visible`, `Wait Until Location Contains`, `Wait Until Element Is Not Visible`, `Wait Until Keyword Succeeds`) espera **activamente** hasta que se cumple una condición — verificando repetidamente en intervalos cortos — con un **timeout máximo configurable**, fallando explícitamente solo si se agota ese tiempo. Esto hace que la suite sea tan rápida como pueda serlo el sistema real, sin sacrificar estabilidad: si el elemento aparece en 200ms, la espera termina en 200ms; si tarda 8 segundos, la espera termina en 8 segundos (siempre que el timeout configurado sea mayor).

Una causa de inestabilidad menos obvia: contaminación de sesión

Hay una causa de inestabilidad que no tiene que ver con tiempos de carga, sino con **compartir una sesión de navegador entre tests independientes**. Si un test deja una sesión autenticada o un estado residual (cookies, alertas abiertas) y el siguiente test reutiliza el mismo navegador, puede comportarse de forma impredecible — no porque el sistema bajo prueba tenga un defecto, sino porque el segundo test arrancó desde un estado que no controló.

Lección real de este curso: durante el desarrollo de las prácticas de este capítulo, se encontró exactamente este problema: una suite que usaba `Suite Setup` para abrir un único navegador compartido entre tres test cases mostraba campos de formulario vacíos de forma intermitente en el segundo y tercer test, después de que el primero completara un login exitoso. La causa no era un problema de espera (los `Wait Until` estaban correctamente configurados) — era que el segundo test heredaba un navegador con una sesión autenticada de un login anterior, y la combinación de navegación y formularios no se comportaba de forma predecible. La solución fue cambiar de `Suite Setup/Suite Teardown` (un navegador compartido) a `Test Setup/Test Teardown` (un navegador nuevo por test), garantizando que cada test arrancara desde un estado limpio. Este es un ejemplo real, no hipotético, de por qué la independencia de tests importa incluso cuando “ahorra tiempo” compartir recursos.

Ejemplo comentado

*** Settings ***

Library	SeleniumLibrary	
Test Setup	Open Browser	<code>\${URL_LOGIN}</code> headlesschrome
Test Teardown	Close All Browsers	

*** Test Cases ***

Iniciar Sesión Con Espera Explícita

```
Go To      ${URL_LOGIN}
Input Text      id:username      tomsmith
Input Password  id:password      SuperSecretPassword!
Click Button    css:button[type='submit']
# Espera activa, no un tiempo fijo arbitrario
Wait Until Location Contains  /secure      timeout=10s
Wait Until Element Is Visible  css:#flash      timeout=10s
```

`Test Setup/Test Teardown` (en vez de `Suite Setup/Suite Teardown`) garantiza que cada test arranca con un navegador completamente nuevo, sin estado heredado del test anterior.

Tabla de referencia rápida

Estrategia de espera	¿Recomendada?	Razón
Sleep con tiempo fijo	No	Ni rápida cuando no se necesita, ni confiable cuando se necesita más
Wait Until Element Is Visible	Sí	Espera activa con timeout configurable
Wait Until Location Contains	Sí	Espera a que la URL cambie (navegación completada)
Navegador compartido entre tests (Suite Setup)	Con cautela	Riesgo de contaminación de sesión entre tests

Errores comunes

- **Usar Sleep “para estar seguro”** — incluso como solución temporal, tiende a quedarse permanentemente en el código y a acumularse en suites grandes, ralentizándolas innecesariamente.
- **Compartir un navegador entre tests que no son verdaderamente independientes entre sí** — la causa raíz real documentada en este mismo curso.
- **Configurar un timeout demasiado corto** “para que la suite sea más rápida” — produce fallos intermitentes exactamente en los momentos de carga más lenta del sistema, que es precisamente cuando más necesitas que la prueba sea confiable.

Puntos clave

- Sleep con tiempo fijo es antipatrón: ni rápido cuando no se necesita, ni confiable cuando se necesita más tiempo.
- Wait Until * espera activamente con un timeout, balanceando velocidad y estabilidad.
- Compartir navegador entre tests independientes es una causa real de tests inestables — usa Test Setup/Teardown por test salvo que el estado compartido sea una decisión deliberada y documentada.

Autoevaluación

1. ¿Por qué Sleep con tiempo fijo es problemático incluso si “funciona la mayoría de las veces”?
2. ¿Qué cambio de Setup/Teardown resolvió el problema real de contaminación de sesión descrito en esta lección?
3. ¿Qué keyword usarías para esperar a que la URL cambie tras un envío de formulario?

Respuestas: 1. Porque es lento cuando no se necesita y poco confiable cuando se necesita más tiempo del estimado — no se adapta al comportamiento real del sistema.

2. Cambiar de Suite Setup/Suite Teardown (navegador compartido) a Test Setup/Test Teardown (navegador nuevo por test). 3. Wait Until Location Contains.

6.4 Evidencias: screenshots, alertas, frames e iframes

Objetivos de la lección

- Aplicar captura de pantalla manual y automática.
- Manejar alertas nativas de JavaScript con Handle Alert.
- Explicar por qué un frame requiere cambio explícito de contexto.

¿Por qué importa?

La capacidad de **demostrar** qué ocurrió durante una ejecución es tan importante como la ejecución misma — especialmente al diagnosticar un fallo que ocurrió en un pipeline de CI/CD, donde nadie estaba mirando la pantalla en tiempo real.

Conceptos clave

Capturas de pantalla: manual vs. automática en fallo

Capture Page Screenshot toma una captura manual en cualquier punto del test. Combinada con Run Keyword If Test Failed, captura **solo cuando el test falla**:

```
[Teardown]    Run Keywords
...           Run Keyword If Test Failed    Capture Page Screenshot
...           AND    Close All Browsers
```

Este patrón evita generar cientos de capturas innecesarias en una suite grande donde la mayoría de los tests pasa. También es válido — y a veces preferible — capturar en **cada** test, no solo en fallo, cuando se necesita evidencia completa para una auditoría o un reporte de regresión exhaustivo; la decisión depende del propósito de la suite, no de una regla universal.

Alertas de JavaScript: por qué necesitan una keyword especial

Las **alertas de JavaScript** (`alert()`, `confirm()`, `prompt()`) **no son parte del árbol DOM** — son ventanas nativas del navegador, renderizadas por el sistema operativo o el motor del navegador, fuera del documento HTML. Por eso SeleniumLibrary necesita una keyword especial, `Handle Alert`, para interactuar con ellas:

```
Click Element    css:button[onclick='jsAlert()']
Handle Alert     action=ACCEPT
```

Intentar interactuar con una alerta usando `Click Element` (como si fuera un elemento normal del DOM) produce un error, porque Selenium no puede “ver” la alerta como parte de la página — `Handle Alert` opera a un nivel distinto, el del navegador mismo, no el del documento.

Tipo de diálogo	Acciones disponibles
<code>alert()</code>	Solo ACCEPT (un botón OK)
<code>confirm()</code>	ACCEPT (OK) o DISMISS (Cancelar)
<code>prompt()</code>	ACCEPT con texto de entrada, o DISMISS

Frames e iframes: cambio explícito de contexto

Los **frames** e **iframes** representan documentos HTML **independientes**, embebidos dentro de la página principal — técnicamente, cada uno tiene su propio árbol DOM. Selenium **no puede interactuar** con elementos dentro de un frame sin que el script cambie explícitamente de contexto hacia ese frame primero, usando `Select Frame`; de lo contrario, el elemento “no existe” desde la perspectiva del documento principal (un error `NoSuchElementException`), aunque sea perfectamente visible en pantalla para un humano. Para volver al documento principal después de trabajar dentro de un frame, se usa `Unselect Frame`.

Ejemplo comentado

*** Test Cases ***

Aceptar Una Alerta Simple De JavaScript

```

    Open Browser      https://the-internet.herokuapp.com/javascript_alerts
headlesschrome
    Click Element     css:button[onclick='jsAlert()']
    Handle Alert      action=ACCEPT
    Element Should Contain  id:result      You successfully clicked an alert
    Close Browser

```

Interactuar Con Un Elemento Dentro De Un Iframe

```

    Open Browser      https://the-internet.herokuapp.com/iframe      headlesschrome
    # Sin Select Frame, el siguiente paso fallaría con NoSuchElementException
    Select Frame      css:#mce_0_ifr
    Input Text        css:#tinymce      Texto escrito dentro del iframe
    Unselect Frame
    Close Browser

```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué keyword maneja una alerta de JavaScript?	Handle Alert
¿Por qué no se puede usar Click Element en una alerta?	No es parte del DOM; es una ventana nativa del navegador
¿Qué keyword cambia el contexto hacia un iframe?	Select Frame
¿Qué error ocurre si interactúas con un elemento de un iframe sin cambiar de contexto?	NoSuchElementException (u equivalente)

Errores comunes

- **Intentar usar Click Element sobre una alerta de JavaScript** — falla porque la alerta no es parte del DOM.
- **Olvidar Unselect Frame después de trabajar dentro de un iframe** — los pasos siguientes del test pueden fallar inesperadamente al buscar elementos que están en el documento principal, no en el frame.
- **Capturar pantalla solo en fallo en una suite que necesita evidencia de auditoría completa** — revisar primero el propósito real de la suite antes de aplicar la regla por defecto.

Puntos clave

- Capturar pantalla en fallo (no en cada test) es la estrategia más común para evitar ruido; capturar siempre es válido para auditorías completas.
- Las alertas de JavaScript requieren Handle Alert, porque no son parte del DOM — son ventanas nativas del navegador.
- Un frame/iframe requiere cambio explícito de contexto (Select Frame/Unselect Frame) antes y después de interactuar con su contenido.

Autoevaluación

1. ¿Por qué una alerta de JavaScript no se puede manejar con Click Element?
2. ¿Qué keyword usarías para interactuar con un campo de texto que está dentro de un iframe?
3. ¿Cuándo conviene capturar pantalla en cada test, no solo en fallo?

Respuestas: 1. Porque no es parte del árbol DOM — es una ventana nativa del navegador, fuera del alcance normal de Selenium sin Handle Alert. 2. Primero Select Frame hacia ese iframe, luego la keyword de interacción normal (Input Text). 3. Cuando se necesita evidencia completa para una auditoría o reporte de regresión exhaustivo, no solo diagnóstico de fallos.

6.5 Patrón Page Object en RF: capa de keywords por página y mantenibilidad

Objetivos de la lección

- Describir el patrón Page Object aplicado a Robot Framework.
- Explicar su beneficio principal de mantenibilidad.
- Aplicar el patrón estructurando localizadores y keywords en un Resource por página.

¿Por qué importa?

Sin este patrón, un cambio de interfaz (un selector que cambia, un campo que se mueve) obliga a buscar y corregir el mismo selector repetido en docenas de test cases — un costo de mantenimiento que crece linealmente con el tamaño de la suite.

Conceptos clave

Qué es el patrón Page Object

El patrón **Page Object** propone que cada pantalla de la aplicación tenga su propia representación en código: un archivo Resource dedicado, con los localizadores como variables y las keywords de interacción de esa pantalla. Es, en esencia, una aplicación del mismo principio de separación por capas visto en BDD (Capítulo 4) y en el diseño de keywords reutilizables (Capítulo 5): el test case habla en lenguaje de negocio, el Page Object traduce ese lenguaje a interacciones concretas con el DOM.

Las dos capas dentro de un Page Object

Dentro de un mismo archivo Resource de Page Object, conviene distinguir dos niveles:

- **Variables de localizadores:** declaradas en `*** Variables ***`, agrupan todos los selectores de esa página en un solo lugar visible.
- **Keywords de interacción:** en `*** Keywords ***`, usan esas variables y exponen acciones con nombres de negocio (Ingresar Credenciales, NO Escribir En Dos Campos Y Hacer Clic).

```
# resources/LoginPage.resource
*** Variables ***
${CAMPO_USUARIO}      id:username
${CAMPO_PASSWORD}     id:password
${BOTON_INGRESAR}     css:button[type='submit']

*** Keywords ***
Ingresar Credenciales
    [Arguments]    ${usuario}    ${password}
    Input Text      ${CAMPO_USUARIO}    ${usuario}
    Input Password  ${CAMPO_PASSWORD}    ${password}
```

Hacer Clic En Ingresar

Click Button `${BOTON_INGRESAR}`

El beneficio principal: mantenibilidad

El beneficio principal es la **mantenibilidad**: cuando la interfaz cambia (un selector, una estructura de página), el ajuste se hace en **un solo lugar** — el Page Object correspondiente — sin tocar ningún test case que lo use. Sin este patrón, un cambio de UI obligaría a buscar y corregir el mismo selector repetido en docenas de test cases, con el riesgo real de dejar alguno sin actualizar.

Cómo se ve un test case que usa un Page Object

*** Settings ***

Resource ../resources/LoginPage.resource

*** Test Cases ***

Iniciar Sesión Exitosamente Usando El Page Object

Abrir Pagina De Login

Ingresar Credenciales tomsmith SuperSecretPassword!

Hacer Clic En Ingresar

Verificar Mensaje Flash Contiene You logged into a secure area

Observa que **ningún selector CSS aparece en el test case** — todo vive dentro del Page Object. Esta es, en la práctica, la señal de que el patrón se aplicó correctamente: si encuentras un `css:` o `xpath:` dentro de un archivo de test cases, es una fuga de abstracción que debería moverse al Page Object correspondiente.

Ejemplo comentado

resources/AlertsPage.resource — Page Object de una segunda pantalla

*** Settings ***

Library SeleniumLibrary

*** Variables ***

`${URL_ALERTAS}` https://the-internet.herokuapp.com/javascript_alerts

`${BOTON_JS_ALERT}` css:button[onclick='jsAlert()']

`${RESULTADO_ALERTA}` id:result

*** Keywords ***

Abrir Pagina De Alertas

Go To `${URL_ALERTAS}`

Wait Until Element Is Visible `${BOTON_JS_ALERT}` timeout=10s

Disparar Alerta Simple Y Aceptar

Click Element `${BOTON_JS_ALERT}`

Handle Alert action=ACCEPT

Verificar Resultado Contiene

[Arguments] `${texto_esperado}`

Element Should Contain `${RESULTADO_ALERTA}` `${texto_esperado}`

Cada pantalla de la aplicación (login, alertas, y cualquier otra) tiene su propio Resource — un proyecto con 10 pantallas tendría 10 Page Objects, cada uno autocontenido.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Dónde viven los selectores en el patrón Page Object?	En el Resource de esa página, como variables
¿Qué beneficio principal aporta el patrón?	Mantenibilidad — el cambio de UI se corrige en un solo lugar
¿Es correcto encontrar un <code>css:</code> directamente en un archivo de test cases?	No, es una fuga de abstracción del patrón
¿Cuántos Page Objects necesita un proyecto con 10 pantallas?	10, uno por pantalla

Errores comunes

- **Dejar un selector CSS/XPath directamente en el test case**, “por simplicidad” en un caso puntual — rompe la mantenibilidad que el patrón busca garantizar de forma consistente.
- **Crear un único Page Object gigante para toda la aplicación** en vez de uno por pantalla — pierde la cohesión y la facilidad de localizar qué archivo modificar ante un cambio de UI específico.
- **Nombrar las keywords del Page Object con el mecanismo técnico** (`Click En Boton Con Id Submit`) en vez de la intención de negocio (`Hacer Clic En Ingresar`) — mezcla la capa de negocio que el test case espera con detalles técnicos.

Puntos clave

- Page Object = localizadores + keywords de una pantalla, encapsulados en un Resource dedicado a esa pantalla.
- El test case solo conoce keywords de negocio del Page Object — nunca selectores directamente.
- Un cambio de interfaz se corrige en un solo lugar (el Page Object de esa pantalla), no en cada test case que la usa.

Autoevaluación

1. Si encuentras un selector CSS directamente dentro de un archivo de test cases, ¿qué patrón se está violando?
2. ¿Cuántos archivos Page Object necesitarías para un proyecto con login, dashboard y configuración (3 pantallas)?
3. ¿Cuál es el beneficio principal del patrón Page Object?

Respuestas: 1. El patrón Page Object — el selector debería vivir en el Resource de esa pantalla, no en el test case. 2. Tres, uno por pantalla. 3. Mantenibilidad: un cambio de interfaz se corrige en un solo lugar.

Resumen del capítulo

SeleniumLibrary conecta Robot Framework con el navegador vía Selenium WebDriver; Selenium Manager (4.6+) elimina la gestión manual de drivers. CSS y XPath son los dos lenguajes de localización, cada uno con sus keywords de interacción correspondientes según el tipo de control HTML. La sincronización correcta usa `Wait Until *`, nunca `Sleep` fijo, y cada test debería manejar su propia sesión de navegador (Test Setup/Teardown) para evitar contaminación de estado — una lección real, no hipotética, encontrada durante el desarrollo de este curso. Las capturas de pantalla y el manejo de alertas/frames dan evidencia y cobertura completa, cada uno con su propia keyword especializada (`Handle Alert`, `Select Frame`) porque no son parte del flujo normal del DOM. El patrón Page Object aplica separación por capas a pantallas web, concentrando el impacto de cambios de UI en un solo lugar por pantalla.

Referencias bibliográficas

- SeleniumLibrary: <https://robotframework.org/SeleniumLibrary/SeleniumLibrary.html>
- Selenium Manager: https://www.selenium.dev/documentation/selenium_manager/
- Patrón Page Object: https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/

Capítulo 7 — Automatización de APIs con RequestsLibrary

Información general

Las APIs REST son hoy el medio de comunicación predominante entre sistemas, y probarlas directamente (sin pasar por una interfaz web) es más rápido y más estable que la automatización de UI. Este capítulo cubre `RequestsLibrary`, la librería estándar de comunidad para pruebas de API en Robot Framework, con ejemplos de código completos basados en peticiones HTTP reales.

Lecciones de este capítulo:

- 7.1 — REST/HTTP: métodos, códigos de estado, headers y payloads
 - 7.2 — Autenticación: Bearer tokens, sesiones persistentes y datos dinámicos
 - 7.3 — Validación JSON: sintaxis extendida, aserciones de esquema y contract testing básico
 - 7.4 — Diseño de suites API: estructura smoke vs. regresión y organización por endpoint
-

7.1 REST/HTTP: métodos, códigos de estado, headers y payloads

Objetivos de la lección

- Describir los métodos HTTP principales y su semántica.
- Interpretar los rangos de códigos de estado HTTP.
- Identificar el rol de headers y payload en una petición.

¿Por qué importa?

Probar una API sin entender la semántica de HTTP lleva a tests que validan el código de estado correcto por casualidad, no por diseño — y a no saber qué método usar cuando el escenario de negocio no especifica explícitamente “haz un GET” o “haz un POST”.

Conceptos clave

Métodos HTTP y su semántica

REST organiza los recursos de un sistema como URLs, operadas con verbos HTTP estándar:

Método	Semántica	¿Tiene efectos secundarios?	¿Es idempotente?
GET	Leer un recurso	No	Sí (repetirlo no cambia el resultado)
POST	Crear un recurso nuevo	Sí	No (repetirlo crea otro recurso)
PUT	Reemplazar un recurso completo	Sí	Sí (repetirlo deja el mismo estado)
DELETE	Eliminar un recurso	Sí	Sí (eliminar dos veces el mismo recurso da el mismo resultado final)

La **idempotencia** es un concepto que aparece en preguntas de certificación: una operación idempotente produce el mismo resultado final sin importar cuántas veces se repita — GET y PUT son idempotentes, POST no lo es (cada llamada exitosa crea un recurso adicional).

Códigos de estado: los rangos que importan

Cada respuesta incluye un **código de estado** de tres dígitos, agrupado en rangos con significado:

Rango	Significado	Ejemplo común
2xx	Éxito	200 OK, 201 Created
4xx	Error causado por el cliente	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Error del servidor	500 Internal Server Error, 503 Service Unavailable

Distinguir 4xx de 5xx importa para el diagnóstico: un 4xx indica que la petición estaba mal formada o no autorizada (responsabilidad del cliente, en este caso de tu test); un 5xx indica que el servidor falló al procesar una petición válida (responsabilidad del sistema bajo prueba).

Headers y payload

Los **headers** llevan metadatos de la petición o respuesta: Content-Type indica el formato del cuerpo (application/json es el más común en APIs modernas), Authorization lleva

las credenciales (lección 7.2). El **payload** (cuerpo de la petición) en POST/PUT típicamente es JSON — un objeto estructurado que el servidor espera en un formato específico.

Ejemplo comentado

*** Settings ***

Library RequestsLibrary

Suite Setup Create Session api https://postman-echo.com verify=True

*** Test Cases ***

Demostrar Los Cuatro Métodos HTTP Principales

GET: leer, sin efectos secundarios

`${respuesta_get}= GET On Session api /get`

Should Be Equal As Numbers `${respuesta_get.status_code}` 200

POST: crear, con payload JSON

`&{payload}= Create Dictionary cliente=Ana Pérez`

`${respuesta_post}= POST On Session api /post json=${payload}`

Should Be Equal As Numbers `${respuesta_post.status_code}` 200

Should Be Equal `${respuesta_post.json()}[json][cliente]` Ana Pérez

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué rango de código indica error del servidor?	5xx
¿Qué método es idempotente pero tiene efectos secundarios?	PUT
¿Qué método NO es idempotente?	POST
¿Qué header indica el formato del cuerpo de la petición?	Content-Type

Errores comunes

- **Asumir que POST es idempotente** — cada llamada exitosa típicamente crea un nuevo recurso; repetir la misma petición POST puede crear duplicados.
- **Confundir un 4xx con un 5xx** al diagnosticar un fallo — el primero apunta a la petición (posiblemente un error en el test), el segundo apunta al servidor.
- **Omitir Content-Type al enviar un payload manualmente** (sin usar el parámetro `json=` que lo configura automáticamente) — el servidor puede interpretar el cuerpo incorrectamente.

Autoevaluación

1. ¿Es GET idempotente? ¿Y POST?
2. ¿Qué rango de código HTTP indica un error causado por el cliente?

3. ¿Qué header configura automáticamente RequestsLibrary cuando usas el parámetro `json=?`

Respuestas: 1. GET sí es idempotente; POST no lo es. 2. 4xx. 3. Content-Type: application/json.

7.2 Autenticación: Bearer tokens, sesiones persistentes y datos dinámicos

Objetivos de la lección

- Explicar el patrón de autenticación Bearer.
- Usar sesiones persistentes para evitar repetir configuración en cada petición.
- Encadenar datos dinámicos entre peticiones.

¿Por qué importa?

Casi ninguna API de producción es completamente abierta — entender cómo se transmite la autenticación, y cómo mantener una sesión configurada una sola vez para toda la suite, es indispensable para cualquier suite de API realista.

Conceptos clave

El esquema Bearer Token

El esquema **Bearer Token** (basado en RFC 6750, parte del estándar OAuth 2.0) transmite la credencial en el header `Authorization: Bearer <token>` — quien “porta” (*bears*) el token tiene acceso, sin necesidad de reenviar usuario/contraseña en cada petición. RequestsLibrary lo implementa simplemente agregando ese header a la petición; **no hay una keyword especial** de “login Bearer” — es responsabilidad del proyecto construir ese header correctamente, típicamente con `Create Dictionary` y pasándolo como `headers=`.

```
&{headers}= Create Dictionary Authorization=Bearer ${TOKEN}
${respuesta}= GET On Session api /recurso-protegido
headers=&{headers}
```

Create Session: sesiones HTTP persistentes

`Create Session` crea una sesión HTTP **persistente**: conserva headers, cookies y configuración TLS entre peticiones dentro de la misma suite, evitando repetir la URL base y configuración en cada llamada.

```
Suite Setup Create Session api https://api.miempresa.com verify=True
```

Una vez creada, cualquier `GET On Session`/`POST On Session` que referencie el alias `api` reutiliza esa configuración — incluyendo, si se configuró así, headers de autenticación que aplican a **todas** las peticiones de la sesión, sin tener que repetirlos en cada llamada individual.

Encadenar datos dinámicos entre peticiones

Una sesión persistente también permite **encadenar datos dinámicos** entre peticiones — por ejemplo, un id devuelto por un POST (creación de un recurso) reutilizado en el GET posterior que lo consulta:

*** Test Cases ***

Crear Un Recurso Y Luego Consultarlo Por Su ID Generado

```
&{payload}= Create Dictionary nombre=Plan Premium
${respuesta_creacion}= POST On Session api /planes
json=${payload}
${id_generado}= Set Variable ${respuesta_creacion.json()[id]}

${respuesta_consulta}= GET On Session api /planes/${id_generado}
Should Be Equal As Numbers ${respuesta_consulta.status_code} 200
```

Este patrón es exactamente el que necesitarás en la Sesión 8 (RPA), donde un proceso completo encadena varias llamadas API usando el resultado de una como entrada de la siguiente.

Ejemplo comentado

*** Settings ***

```
Library RequestsLibrary
Suite Setup Create Session api https://postman-echo.com verify=True
```

*** Variables ***

```
${TOKEN_DEMO} token-demo-12345
```

*** Test Cases ***

Autenticar Con Bearer Token Y Validar Que El Servidor Lo Recibió

```
[Documentation] postman-echo.com/headers refleja en la respuesta
... todos los headers recibidos – incluido Authorization
... – ideal para validar el patrón Bearer sin depender
... de un servicio de autenticación real.
&{headers}= Create Dictionary Authorization=Bearer ${TOKEN_DEMO}
${respuesta}= GET On Session api /headers headers=&{headers}
Should Be Equal As Numbers ${respuesta.status_code} 200
Should Be Equal ${respuesta.json()[headers][authorization]} Bearer
${TOKEN_DEMO}
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿En qué header se transmite el Bearer Token?	Authorization, con el formato Bearer <token>
¿Qué keyword crea una sesión persistente?	Create Session
¿Qué permite encadenar entre un POST y un GET posterior?	Una variable que captura el dato dinámico (ej. un id) de la respuesta del POST
¿RFC en el que se basa Bearer Token?	RFC 6750

Errores comunes

- **Repetir Create Session en cada test case** en vez de usarla una sola vez en Suite Setup — pierde el beneficio de persistencia y configuración compartida.
- **Olvidar verify=True al crear la sesión** — genera advertencias de seguridad y, en proyectos reales, deja sin verificar el certificado TLS del servidor.
- **No capturar el id generado por un POST en una variable**, y en su lugar intentar adivinarlo o hardcodearlo — frágil, porque el id real puede cambiar entre ejecuciones.

Autoevaluación

1. ¿En qué formato exacto se transmite un Bearer Token en el header Authorization?
2. ¿Qué keyword evita repetir la URL base y configuración en cada petición de una suite?
3. ¿Cómo encadenarías el id generado por un POST para usarlo en un GET posterior?

Respuestas: 1. Authorization: Bearer <token>. 2. Create Session, usada típicamente en Suite Setup. 3. Capturando `${respuesta.json()[id]}` en una variable después del POST, y usándola en la URL del GET siguiente.

7.3 Validación JSON: sintaxis extendida, aserciones de esquema y contract testing básico

Objetivos de la lección

- Validar valores específicos dentro de una respuesta JSON.
- Describir el concepto de contract testing.
- Diferenciar validación de valor de validación de estructura.

¿Por qué importa?

Validar solo el código de estado (200 OK) de una respuesta no garantiza que el contenido sea correcto — un servidor puede responder 200 con un cuerpo vacío o con datos incorrectos. La validación real de una API necesita mirar dentro del cuerpo de la respuesta.

Conceptos clave

Acceso a valores anidados con sintaxis extendida

Una vez que tienes la respuesta como diccionario Python (`respuesta.json()`), Robot Framework permite acceder a cualquier valor anidado con la **sintaxis extendida** `${respuesta.json()[clave][subclave]}` — suficiente para la mayoría de las validaciones de una suite de pruebas, sin necesitar ninguna librería adicional:

```
${respuesta}=    GET On Session    api    /clientes/123
Should Be Equal    ${respuesta.json()[nombre]}    Ana Pérez
Should Be Equal    ${respuesta.json()[direccion][ciudad]}    Guatemala
```

JSONPath para consultas más complejas

Para estructuras muy complejas (listas dentro de listas, búsqueda condicional dentro de un arreglo) o consultas dinámicas, existe `JSONLibrary` (instalada con `pip install robotframework-jsonlibrary`), con soporte de **JSONPath** — un lenguaje de consulta para JSON, análogo a XPath para XML. Para el alcance práctico de este curso, la sintaxis extendida de Robot Framework cubre la gran mayoría de los casos; JSONPath se vuelve valioso cuando necesitas filtrar dentro de listas grandes (`$.items[?(@.precio > 100)]`).

Validación de valor vs. validación de estructura

Estas son dos preguntas distintas que conviene no confundir:

- **Validación de valor:** “¿el campo estado es exactamente 'ACTIVO'?” — lo que has hecho hasta ahora con `Should Be Equal`.
- **Validación de estructura:** “¿la respuesta tiene un campo estado de tipo string, y un campo id de tipo número, sin campos inesperados?” — independiente de cuál sea el valor específico.

Contract testing

El **contract testing** valida que la *estructura* de una respuesta (qué campos existen, de qué tipo) se mantiene estable entre versiones de una API — detectando cambios incompatibles (un campo que desaparece, un tipo que cambia de string a número) antes de que lleguen a producción y rompan a los sistemas que consumen esa API. Es un complemento, no un reemplazo, de las validaciones de valor específico: una API puede pasar todas las validaciones de valor de hoy y, sin embargo, romper silenciosamente a un cliente si cambia su estructura sin que nadie lo detecte a tiempo.

Ejemplo comentado

```
*** Test Cases ***
```

```
Validar Valor Específico Dentro De Una Respuesta JSON
```

```
    ${respuesta}=    GET On Session    api    /clientes/123
    Should Be Equal As Numbers    ${respuesta.status_code}    200
    Should Be Equal    ${respuesta.json()[nombre]}    Ana Pérez
    Should Be Equal    ${respuesta.json()[plan][nombre]}    Premium
```

Validar Que Un Campo Esperado Existe (Estructura, No Valor)

[Documentation] Validación de contrato básica: el campo debe existir,
... independientemente de cuál sea su valor exacto hoy.
`${respuesta}= GET On Session api /clientes/123`
Dictionary Should Contain Key `${respuesta.json()}` fecha_alta

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué sintaxis accede a un valor anidado sin librerías adicionales?	<code>\${respuesta.json()}[clave][subclave]</code>
¿Qué librería agrega soporte de JSON-Path?	JSONLibrary
¿Qué valida el contract testing?	La estructura de la respuesta, no solo valores puntuales
¿Es lo mismo validar un valor que validar una estructura?	No, son dos preguntas distintas y complementarias

Errores comunes

- **Validar solo el código de estado y asumir que el contenido es correcto** — un 200 OK no garantiza que el cuerpo tenga los datos esperados.
- **Instalar JSONLibrary para casos simples** que la sintaxis extendida de Robot Framework ya resuelve — agrega una dependencia innecesaria.
- **Confundir contract testing con validación de valor** — son complementarios, no intercambiables; un proyecto serio necesita ambos tipos de validación.

Autoevaluación

1. ¿Qué sintaxis usarías para acceder al campo ciudad dentro de direccion dentro de la respuesta JSON?
2. ¿Qué detecta el contract testing que una validación de valor específico no detecta necesariamente?
3. ¿Cuándo conviene usar JSONLibrary/JSONPath en vez de la sintaxis extendida de Robot Framework?

Respuestas: 1. `${respuesta.json()}[direccion][ciudad]`. 2. Cambios de estructura (un campo que desaparece o cambia de tipo) que rompen a los consumidores de la API. 3. Para consultas complejas o condicionales dentro de listas grandes, donde la sintaxis extendida se vuelve impráctica.

7.4 Diseño de suites API: estructura smoke vs. regresión y organización por endpoint

Objetivos de la lección

- Diferenciar una suite smoke de una suite de regresión.
- Aplicar `expected_status` para probar deliberadamente respuestas de error.
- Organizar una suite de API por endpoint.

¿Por qué importa?

Sin esta distinción, un equipo termina ejecutando la suite completa de API en cada commit (lento, costoso) o, en el otro extremo, evita ejecutar la suite completa con la frecuencia que debería (cobertura insuficiente antes de un release).

Conceptos clave

Smoke vs. regresión

Una suite **smoke** agrupa pocos casos críticos, pensados para ejecutarse en **cada** despliegue, completándose en segundos o pocos minutos — su objetivo es detectar rápidamente si algo básico se rompió (el servicio no responde, la autenticación falla por completo). Una suite de **regresión** cubre exhaustivamente comportamientos, casos límite y escenarios negativos, reservada para ejecutarse antes de un release, donde el tiempo de ejecución es menos crítico que la cobertura.

	Suite smoke	Suite de regresión
Cantidad de casos	Pocos, los más críticos	Muchos, cobertura exhaustiva
Frecuencia de ejecución	Cada despliegue/commit	Antes de un release
Tiempo objetivo	Segundos a pocos minutos	Minutos a horas, según el proyecto
Incluye casos negativos extensos	Rara vez	Sí, sistemáticamente

`expected_status=any`: pruebas negativas sin interrupciones

Por defecto, `RequestsLibrary` lanza una excepción ante cualquier respuesta `4xx/5xx` — apropiado cuando siempre esperas éxito, pero un obstáculo cuando **el objetivo del test es justamente validar un error esperado** (por ejemplo, que un endpoint rechace correctamente datos inválidos con un `400`). El parámetro `expected_status=any` desactiva esa excepción automática, dejando el código de estado disponible para que el test lo compare explícitamente:

```
{respuesta}= GET On Session api /status/404 expected_status=any
Should Be Equal As Numbers ${respuesta.status_code} 404
```

Sin `expected_status=any`, esta línea lanzaría una excepción al recibir el `404`, deteniendo el test antes de que el `Should Be Equal As Numbers` pudiera ejecutarse — exactamente lo opuesto de lo que el test necesita validar.

Organización por endpoint

Para proyectos con muchos endpoints, organizar la suite **por recurso** (un archivo por endpoint o grupo de endpoints relacionados) facilita la navegación:

```
tests/
├── smoke/
│   └── salud_servicio.robot
└── regression/
    ├── clientes/
    │   └── crud_clientes.robot
    └── planes/
        └── crud_planes.robot
```

Ejemplo comentado

```
*** Settings ***
Library           RequestsLibrary
Library           DataDriver      ${CURDIR}/../data/casos_api.csv      dialect=excel
encoding=utf_8
Suite Setup       Create Session   api      https://postman-echo.com
verify=True
Test Template     Verificar Status Code Del Endpoint
```

*** Test Cases ***

Caso De Ejemplo De API

*** Keywords ***

Verificar Status Code Del Endpoint

```
[Documentation]    expected_status=any evita que RequestsLibrary lance
...               una excepción automática ante un 4xx/5xx – así
...               podemos validar casos negativos sin que el GET
...               "falle" antes de llegar al assert.
```

```
[Arguments]       ${endpoint}    ${status_esperado}
${respuesta}=     GET On Session  api      ${endpoint}    expected_status=any
Should Be Equal As Numbers    ${respuesta.status_code}    ${status_esperado}
```

*** Test Cases ***
\${endpoint}, \${status_esperado}, [Tags]

Servicio responde correctamente (200), /status/200, 200, "smoke, positivo"

Endpoint inexistente devuelve 404, /status/404, 404, "regresion, negativo"

Error interno del servidor devuelve 500, /status/500, 500, "regresion, negativo"

Combinando DataDriver (Capítulo 5) con esta suite, un solo archivo CSV cubre casos positivos y negativos de forma escalable, sin un test case por cada código de estado.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué tipo de suite se ejecuta en cada despliegue?	Smoke
¿Qué parámetro evita la excepción automática ante un 4xx/5xx?	<code>expected_status=any</code>
¿Qué suite tiene mayor cobertura de casos negativos?	Regresión

Errores comunes

- **Ejecutar la suite de regresión completa en cada commit** — consume tiempo de pipeline innecesario; reserva eso para antes de un release.
- **Olvidar `expected_status=any` al probar deliberadamente un error** — el test falla con una excepción no controlada antes de llegar a la validación real que querías hacer.
- **No tener ninguna suite smoke**, ejecutando siempre todo o nada — pierde la posibilidad de una verificación rápida tras cada despliegue.

Puntos clave

- Suite smoke: pocos casos críticos, cada despliegue, rápida. Suite regresión: cobertura completa, antes de release.
- `expected_status=any` es necesario para probar deliberadamente respuestas 4xx/5xx.
- Organizar por endpoint (un archivo por recurso de la API) facilita la navegación en suites grandes.

Autoevaluación

1. ¿Qué tipo de suite (smoke o regresión) ejecutarías inmediatamente después de cada despliegue a producción?
2. Sin `expected_status=any`, ¿qué pasa al hacer un GET a un endpoint que devuelve 404?
3. ¿Cómo organizarías una suite de API con 20 endpoints distintos?

Respuestas: 1. Smoke. 2. `RequestsLibrary` lanza una excepción automática, deteniendo el test antes de cualquier validación explícita. 3. Por endpoint o grupo de endpoints relacionados, en archivos separados dentro de carpetas `smoke/` y `regression/`.

Resumen del capítulo

REST organiza recursos como URLs operadas con verbos HTTP (con semántica e idempotencia distintas para cada uno); los códigos de estado comunican éxito o tipo de error. El patrón Bearer Token transmite la credencial en un header, y `Create Session` mantiene configuración persistente entre peticiones, habilitando el encadenamiento de datos dinámicos entre un POST y un GET posterior. La sintaxis extendida de Robot

Framework valida JSON sin librerías adicionales para la mayoría de los casos; el contract testing complementa la validación de valores con la validación de estructura. Las suites de API se organizan en smoke (rápida, crítica, cada despliegue) y regresión (completa, antes de release), y `expected_status=any` habilita pruebas negativas deliberadas sin que RequestsLibrary interrumpa la ejecución con una excepción automática.

Referencias bibliográficas

- RequestsLibrary: <https://marketsquare.github.io/robotframework-requests/doc/RequestsLibrary.html>
- RFC 6750 (Bearer Token Usage): <https://datatracker.ietf.org/doc/html/rfc6750>
- postman-echo.com (servicio de eco para pruebas): <https://learning.postman.com/docs/reference/developer-resources/echo-api/>

Capítulo 8 — Robot Framework Aplicado a RPA

Información general

Este capítulo retoma el segundo dominio de Robot Framework presentado en la Sesión 1: la RPA. Aquí dejas de verificar comportamiento (PASS/FAIL) y empiezas a ejecutar procesos de negocio completos, con manejo de archivos, orquestación de varias capas tecnológicas, y trazabilidad operativa — con ejemplos de código completos en cada lección.

Lecciones de este capítulo:

- 8.1 — Automatización de archivos: CSV, Excel (openpyxl), PDF y gestión de carpetas
 - 8.2 — Orquestación de procesos: entrada/salida, parametrización y variables de proceso
 - 8.3 — Integración E2E: web + API + archivos en un único flujo RPA
 - 8.4 — Manejo de errores en RPA: retries, timeouts, control de fallas
 - 8.5 — Logging, trazabilidad y estandarización en RPA
-

8.1 Automatización de archivos: CSV, Excel, PDF y gestión de carpetas

Objetivos de la lección

- Identificar las librerías Python para trabajar con cada formato de archivo.
- Aplicar lectura/escritura de Excel desde una librería propia.
- Generar un reporte en PDF desde datos procesados.

¿Por qué importa?

En procesos RPA, los archivos son el medio de entrada/salida más común: facturas, reportes, listados de clientes. Saber qué librería usar para cada formato — y por qué no necesitas licencias de Office para ninguna de ellas — es la base técnica de cualquier proceso RPA.

Conceptos clave

Las librerías por formato

Formato	Librería	Nativa de Python
CSV	csv (módulo estándar)	Sí, sin instalar nada
Excel (.xlsx)	openpyxl	No, pip install openpyxl
PDF (generar)	fpdf2	No, pip install fpdf2
PDF (leer texto existente)	pdfplumber	No, pip install pdfplumber

Ninguna de estas librerías requiere Microsoft Office instalado — todas trabajan directamente con el formato de archivo, sin depender de una aplicación externa.

El patrón recomendado: Python puro + pytest + keyword

El patrón recomendado, ya familiar desde el Capítulo 5, es separar la lógica en funciones Python puras, probarlas con pytest antes de tocar Robot Framework, y exponerlas como keywords:

```
@dataclass
class Cliente:
    nombre: str
    plan: str
    consumo_gb: float
    costo_base: float
    costo_total: float = 0.0

def leer_clientes_excel(ruta_excel: str | Path) -> list[Cliente]:
    """Lee la hoja 'Clientes' de un Excel y devuelve una lista de Cliente."""
    ruta = Path(ruta_excel)
    if not ruta.is_file():
        raise FileNotFoundError(f"No existe el archivo: {ruta}")
    wb = openpyxl.load_workbook(ruta, data_only=True)
    hoja = wb["Clientes"]
    clientes = []
    for fila in hoja.iter_rows(min_row=2, values_only=True):
        if fila[0] is None:
            continue
        nombre, plan, consumo_gb, costo_base = fila
        clientes.append(Cliente(nombre=nombre, plan=plan, consumo_gb=consumo_gb,
                                costo_base=costo_base))
    return clientes
```

La orquestación del proceso vive en .robot, la lógica de cada paso vive en Python — la misma división de responsabilidades que conociste en el Capítulo 5, aplicada ahora al dominio RPA.

Generar un reporte PDF desde datos procesados

```
def generar_reporte_pdf(clientes: list[Cliente], ruta_salida: str | Path) -> Path:
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Helvetica", "B", 14)
    pdf.cell(0, 10, "Reporte de Clientes", new_x="LMARGIN", new_y="NEXT")
    for cliente in clientes:
        pdf.cell(0, 8, f"{cliente.nombre} - {cliente.plan} - Q{cliente.costos_total:.2f}", new_x="LMARGIN", new_y="NEXT")
    ruta = Path(ruta_salida)
    ruta.parent.mkdir(parents=True, exist_ok=True)
    pdf.output(str(ruta))
    return ruta
```

fpdf2 construye el documento de forma incremental (página, fuente, celdas de texto) y lo guarda con `output()` — un modelo de construcción imperativo, distinto al de lectura de Excel, pero igualmente simple de usar desde una keyword propia.

Ejemplo comentado

```
*** Settings ***
Library    ../scripts/procesar_clientes.py
Library    OperatingSystem

*** Test Cases ***
Leer Excel Y Generar Reporte PDF
    @{clientes}=    Leer Clientes Excel    ${CURDIR}/../data/clientes.xlsx
    @{transformados}=    Transformar Clientes    ${clientes}
    ${ruta}=    Generar Reporte Pdf    ${transformados}    ${CURDIR}/../reportes/
reporte.pdf
    File Should Exist    ${ruta}
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué librería lee/escribe archivos .xlsx sin Office?	openpyxl
¿Qué librería genera PDFs desde Python?	fpdf2
¿El módulo csv requiere instalación adicional?	No, es parte de la librería estándar de Python

Errores comunes

- **Intentar leer un .xlsx con el módulo csv** — son formatos binarios distintos; .xlsx requiere openpyxl (o una librería equivalente).

- **Olvidar** `ruta.parent.mkdir(parents=True, exist_ok=True)` antes de guardar un archivo de salida — falla si la carpeta destino no existe todavía.
- **No probar la función de lectura/transformación con pytest antes de integrarla** — dificulta saber si un fallo viene de la lógica o de la integración con Robot Framework.

Autoevaluación

1. ¿Qué librería usarías para leer un archivo Excel `.xlsx` desde Python?
2. ¿Es necesario tener Microsoft Office instalado para usar `openpyxl` o `fpdf2`?
3. ¿Dónde debería vivir la lógica de transformación de datos: en el archivo `.robot` o en una función Python separada?

Respuestas: 1. `openpyxl`. 2. No, ninguna de las dos librerías lo requiere. 3. En una función Python separada, probada con `pytest` antes de exponerse como keyword — el patrón recomendado desde el Capítulo 5.

8.2 Orquestación de procesos: entrada/salida, parametrización y variables de proceso

Objetivos de la lección

- Describir el concepto de entrada/salida en un proceso RPA.
- Aplicar logging explícito por etapa para mejorar la diagnosticabilidad.
- Usar expresiones Python embebidas para cálculos simples dentro de un valor.

¿Por qué importa?

Un proceso RPA que corre sin nadie observando la pantalla necesita poder explicarse a sí mismo después de los hechos — si algo sale mal a las 2am, el log es la única fuente de información disponible para diagnosticar qué ocurrió.

Conceptos clave

Entradas y salidas explícitas

Un proceso RPA modela explícitamente sus **entradas** (un archivo, una variable de línea de comandos, una llamada a una API) y sus **salidas** (un archivo generado, un registro creado, una notificación enviada). A diferencia de un test, que termina en un veredicto PASS/FAIL, un proceso RPA termina en un **resultado operativo** concreto y verificable — un archivo que existe y tiene contenido, no solo una aserción lógica.

Por qué el logging explícito por etapa es indispensable

El **logging explícito por etapa** es una práctica fundamental en RPA:

```
Log    ETAPA 1/3: Leyendo clientes desde ${ARCHIVO_ENTRADA}
@{clientes}=    Leer Clientes Excel    ${ARCHIVO_ENTRADA}
Log    ETAPA 1/3 completada: ${len($clientes)} clientes leídos
```



```
Log    ETAPA 2/3: Transformando datos
@{clientes_transformados}=    Transformar Clientes    ${clientes}
Log    ETAPA 2/3 completada
```

Cuando el proceso corre desatendido (sin nadie observando la pantalla), el log es la única ventana hacia lo que realmente ocurrió. Sin logging granular, diagnosticar un fallo en producción significa adivinar en qué punto del proceso algo salió mal — con logging por etapa, el último mensaje visible en el log indica exactamente dónde se detuvo el proceso.

Expresiones Python embebidas: `${...}`

La sintaxis `${expresión}` (doble llave) le indica a Robot Framework que evalúe lo que está dentro como **código Python real**, usando el prefijo `$nombre` (sin llaves) para referenciar variables de Robot Framework dentro de esa expresión:

```
Log    ETAPA 1/3 completada: ${len($clientes)} clientes leídos
```

Aquí, `$clientes` se resuelve a la lista real de objetos Python, y `len(...)` es la función estándar de Python aplicada directamente — sin necesitar una keyword adicional solo para contar elementos. Esta sintaxis es útil para cálculos simples de una línea; para lógica más compleja, sigue siendo preferible extraerla a una keyword Python propia (Capítulo 5).

Variables de proceso: estado del robot

En procesos RPA más largos, las “variables de proceso” almacenan el estado acumulado del robot a lo largo de su ejecución: contadores de registros procesados, una lista de errores encontrados para reportar al final, un identificador único de la ejecución (`ID_EJECUCION`) generado al inicio para correlacionar todos los logs de esa corrida específica.

Ejemplo comentado

```
*** Test Cases ***
```

Proceso RPA Completo Con Logging Por Etapa

```
[Documentation]    Cada etapa se registra explícitamente – si el
...               proceso falla, el último mensaje visible indica
...               dónde se detuvo.
Log    ETAPA 1/3: Leyendo clientes desde ${ARCHIVO_ENTRADA}
@{clientes}=    Leer Clientes Excel    ${ARCHIVO_ENTRADA}
Log    ETAPA 1/3 completada: ${len($clientes)} clientes leídos

Log    ETAPA 2/3: Transformando datos (IVA + clasificación de consumo)
@{clientes_transformados}=    Transformar Clientes    ${clientes}
Log    ETAPA 2/3 completada

Log    ETAPA 3/3: Generando reporte PDF en ${ARCHIVO_SALIDA}
${ruta_generada}=    Generar Reporte Pdf    ${clientes_transformados}
${ARCHIVO_SALIDA}
Log    ETAPA 3/3 completada: reporte en ${ruta_generada}
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué produce un proceso RPA al terminar, a diferencia de un test?	Un resultado operativo, no un veredicto PASS/FAIL
¿Qué sintaxis evalúa código Python embebido dentro de un valor de RF?	<code>{{expresión}}</code> (doble llave)
¿Por qué es indispensable el logging por etapa en RPA?	Porque el proceso corre desatendido; el log es la única evidencia disponible

Errores comunes

- **No registrar el inicio/fin de cada etapa**, dejando un único log genérico al final del proceso — dificulta saber en qué etapa ocurrió un fallo.
- **Usar `{{...}}` para lógica compleja de varias líneas** — la sintaxis está pensada para expresiones simples; lógica más elaborada debería ser una keyword Python propia.
- **No definir un identificador único de ejecución** en procesos que corren repetidamente — dificulta correlacionar los logs de una corrida específica cuando varias ejecuciones se superponen en el tiempo.

Autoevaluación

1. ¿Qué diferencia el resultado final de un proceso RPA del resultado final de un test?
2. ¿Qué sintaxis usarías para contar cuántos elementos tiene una lista, embebiendo Python directamente en un Log?
3. ¿Por qué el logging por etapa es más importante en RPA que en una suite de pruebas típica?

Respuestas: 1. Un proceso RPA produce un resultado operativo (archivo, registro); un test produce un veredicto PASS/FAIL. 2. `{{len($variable)}}`. 3. Porque un proceso RPA suele correr desatendido (sin nadie observando), y el log es la única fuente de diagnóstico disponible después de los hechos.

8.3 Integración E2E: web + API + archivos en un único flujo RPA

Objetivos de la lección

- Diseñar un proceso que combine varias tecnologías en capas secuenciales.
- Aplicar el patrón de traspaso de datos entre capas.
- Mantener cada capa enfocada en su librería correspondiente.

¿Por qué importa?

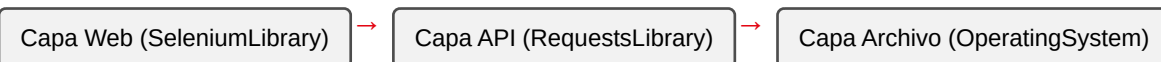
Los procesos RPA del mundo real casi nunca usan una sola tecnología — y sin un patrón claro de cómo combinarlas, el código tiende a mezclar responsabilidades de forma que

cualquier cambio en una tecnología obliga a tocar código que debería ser independiente de ella.

Conceptos clave

Por qué los flujos reales combinan tecnologías

Es común que un proceso RPA navegue un sitio web para extraer un dato (capa web), lo envíe a un sistema externo vía API (capa API), y registre el resultado en un archivo local para trazabilidad (capa archivo) — las tres tecnologías que ya conoces de capítulos anteriores, combinadas en un solo flujo secuencial dentro de un mismo proceso.



El traspaso de datos entre capas

El traspaso de datos entre capas se hace con **variables de Robot Framework**: el valor extraído en la capa web se pasa como argumento a la keyword de la capa API, y el resultado de esa llamada (junto con el dato original) se pasa a la capa de archivo:

```
${apellido_extraido}=    Get Text    ${SELECTOR_NOMBRE}
# El dato de la capa web se pasa a la capa API
&{payload}=    Create Dictionary    apellido_cliente=${apellido_extraido}
${respuesta}=    POST On Session    api    /post    json=${payload}
# El dato original Y el resultado de la API se pasan a la capa archivo
Append To File    ${ARCHIVO_SALIDA}    ${apellido_extraido},
${respuesta.status_code}\n
```

Cada capa mantiene su librería correspondiente

Cada capa debería seguir usando su librería correspondiente (SeleniumLibrary para la capa web, RequestsLibrary para la capa API, OperatingSystem para la capa de archivos) **sin mezclar responsabilidades** — la capa API no debería conocer detalles de cómo se extrajo el dato de la página web, solo debería recibirlo como un argumento ya resuelto.

Ejemplo comentado

```
*** Test Cases ***
Proceso RPA E2E: Extraer, Registrar Vía API Y Archivar
    [Teardown]    Close All Browsers

    Log    CAPA WEB: extrayendo dato de la tabla
    Open Browser    ${URL_TABLA}    headlesschrome
    Wait Until Element Is Visible    ${SELECTOR_NOMBRE}    timeout=10s
    ${apellido_extraido}=    Get Text    ${SELECTOR_NOMBRE}

    Log    CAPA API: registrando el dato extraído
    &{payload}=    Create Dictionary    apellido_cliente=${apellido_extraido}
    origen=rpa-e2e
    ${respuesta}=    POST On Session    api    /post    json=${payload}
```

```
Log      CAPA ARCHIVO: registrando el resultado localmente
Create Directory    ${CURDIR}/../salida
Append To File      ${ARCHIVO_SALIDA}    ${apellido_extraido},
${respuesta.status_code}\n
```

Observa cómo cada capa registra su inicio con Log (la práctica de la lección 8.2), y cómo el dato fluye linealmente de una capa a la siguiente sin que ninguna capa necesite conocer el detalle interno de la anterior.

Tabla de referencia rápida

Capa	Librería	Recibe de la capa anterior
Web	SeleniumLibrary	(es la primera capa)
API	RequestsLibrary	El dato extraído de la web, como argumento
Archivo	OperatingSystem	El dato original + el resultado de la API

Errores comunes

- **Llamar a SeleniumLibrary desde dentro de una keyword de la capa API** — mezcla responsabilidades; cada capa debería recibir datos ya resueltos, no ir a buscarlos ella misma a otra capa.
- **No capturar el dato extraído en una variable antes de pasarlo a la siguiente capa** — sin esa captura explícita, no hay forma de encadenar el flujo.
- **Omitir el Teardown que cierra el navegador** cuando el flujo incluye una capa web — deja procesos de navegador abiertos si el flujo falla a mitad de camino.

Autoevaluación

1. ¿Qué librería correspondería a la “capa archivo” de un flujo E2E?
2. ¿Cómo se traspasa un dato de la capa web a la capa API en Robot Framework?
3. ¿Debería la capa API conocer el selector CSS usado para extraer un dato en la capa web?

Respuestas: 1. OperatingSystem. 2. Capturándolo en una variable de Robot Framework y pasándolo como argumento a la keyword de la capa API. 3. No — cada capa debe mantenerse enfocada en su tecnología, recibiendo datos ya resueltos de la capa anterior.

8.4 Manejo de errores en RPA: retries, timeouts y control de fallas

Objetivos de la lección

- Aplicar Run Keyword And Return Status para validaciones booleanas simples.
- Diseñar un checklist de calidad explícito por capa.
- Diferenciar este patrón de los tres del Capítulo 3.

¿Por qué importa?

En un proceso de varias capas, un fallo en una sola capa no debería producir un mensaje de error genérico al final — debe ser **localizable**: ¿falló la extracción web, la llamada API, o la escritura del archivo? Sin esa localización, diagnosticar un fallo en producción se vuelve mucho más lento.

Conceptos clave

Por qué un assert ciego no es suficiente en un proceso de varias capas

Un único `Should Be True` al final de un proceso de tres capas, validando todo de golpe, no te dice **cuál** de las tres capas falló — solo que “algo” no salió bien. Para procesos RPA, donde diagnosticar rápido importa operativamente, esto es insuficiente.

El checklist de calidad explícito

Un **checklist de calidad** explícito valida cada capa por separado y registra el resultado (PASS/FAIL) de cada una en el log **antes** de fallar el test, resolviendo el problema de diagnosticabilidad:

*** Keywords ***

Ejecutar Checklist De Calidad

```
[Arguments]    ${apellido}    ${status_api}    ${ruta_archivo}
${checklist}=  Create List
```

```
${item_web}=   Set Variable If    ""${apellido}"" != ""    PASS    FAIL
Append To List    ${checklist}    Capa Web (dato no vacío): ${item_web}
```

```
${item_api}=   Set Variable If    ${status_api} == 200    PASS    FAIL
Append To List    ${checklist}    Capa API (status 200): ${item_api}
```

```
${archivo_existe}=  Run Keyword And Return Status    File Should Exist
${ruta_archivo}
```

```
${item_archivo}=   Set Variable If    ${archivo_existe}    PASS    FAIL
Append To List    ${checklist}    Capa Archivo (existe): ${item_archivo}
```

```
FOR    ${item}    IN    @${checklist}
    Log    CHECKLIST: ${item}
END
```

```
Should Not Contain    ${checklist}    Capa Web (dato no vacío): FAIL
Should Not Contain    ${checklist}    Capa API (status 200): FAIL
Should Not Contain    ${checklist}    Capa Archivo (existe): FAIL
```

Si la capa API falla, el log mostrará explícitamente `CHECKLIST: Capa API (status 200): FAIL` junto a las otras dos capas en `PASS` — localizando el problema sin ambigüedad, antes incluso de que el test termine de fallar.

Run Keyword And Return Status

Run Keyword And Return Status (similar a Run Keyword And Ignore Error, visto en el Capítulo 3, pero devolviendo **solo** el booleano de éxito/fallo, sin el mensaje) es útil precisamente para construir este tipo de checklist, donde solo necesitas saber si una verificación pasó o no, sin propagar la excepción original ni necesitar el mensaje de error detallado.

Keyword (Capítulo 3)	Qué devuelve	Uso típico en checklist
Run Keyword And Ignore Error	(estado, mensaje)	Cuando necesitas el mensaje de error
Run Keyword And Return Status	Solo True/False	Cuando solo necesitas el booleano, como en un checklist

Ejemplo comentado

Ver el bloque de Ejecutar Checklist De Calidad arriba — es el ejemplo central de esta lección, combinando FOR (Capítulo 3), Set Variable If, y Run Keyword And Return Status en un patrón reutilizable para cualquier proceso de varias capas.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué devuelve Run Keyword And Return Status?	Solo un booleano (True/False)
¿En qué se diferencia de Run Keyword And Ignore Error?	Esta última también devuelve el mensaje de error
¿Qué resuelve un checklist de calidad explícito que un assert único no resuelve?	Localiza en qué capa específica ocurrió el fallo

Errores comunes

- **Usar un único Should Be True para validar las tres capas de golpe** — no localiza cuál capa específicamente falló.
- **Usar Run Keyword And Ignore Error cuando solo necesitas el booleano** — agrega un valor de retorno (el mensaje) que no se usa, complicando innecesariamente el código.
- **No registrar el checklist en el log antes de fallar** — si el test falla antes de que el FOR con los Log se ejecute, se pierde la visibilidad que el patrón busca dar.

Puntos clave

- Un checklist de calidad explícito localiza el fallo en la capa correspondiente, en vez de un mensaje de error genérico.
- Run Keyword And Return Status devuelve solo el booleano de éxito/fallo (sin mensaje), ideal para checklists donde no necesitas el detalle del error.

- Este patrón complementa, no reemplaza, los tres patrones de manejo de fallas del Capítulo 3.

Autoevaluación

1. ¿Qué problema de diagnóstico resuelve un checklist de calidad explícito frente a un assert único?
2. ¿Qué keyword devuelve solo un booleano, sin mensaje de error, ante el resultado de otra keyword?
3. ¿En qué se diferencia Run Keyword And Return Status de Run Keyword And Ignore Error?

Respuestas: 1. Localiza específicamente en qué capa ocurrió el fallo, en vez de un mensaje genérico de “algo falló”. 2. Run Keyword And Return Status. 3. Return Status devuelve solo el booleano; Ignore Error devuelve el booleano y además el mensaje de error.

8.5 Logging, trazabilidad y estandarización en RPA

Objetivos de la lección

- Explicar por qué la trazabilidad es crítica en procesos RPA empresariales.
- Describir el rol de `output.xml` como fuente de auditoría.
- Aplicar estandarización de formato de mensajes de log entre procesos.

¿Por qué importa?

A diferencia de una suite de pruebas (que se ejecuta y se descarta una vez interpretado el resultado), un proceso RPA empresarial a menudo necesita **evidencia auditable** de cada ejecución — qué se procesó, cuándo, con qué resultado — por requerimientos de cumplimiento o de auditoría interna.

Conceptos clave

`output.xml` como registro de auditoría

El mismo `output.xml` que Robot Framework genera para cualquier ejecución — ya familiar desde el Capítulo 1 — cumple este rol de auditoría: contiene el detalle completo de cada keyword ejecutada, con sus argumentos y mensajes de Log, sirviendo como registro completo **sin necesidad de construir un sistema de logging adicional** desde cero. Para procesos RPA con requisitos de auditoría formales, conservar el `output.xml` de cada ejecución (no solo los reportes HTML derivados) es la práctica recomendada, porque es la fuente de verdad completa.

Estandarizar el formato de los mensajes

Estandarizar el formato de los mensajes de log entre distintos procesos RPA de un mismo equipo facilita que cualquier persona del equipo pueda diagnosticar un proceso ajeno sin tener que descifrar una convención distinta cada vez:

ETAPA N/M: <descripción de qué está haciendo esta etapa>
ETAPA N/M completada: <resultado breve de esta etapa>
CHECKLIST: <nombre de la capa> (<qué se valida>): PASS/FAIL

Esta convención (la misma usada en las lecciones 8.2 y 8.4 de este capítulo) no es una regla impuesta por Robot Framework — es una decisión de diseño del equipo, pero su consistencia entre procesos distintos es lo que realmente aporta valor: un ingeniero que nunca vio el proceso A puede diagnosticarlo rápidamente si ya conoce la convención usada en el proceso B.

Procesar output.xml programáticamente

Para necesidades de auditoría más sofisticadas (dashboards, alertas automáticas basadas en resultados históricos), output.xml puede procesarse directamente con Python, usando la librería estándar xml.etree.ElementTree — la misma técnica que usaste en el Capítulo 1 para extraer métricas de una ejecución, aplicable también a procesos RPA, no solo a suites de pruebas.

Ejemplo comentado

```
"""Extrae métricas de un output.xml de un proceso RPA para un dashboard
de auditoría – la misma técnica vista en el Capítulo 1, Práctica 2."""
import xml.etree.ElementTree as ET

def extraer_metricas_auditoria(ruta_output_xml: str) -> dict:
    arbol = ET.parse(ruta_output_xml)
    nodo_total = arbol.find("./statistics/total/stat")
    return {
        "pasados": int(nodo_total.get("pass", 0)),
        "fallidos": int(nodo_total.get("fail", 0)),
        "timestamp_ejecucion": arbol.getroot().get("generated"),
    }
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué artefacto sirve como registro de auditoría completo?	output.xml
¿Es necesario un sistema de logging adicional para auditoría básica?	No, output.xml ya lo cubre
¿Qué librería de Python procesa output.xml programáticamente?	xml.etree.ElementTree (estándar, sin instalar nada)

Errores comunes

- **Borrar el output.xml de ejecuciones pasadas** cuando hay un requisito de auditoría formal — es la fuente de verdad completa; los reportes HTML son solo vistas derivadas.

- **Usar un formato de mensaje distinto en cada proceso RPA del equipo** — dificulta que cualquier persona del equipo diagnostique un proceso que no escribió.
- **Construir un sistema de logging externo desde cero** cuando `output.xml` ya cubre las necesidades básicas de auditoría — esfuerzo innecesario para el alcance real del problema.

Puntos clave

- `output.xml` sirve como registro de auditoría completo, sin necesidad de sistemas de logging adicionales para necesidades básicas.
- Estandarizar el formato de los mensajes de log entre procesos facilita el diagnóstico por cualquier miembro del equipo, no solo el autor original.
- `xml.etree.ElementTree` (librería estándar de Python) es suficiente para procesar `output.xml` programáticamente, sin instalar nada adicional.

Autoevaluación

1. ¿Qué artefacto de Robot Framework cumple el rol de registro de auditoría, sin construir nada adicional?
2. ¿Por qué estandarizar el formato de los mensajes de log entre procesos distintos del mismo equipo aporta valor?
3. ¿Qué librería de Python (sin instalación adicional) procesa `output.xml`?

Respuestas: 1. `output.xml`. 2. Porque permite que cualquier persona del equipo diagnostique un proceso ajeno sin aprender una convención nueva cada vez. 3. `xml.etree.ElementTree`.

Resumen del capítulo

Los procesos RPA automatizan archivos (CSV, Excel, PDF) con el mismo patrón Python+pytest+keyword del Capítulo 5, sin depender de Microsoft Office. Modelan entradas/salidas explícitas, con logging granular por etapa (incluyendo expresiones Python embebidas `${...}` para cálculos simples), indispensable cuando corren desatendidos. Los flujos E2E combinan capas (web, API, archivos) secuenciales, traspasando datos entre ellas por variables, cada capa enfocada en su librería correspondiente. Un checklist de calidad explícito, construido con `Run Keyword And Return Status`, localiza el fallo en la capa correspondiente — complementando, no reemplazando, los tres patrones de manejo de fallas del Capítulo 3. `output.xml` provee trazabilidad auditable sin sistemas adicionales, y estandarizar el formato de los mensajes de log entre procesos del mismo equipo acelera el diagnóstico colectivo.

Referencias bibliográficas

- `openpyxl`: <https://openpyxl.readthedocs.io/>

- fpdf2: <https://py-pdf.github.io/fpdf2/>
- Run Keyword And Return Status: <https://robotframework.org/robotframework/latest/libraries/BuiltIn.html#Run%20Keyword%20And%20Return%20Status>

Capítulo 9 — Ejecución Avanzada, Reporting y Preparación RFCP

Información general

Este capítulo de cierre consolida el dominio del CLI de Robot Framework, profundiza en el reporting (la base de cualquier *quality gate*), y conecta todo lo aprendido con la integración en pipelines de CI/CD — preparando explícitamente para la certificación RFCP con una visión integrada de las 9 sesiones del curso.

Lecciones de este capítulo:

- 9.1 — CLI avanzado: `--variable`, `--include/--exclude`, `--suite`, `--rerunfailed` y `pare-tags`
 - 9.2 — Reporting: análisis de `report.html`, `log.html`, métricas y `output.xml` con `rebot`
 - 9.3 — Integración CI/CD conceptual: pipelines GitHub Actions/Jenkins y quality gates
-

9.1 CLI avanzado: variables, filtros y paretags

Objetivos de la lección

- Aplicar las opciones de CLI más usadas en proyectos reales.
- Combinar tags con operadores lógicos (paretags).
- Reejecutar selectivamente los tests fallidos de una ejecución previa.

¿Por qué importa?

Dominar el CLI de Robot Framework es, en la práctica, tan importante como dominar la sintaxis `.robot` — un proyecto real rara vez ejecuta “toda la suite, siempre, igual”; necesita adaptarse a distintos contextos (un commit, un release, un diagnóstico puntual) sin tocar el código fuente.

Conceptos clave

El patrón general y su filosofía

El patrón general del CLI de Robot Framework es `robot [opciones] [fuente]`. **Ninguna opción modifica los archivos** `.robot` — toda la configuración vive en el comando de ejecución, lo que permite adaptar una misma suite a distintos entornos (staging, producción, distintos conjuntos de datos) sin tocar el código fuente ni mantener copias paralelas de la suite.

Las opciones más usadas

Opción	Qué hace	Ejemplo
<code>--variable NOMBRE:valor</code>	Injecta o sobrescribe una variable en tiempo de ejecución	<code>--variable BASE_URL:https:// staging.com</code>
<code>--include TAG / --exclude TAG</code>	Filtra por tag	<code>--include smoke</code>
<code>--suite NOMBRE</code>	Delimita la ejecución a una suite específica	<code>--suite Facturacion</code>
<code>--rerunfailed <output.xml></code>	Reejecuta solo los tests que fallaron en esa ejecución previa	<code>--rerunfailed reports/ output.xml</code>
<code>--outputdir <carpeta></code>	Define dónde se guardan los reportes	<code>--outputdir reports</code>

`--variable` tiene **precedencia sobre las variables definidas en el archivo** — recordando la jerarquía de precedencia completa vista en el Capítulo 2 (CLI > test/keyword > suite > Resource).

Paretags: combinación lógica de tags

Los **paretags** combinan condiciones de tags con operadores AND, OR, NOT — siempre **en mayúsculas y sin espacios** alrededor del operador:

```
# Solo tests que tienen AMBOS tags
robot --include smokeANDcritico tests/

# Tests que tienen AL MENOS UNO de los dos tags
robot --include criticoORregresion tests/

# Tests de regresión que NO sean básicos
robot --include regresion --exclude basico tests/
```

Esto permite expresar filtros precisos como “ejecuta los tests críticos de regresión, pero no los que están marcados como en progreso”, sin necesitar tags compuestos artificiales como `regresion-critico-no-wip`.

--rerunfailed: ahorro de tiempo en suites grandes

`--rerunfailed <output.xml>` reejecuta solo los tests que fallaron en una ejecución previa, ahorrando tiempo en suites grandes — particularmente valioso para distinguir un fallo real (que persiste en el rerun) de un fallo intermitente (*flaky*, que podría no reproducirse en el rerun).

Ejemplo comentado

```
# Ejecución completa, guardando el output con un nombre específico
robot --outputdir reports --output output1.xml tests/suite.robot
```

```
# Reejecutar SÓLO lo que falló en esa ejecución
```

```
robot --outputdir reports --rerunfailed reports/output1.xml --output output2.xml
tests/suite.robot
```

Si la suite original tenía 100 tests con 3 fallidos, el rerun ejecuta exactamente esos 3 — no los 100 — ahorrando el tiempo de los 97 que ya se sabe que pasaron.

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Modifica <code>--variable</code> el archivo <code>.robot</code> ?	No, ninguna opción de CLI lo hace
¿Cómo se escriben los operadores de paretags?	En mayúsculas, sin espacios (<code>smokeANDcritico</code>)
¿Qué reejecuta <code>--rerunfailed</code> ?	Solo los tests que fallaron en la ejecución indicada

Errores comunes

- **Escribir paretags con espacios** (`smoke AND critico`) — Robot Framework no lo reconoce como un operador lógico, lo trata como dos tags separados sin relación.
- **Asumir que `--variable` cambia el archivo fuente** — solo afecta esa ejecución específica; el archivo `.robot` permanece sin cambios.
- **Usar `--rerunfailed` sin haber guardado el `output.xml` de la ejecución original con `--output`** — sin ese archivo, no hay nada que reejecutar selectivamente.

Autoevaluación

1. ¿Cómo escribirías un filtro para ejecutar tests que sean `smoke` o `critico` (cualquiera de los dos)?
2. ¿Qué comando reejecuta solo los tests que fallaron en `reports/output1.xml`?
3. Verdadero o falso: `--variable BASE_URL:https://x.com` modifica permanentemente el valor de `${BASE_URL}` en el archivo `.robot`.

Respuestas: 1. `--include smokeORcritico`. 2. `robot --rerunfailed reports/output1.xml tests/`. 3. Falso — solo afecta esa ejecución, el archivo fuente no cambia.

9.2 Reporting: `report.html`, `log.html` y `rebot`

Objetivos de la lección

- Interpretar la información de cada artefacto de reporte.
- Combinar varios `output.xml` en un solo reporte con `rebot`.

- Aplicar `rebot --merge` al patrón ejecutar/reintentar/combinar.

¿Por qué importa?

Sin entender `rebot`, el patrón “ejecutar, reintentar fallidos, combinar resultados” (lección 9.1) se queda incompleto — tendrías dos reportes separados (la ejecución original y el rerun) sin una forma clara de presentar un resultado final unificado.

Conceptos clave

Los tres artefactos, repaso integrador

Robot Framework genera siempre tres artefactos, ya familiares desde el Capítulo 1: `output.xml` (los datos en bruto, la fuente de verdad), `log.html` (detalle de cada keyword ejecutada) y `report.html` (resumen ejecutivo con estadísticas, incluyendo el desglose “by tag” que viste en el Capítulo 5).

rebot: regenerar sin reejecutar

rebot es la herramienta que regenera `log.html/report.html` a partir de uno o varios `output.xml`, **sin necesidad de volver a ejecutar los tests**. Esto es posible porque `output.xml` contiene toda la información necesaria para reconstruir las vistas HTML — son completamente derivadas, no datos adicionales.

```
# Regenerar solo los reportes HTML a partir de un output.xml existente
rebot --outputdir reports output.xml
```

rebot --merge: el mecanismo formal detrás del patrón de reintento

El uso más valioso de `rebot` es `--merge`: combina varios archivos `output.xml` (por ejemplo, una ejecución original y su rerun de fallidos) en un solo reporte consolidado, usando el **resultado más reciente** de cada test que se repite entre los archivos:

```
rebot --outputdir reports --merge --output merged.xml reports/output1.xml
reports/output2.xml
```

Si un test falló en `output1.xml` pero pasó en el rerun (`output2.xml`), el reporte combinado lo muestra como PASS — el resultado más reciente “gana”. Este es el mecanismo formal detrás del patrón “ejecutar, reintentar fallidos, combinar” que practicaste en sesiones anteriores: no es solo una conveniencia de presentación, es la forma correcta de reportar un resultado final cuando hubo reintentos.

rebot también filtra

Además de combinar, `rebot` puede aplicar los mismos filtros de tags (`--include/--exclude`) sobre un `output.xml` ya existente, generando un reporte filtrado sin volver a ejecutar nada — útil para producir, por ejemplo, un reporte ejecutivo que solo muestre los resultados de smoke a partir de una ejecución completa que incluía mucho más.

Ejemplo comentado

1. Ejecución completa

```
robot --outputdir reports --output output1.xml tests/suite.robot
```

Resultado: 4 tests, 3 passed, 1 failed

2. Reintentar solo el fallido

```
robot --outputdir reports --rerunfailed reports/output1.xml --output output2.xml tests/suite.robot
```

3. Combinar ambos en un solo reporte final

```
rebot --outputdir reports --merge --output merged.xml reports/output1.xml reports/output2.xml
```

El reporte combinado refleja el resultado MÁS RECIENTE de cada test

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿rebot reejecuta los tests?	No, solo regenera/combina reportes desde output.xml existentes
¿Qué resultado “gana” al combinar con --merge?	El más reciente de cada test repetido entre los archivos
¿Puede rebot aplicar --include/--exclude?	Sí, sobre un output.xml ya existente

Errores comunes

- **Intentar usar rebot para “arreglar” un test que falló** — rebot solo regenera/combina reportes; no puede cambiar el resultado real de una ejecución, solo presentarlo de otra forma.
- **Olvidar --output al ejecutar la versión original**, dejando el nombre por defecto (output.xml), y luego sobrescribirlo accidentalmente con el rerun antes de poder combinarlos.
- **Asumir que --merge promedia o suma resultados** — no lo hace; toma el resultado más reciente de cada test, no un cálculo agregado.

Puntos clave

- output.xml es la fuente de verdad; log.html/report.html se derivan completamente de él.
- rebot regenera reportes sin reejecutar tests, y puede aplicar filtros de tags sobre un output.xml existente.
- rebot --merge combina varios output.xml, priorizando el resultado más reciente de cada test repetido — el mecanismo formal del patrón “ejecutar, reintentar, combinar”.

Autoevaluación

1. ¿Necesita rebot ejecutar los tests de nuevo para generar un reporte?

2. Si un test falló en la primera ejecución y pasó en el rerun, ¿qué muestra el reporte combinado con `--merge`?
3. ¿Puede rebot filtrar por tags sobre un `output.xml` ya existente, sin reejecutar nada?

Respuestas: 1. No, regenera/combina a partir de `output.xml` existentes. 2. PASS — el resultado más reciente de cada test repetido es el que se refleja. 3. Sí.

9.3 Integración CI/CD conceptual: pipelines y quality gates

Objetivos de la lección

- Describir las etapas típicas de un pipeline CI/CD con Robot Framework.
- Explicar el concepto de quality gate.
- Conectar el contenido del curso con la preparación para RFCP.

¿Por qué importa?

La automatización solo alcanza su máximo valor operativo cuando deja de ejecutarse manualmente y se integra en el flujo de trabajo normal del equipo — un pipeline que ejecuta pruebas automáticamente ante cada cambio de código, sin que nadie tenga que recordar hacerlo.

Conceptos clave

Las etapas típicas de un pipeline con Robot Framework

Un pipeline de CI/CD (GitHub Actions, Jenkins, GitLab CI, entre otros) ejecuta automáticamente una secuencia de pasos ante un evento (un `push`, un *pull request*):

1. Instalar dependencias (`pip install -r requirements.txt`, dentro de un entorno aislado — recordando el Capítulo 1).
2. Ejecutar la suite `smoke` (rápida) en cada commit.
3. Reservar la suite de regresión completa para antes de un release — el mismo patrón `smoke/regresión` que practicaste en el Capítulo 7.
4. Publicar los artefactos de reporte (`output.xml`, `log.html`, `report.html`) como evidencia de la ejecución.

Quality gate: la condición que decide si el pipeline avanza

Un **quality gate** es una condición mínima que debe cumplirse para que el pipeline continúe — por ejemplo, “0 tests fallidos” o “pass rate $\geq$ 95%”. El **exit code** de robot (0 si todo pasó, distinto de 0 si hubo fallos) es la señal técnica que cualquier pipeline usa para decidir si avanza o se detiene — la misma señal que viste por primera vez en el Capítulo 1, al ejecutar tu primera suite con un fallo intencional y observar que el comando terminaba con un código distinto de cero.

```
# Fragmento conceptual de un workflow de GitHub Actions
- name: Ejecutar suite smoke
```



```
run: robot --include smoke tests/
# Si robot termina con exit code != 0, este step falla,
# y el pipeline se detiene automáticamente (comportamiento por defecto).
```

Cómo este curso te preparó para RFCP

Este capítulo, junto con todo el curso, te deja preparado para la certificación **RFCP**: el examen evalúa exactamente el lenguaje y la plataforma central que cubrieron las 9 sesiones —

- Sintaxis y estructura de archivos (Capítulo 1).
- Variables, Resources, tags, Setup/Teardown (Capítulo 2).
- Control de flujo y manejo de fallas (Capítulo 3).
- BDD nativo y separación de capas (Capítulo 4).
- Data-driven testing y extensión con Python (Capítulo 5).
- (Las librerías de dominio — Selenium, Requests — de los Capítulos 6 y 7 son valiosas profesionalmente, pero **no entran en el examen RFCP**, como aprendiste en la lección 1.2.)
- Aplicación a RPA (Capítulo 8).
- Ejecución avanzada por CLI y reporting (este capítulo).

— sin depender de ninguna librería de dominio específico. Repasar las autoevaluaciones de cada lección de este manual, junto con el simulacro completo de la Práctica 18 de la guía práctica, es la preparación más directa para el examen real.

Ejemplo comentado

```
# Patrón típico de quality gate en un script de pipeline
robot --include smoke --outputdir results/ tests/
EXIT_CODE=$?

if [ $EXIT_CODE -ne 0 ]; then
    echo "Quality gate falló: hay tests smoke fallidos. Deteniendo el pipeline."
    exit 1
fi

echo "Quality gate superado. Continuando con el despliegue."
```

Tabla de referencia rápida

Pregunta del examen RFCP (estilo)	Respuesta
¿Qué suite se ejecuta típicamente en cada commit?	smoke
¿Qué señal usa un quality gate para decidir si el pipeline continúa?	El exit code de robot
¿Las librerías de dominio (Selenium, Requests) entran en el examen RFCP?	No

Errores comunes

- **Ejecutar la suite de regresión completa en cada commit dentro del pipeline** — vuelve el ciclo de desarrollo innecesariamente lento; reserva eso para antes de un release.
- **No revisar el exit code de robot en un script de pipeline personalizado** — sin verificarlo explícitamente, el pipeline puede continuar incluso si la suite falló.
- **Estudiar SeleniumLibrary/RequestsLibrary en profundidad para el examen RFCP** — son valiosas profesionalmente, pero no son el foco de esa certificación específica; repasa mejor los Capítulos 1-5, 8 y 9.

Puntos clave

- Un pipeline CI/CD típico ejecuta smoke en cada commit y regresión antes de release.
- Un quality gate usa el exit code de robot para decidir si el pipeline continúa o se detiene.
- La certificación RFCP evalúa precisamente el lenguaje y la plataforma central — los Capítulos 1 al 5, 8 y 9 de este manual (incluyendo `*** Tasks ***`, parte del lenguaje central), sin las librerías de dominio de los Capítulos 6-7 (Selenium, Requests).

Autoevaluación

1. ¿Qué suite (smoke o regresión) es más apropiada para ejecutar en cada commit de un pipeline?
2. ¿Qué usa un quality gate para decidir si el pipeline continúa?
3. Para preparar el examen RFCP específicamente, ¿qué capítulos de este manual son más relevantes: 1-5, 8 y 9, o 6-7?

Respuestas: 1. Smoke. 2. El exit code de robot (0 = todo pasó, distinto de 0 = hubo fallos). 3. Los Capítulos 1-5, 8 y 9 — cubren el lenguaje y la plataforma central, incluyendo `*** Tasks ***`/RPA; los Capítulos 6-7 cubren librerías de dominio específico (Selenium, Requests), fuera del alcance de RFCP.

Resumen del capítulo

El CLI de Robot Framework es completamente configurable sin tocar el código fuente: `--variable`, `--include/--exclude`, `--suite`, `--rerunfailed` y los paretags (siempre en mayúsculas, sin espacios) cubren la mayoría de las necesidades de ejecución avanzada. `robot` regenera y combina reportes sin reejecutar tests — `--merge` es la pieza formal detrás del patrón “ejecutar, reintentar, combinar”, priorizando siempre el resultado más reciente de cada test repetido. Un pipeline de CI/CD ejecuta smoke en cada commit y regresión antes de release, usando el exit code de robot como quality gate. La certificación RFCP evalúa precisamente el lenguaje y la plataforma central — los Capítulos 1 a 5, 8 y 9 de este manual (incluyendo RPA y `*** Tasks ***`, parte del lenguaje central), en contraste con las librerías de dominio específico (Selenium, Requests) de los Capítulos 6 y 7, que son valiosas profesionalmente pero no forman parte de ese examen.

Referencias bibliográficas

- Robot Framework User Guide — Configuring execution: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#configuring-execution>
- rebot: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#rebot>
- Robot Framework Foundation (certificación RFCEP): <https://robotframework.org/foundation/>